האוניברסיטה הפתוחה
המחלקה למתמטיקה ולמדעי המחשב

סקירת מודלים המבוססים **LMU** ושילובם בטכנולוגיות נוירומורפיות

עבודה מסכמת זו הוגשה כחלק מהדרישות לקבלת תואר
"מוסמך למדעים" .M.Sc במדעי המחשב
האוניברסיטה הפתוחה
המחלקה למתמטיקה ומדעי המחשב

על-ידי
גל בר לב

העבודה הוכנה בהדרכתה של ד"ר הדר כהן דוויק

פברואר 2025

תקציר

חישוב ועיבוד נתונים תלוי זמן הוא אתגר מרכזי בתחום למידת המכונה, בעיקר בשל הצורך בשימור מידע לאורך זמן תוך שמירה על יעילות חישובית. בעוד שמודלים כמו LSTM ו- GRU הציעו פתרונות חלקיים, הן סובלות ממגבלות כגון איבוד מידע הדרגתי, עומס חישובי גבוה ואי יכולת להתמודד עם תלות טווח ארוך. במקביל, טרנספורמרים הפכו למובילים בתחומים רבים, אך דרישות החישוב הגבוהות שלהם הופכות אותם לפחות אידיאליים עבור מערכות זמן אמת וחומרות חסכוניות באנרגיה. כדי להתמודד עם מגבלות אלו, פותח מודל Legendre Memory Unit - LMU, כחלופה חסכונית ויעילה, המותאמת במיוחד למערכות חישוב נוירומורפי, בהן חיסכון באנרגיה הוא קריטי. עבודה זו מנתחת את התפתחות ה- LMU, את יתרונותיו למול מודלים מודרניים, ואת יישומיו בתחומים מגוונים. תוך התמקדות בהיבטים נוירומורפים בניסיונם למצבם כאלטרנטיבה לטרנספורמרים. הממצאים מראים כי LMU מצטיין במשימות בהן נדרש שימור מידע ארוך טווח עם צריכת אנרגיה נמוכה, במיוחד במערכות מבוססות ספייקים וחומרות נוירומורפיות, ומספקים תובנות להמשך פיתוחו ושילובו בטכנולוגיות מתקדמות.

תוכן עניינים

1. מבוא 5
1.1 מטרת העבודה וחשיבותה 5
2. רקע תיאורטי 6
2.1 עיבוד נתונים תלוי זמן 6
2.1.1 מבנה רשתות נשנות והבעיות המרכזיות שלהן 7
2.1.2 התפתחויות קודמות: ConvLSTM, GRU, LSTM 8
2.2 הצגת הבעיה והמוטיבציה 12
2.2.1 מגבלות חישוביות של רשתות חוזרות 12
2.2.2 טרנספורמרים כפתרון חלקי 14
2.2.3 הצורך בגישה חישובית חדשה 15
2.3 רקע תיאורטי על חישוב נוירומורפי 17
2.3.1 יסודות החישוב הנוירומורפי והשראה ביולוגית 17
2.3.2 חישוב מבוסס אירועים (Event-Based Computation) 21
2.3.3 ההבדלים בין חישוב נוירומורפי לבין חישוב קלאסי 22
3. LMU 24
3.1 סקירה כללית של פתרונות חישוביים קיימים מבוססי LMU 24
3.1.1 פתרונות חישוביים קיימים 24
3.1.2 פתרונות חישוביים קיימים – היבט נוירומורפי 26
3.2 LMU – עקרונות פעולה ויישום 27
3.2.1 ארכיטקטורה בסיסית 27
3.2.2 HIPPO כמסגרת תיאורטית 31
3.2.3 SSM – ניהול דינמי של מידע בסדרות זמן 38
3.3 הרחבות ושיפורים למודלים מבוססי LMU 42
3.3.1 חישוב מקבילי של LMU 42
3.3.2 Mamba - שילוב עם SSM 47
3.3.3 השוואה בין HiPPO, FF-LMU ו-Mamba 52
4. LMU בעולם החישוב הניומורפי 56
4.1 שילוב LMU בחישוב נוירומורפי 56
4.1.1 התאמת הדנמיקה של LMU לארכיטקטורה מבוססת ספייקים 57
4.1.2 יתרונות ייחודיים של LMU בסביבה נוירומורפית 59
4.2 הרחבות ושיפורים ל-LMU בחישוב נוירומורפי 61
4.2.1 L2MU: יישום נוירומורפי טבעי של LMU באמצעות נוירוני LIF 61
4.2.2 LMUFormer 67
4.2.3 SpikMamba 73
5. יישומים מעשיים של LMU 79
5.1 עיבוד אותות ביולוגיים 79
5.2 זיהוי פעילות אנושית HAR 80
5.3 שילוב במערכות IOT ו-Edge Computing 80
5.4 פוטנציאל בעיבוד שפה טבעית NLP 81
6. אתגרים ומבט לעתיד 82

6.1 מגבלות מחקריות קיימות 82
6.1.1 קשיי אימון ורציפות בזמן 82
6.1.2 מגבלות בזיכרון וביכולת הכללה 83
6.1.3 פערי ביצועים במשימות גדולות 84
6.2 אתגרים בשילוב LMU עם ייצוג וחומרה נוירומופיים 85
6.2.1 ייצוג חישוב לינארי באמצעות ספייקים 85
6.2.2 מגבלות קישוריות ותזמון בחומרה נוירומורפית 85
6.3 כיווני מחקר ופיתוח עתידיים 86
6.3.1 הוספת קשב וארכיטקטורות היברידיות 86
6.3.2 גרסאות ספייקיות מתקדמות ומשולבות 87
6.3.3 התאמה למשימות בעולם האמתי 88
7. סיכום 89
9. ביבליוגרפיה : 90
10. נספחים : 92
Abstract 94
Table of Contents 95

רשימת איורים:

איור 1 : Long Short-Term Memory 9
איור 2 : Gated Recurrent Unit 10
איור 3 : מבנה פנימי של ConvLSTM 11
איור 4 : הפיכת תמונה ממימד D2 לטנזור D3 11
איור 5 : מנגון תשומת הלב בטרנספורמר 14
איור 6 : השוואה סכמטית בין מערכות העצבים הביולוגיות לבין מערכות חישוב מלאכותיות מבוססות רשתות עצביות מתקפיות 17
איור 7 : מבנה הנוירון 18
איור 8 : דחף עצבי בנוירון 19
איור 9 : מבנה המעגל החשמלי של נוירון מסוג LIF 20
איור 10 : ההתנהגות נוירון מסוג LIF כתגובה לקלט 20
איור 11 : ארכיטקטורת LMU 28
איור 12 : פולינומים של לג'נדר עבור d=12 30
איור 13 : שלבים במסגרת HiPPO 33
איור 14 : אינטגרציה של HiPPO בתוך מודל RNN בסיסי 34
איור 15 : הגדרת SSM 38
איור 16 : מודל SaShiMi מבוסס S4 לעיבוד אודיו וסינתזת גלי קול גולמיים 40
איור 17 : Selective SSM במודל Mamba 48
איור 18 : היררכית הזיכרון של GPU 49
איור 19 : גרף דיוק במבחן אורך טקסט עבור מודלים שונים 54
איור 20 : מבנה חיבורים של LMU המיושם בעזרת נוירונים מבוססי ספייקים 58
איור 21 : הקשר בין מספר הנוירונים במודל לבין שגיאת השחזור במימוש של LMU באמצעות Poisson neurons 60
איור 22 : ארכיטקטורת מודל L2MU 64
איור 23 :ארכיטקטורת LMUFormer - המבנה של בלוק הconvolutional patch Embedding, וניתוח ההשהיה של דגימות בקלט לאורך שכבות 69
איור 24 : המבנה הפנימי של בלוק Spiking LMU ואת פעולת הנוירון הבודד בתרשים הימני 70
איור 25 : שיטות לזיהוי HAR 73
איור 26 : ארכיטקטורת מודל SpikMamba 74

1. מבוא

בעשור האחרון, עיבוד נתונים תלויי זמן הפך למרכיב אינטגרלי במגוון רחב של יישומים, החל מניתוח אותות רפואיים ועד לחיזוי מגמות בשוק ההון. עם זאת משימה זו טומנת בחובה אתגרים ייחודיים, הנובעים בעיקר מהצורך לנהל זיכרון רציף וללכוד תלויות ארוכות טווח בין נקודות מידע המרוחקות זו מזו בסדרת נתונים.

אחד הפתרונות הבולטים בתחום היה פיתוח רשתות נשנות (RNN), המסוגלות לעבד רצפים תוך שמירה על מידע ממסגרות זמן קודמות. עם זאת, רשתות אלו סבלו ממגבלות מהותיות, ביניהן בעיות הגרדיאנט הנעלם, שהגבילה את היכולת להתמודד עם תלות ארוכת טווח. כדי להתגבר על מגבלות אלו, פותחו מודלים מתקדמים כמו LSTM ו- GRU [1], אשר שילבו מנגנוני שערים מורכבים שמטרתם לווסת את זרימת המידע ולשמר רלוונטיות לאורך צעדי זמן רבים. למרות ההתקדמות המשמעותית שהושגה באמצעות ארכיטקטורות אלו, הן עדיין מתמודדות עם אתגרים, כגון דרישות חישוביות גבוהות וקושי בעיבוד מקבילי. בנוסף, מודלים אלו אינם מותאמים באופן אופטימלי לעיבוד נתונים המאופיינים במבנים מרחביים מורכבים. כמענה לאתגר זה הוצע מודל ה- ConvLSTM המשלב שכבות קונבולוציה עם מנגנוני LSTM במטרה ללכוד תלויות מרחביות זמניות [4]. המודל מציג יתרון בשמירה על תלות גלובלית ומקומית בנתונים מרחביים, אך מורכבותו החישובית הגבוהה והדרישות האנרגטיות הגבילו את השימוש בו בסביבות עתירות נתונים.

בעשור האחרון, תחום מודלים מבוססי מרחב-מצב (SSM) עבר התפתחות משמעותית, והציע גישה אלטרנטיבית לייצוג ולעיבוד נתונים סדרתיים [10]. מודלים אלו, ביניהם LMU, משתמשים בטרנספורמציות לינאריות על מנת למפות את סדרת הנתונים למצב חבוי, ובכך לאפשר לכידה יעילה של תלויות ארוכות טווח. מחקר זה הציב את LMU כבסיס ראשוני ומבטיח לפתרון בעיות חישוביות מורכבות תוך שמירה על יעילות אנרגטית.

1.1 מטרת העבודה וחשיבותה

מטרת העבודה היא לסקור את התפתחותם, השוואתם, שיפורם ויישומיהם של מודלים מבוססי LMU, תוך התמקדות בהיבטים נוירומורפים ובניסיונות למצבם כאלטרנטיבה לטרנספורמרים. העבודה מתייחסת למודלים מרכזיים כמו Mamba [9] ו-LMU קלאסי [3], ומציגה את תרומתם לחדשנות בתחומים מגוונים, תוך שמירה על יעילות חישובית ואנרגטית. המטרה היא להדגיש כיצד LMU ומודלים מתקדמים המבוססים על אותם עקרונות, יכולים לשמש כבסיס טכנולוגי לפתרון בעיות חישוביות מורכבות ולעיצוב טכנולוגיות חישוב עתידיות. חשיבות העבודה טמונה בחקר הפוטנציאל של מודלים מבוססי LMU כפתרון לאתגרים קריטיים בחישוביות תלוית זמן ובתחום הנוירומורפיה. LMU מציע חלופה יעילה אנרגטית ומדויקת למודלים קיימים כמו טרנספורמרים, שנתקלים במגבלות סקלביליות ודרישות חישוביות כבדות. העבודה מדגישה את תרומת LMU לחדשנות במערכות נוירומורפיות, תחום הנמצא בשלבי פיתוח ראשוניים, ומראה כיצד ניתן לעצב בעזרתו מערכות חישוביות חכמות, חסכוניות ואפקטיביות יותר. את עבודה זו ביצענו על ידי קריאה וניתוח של המאמרים האקדמיים הרלוונטיים, הבנה מעמיקה של המודלים ושל התהליכים החישוביים שהם מציעים, וניתוח המבנה המתמטי שלהם. בנוסף, נערך מחקר על טכניקות למידה, אסטרטגיות הערכה ומדדי ביצוע, תוך השוואה בין המודלים המבוססים על תוצאות ניסוייות.

2. רקע תיאורטי

תחום עיבוד הנתונים תלויי הזמן זוכה בשנים האחרונות להתעוררות והתחדשות וממלא תפקיד מרכזי בתחומים רבים החל מעיבוד אותות, דיבור ועד לזיהוי פעילות אנושית ועיבוד שפה טבעית. בהם קיים צורך במודלים המסוגלים לשמר מידע היסטורי בצורה יעילה.

תחום זה מהווה אתגר משמעותי, מאחר ומשימות רבות דורשות שימור מידע לאורך זמן והבנה של הקשרים הסדרתיים בין פרטי המידע. מודלים מסורתיים כמו RNN ויחידות LSTM פותחו כדי לתת מענה לאתגרים אלו, הם אפשרו למידה של דפוסים לאורך ציר הזמן. עם זאת, למודלים אלו ישנן מגבלות חישוביות, בעיקר בכל הנוגע לשימור מידע לאורך זמן רב וליכולת להתמודד עם תלות ארוכת טווח בנתונים.

במקביל, מודלים כמו טרנספורמרים הציעו פתרון המבוסס על מנגנוני Self-Attention, המאפשרים ניתוח סדרות נתונים ללא צורך בעיבוד סדרתי. אף על פי שהטרנספורמרים הפכו דומיננטיים בתחומים כמו עיבוד שפה טבעית וראייה ממוחשבת, הם עדיין דורשים כוח חישובי משמעותי, מה שהופך אותם לפחות אידיאליים עבור מערכות בזמן אמת או חומרות מוגבלות משאבים כמו IoT ו- Edge Computing.

על רקע זה פותח מודל ה- LMU (Legendre Memory Unit), אשר נועד לספק פתרון המשלב את היתרונות של שימור מידע לאורך זמן עם צריכת משאבים מינימלית. LMU מבוסס על פולינומים אורתוגונליים מסוג לג׳נדר, המאפשרים שמירה רציפה של מידע והפחתה משמעותית בדרישות החישוביות, מה שהופך אותו לאידיאלי עבור מערכות חישוב נוירומורפיות, רשתות מבוססות ספייקים (SNN), ומערכות זמן אמת. עבודה זו עוסקת בניתוח מעמיק של מודלים מבוססי LMU, תוך הצגת ההיבטים המתמטיים, הארכיטקטוניים והיישומיים שלהם, השוואתם למודלים אחרים כגון טרנספורמרים ורשתות חוזרות, ובחינת הפוטנציאל שלהם בתחום החישוב הנוירומורפי. בפרקים הבאים יוצגו העקרונות התיאורטיים של עיבוד נתונים תלוי - זמן, האתגרים המרכזיים של רשתות עצביות מסורתיות, הצורך בשיטות חישוביות חדשות, וייחודיות החישוב הנוירומורפי והשפעותיו על למידת מכונה מודרנית.

2.1 עיבוד נתונים תלוי זמן

עיבוד נתונים תלוי זמן או עיבוד נתונים סדרתיים הוא תחום מפתח במודלים של למידה עמוקה, המתמקד בניתוח והסקת מסקנות מנתונים המשתנים לאורך זמן. תחום זה נפוץ בתחומים רבים, כולל דיבור, שפה טבעית, וידאו, ועוד.

עיבוד נתונים סדרתיים שונה מניתוח נתונים סטטיים, בכך שהוא מתמקד בהבנת הקשרים והתלותיות בין נקודות עוקבות בסדרה. לדוגמא, בניתוח טקסט, סדר המילים משפיע על המשמעות הכוללת של המשפט. בחיזוי מזג אוויר, טמפרטורות ולחצים בעבר משפיעים על תנאי מזג אוויר העתידיים.

האתגר האמתי בעיבוד נתונים סדרתיים מתעורר כאשר הנתונים הופכים מורכבים ודורשים הבנת תלות ארוכת טווח תוך שמירה על יעילות חישובית אנרגטית. אתגר זה כולל התמודדות עם בעיות מבניות כמו vanishing gradients, אשר מגבילות את היכולת ללמוד תלותיות ארוכות טווח. כדי להתמודד עם האתגרים הללו פותחו בעשורים האחרונים מודלים כמו RNN, LSTM ו- GRU. מודלים אלו הניחו את התשתית לפיתוחים מתקדמים יותר, המאפשרים להתמודד עם בעיות מורכבות ולעצב טכנולוגיות חישוביות עתידיות. מודלים אלו מהווים אבן דרך משמעותית בהתפתחות עיבוד נתונים תלויי זמן, עליהם, ועל ההתפתחויות שבאו בעקבותיהם, נרחיב בהמשך העבודה.

2.1.1 מבנה רשתות נשנות והבעיות המרכזיות שלהן

רשתות נשנות הן אחד מהמודלים הראשונים שנועדו להתמודד עם עיבוד נתונים תלויי זמן, תוך שמירה על תלות בין שלבים עוקבים את המידע. בניגוד לרשתות עצביות רגילות, בהן המידע זורם רק בכיוון אחד, RNN מכיל חיבורים חוזרים המאפשר למודל לשמור מידע מהעבר ולהשתמש בו בניתוח השלבים הבאים.
רשתות RNN מורכבות מבלוקים המחוברים לאורך ציר זמן, כך שהמידע הזורם במודל אינו מוגבל רק לשכבות העיבוד הנוכחיות, אלא גם לערכים קודמים. בכל שלב, הרשת מקבלת קלט חדש, אך בנוסף גם שומרת מצב פנימי h_t של המידע הנלמד בשלבים הקודמים שבאמצעותו היא לומדת תלות סדרתית בין הנתונים.

בצורה פורמלית, בהינתן רצף $x = (x_1, x_2, \dots, x_T)$, עדכון המצב הנסתר יהיה:

<u>משוואה 1:</u>

$$h_t = \begin{cases} 0, & t = 0 \\ \phi(h_{t-1}, x_t), & \text{otherwise} \end{cases}$$

כאשר ϕ היא פונקציה לא לינארית, כמו שילוב של פונקציית סיגמואיד לוגיסטי עם טרנספורמציה לינארית.
זה אומר: לכל טוקן ברצף הקלט, רשת ה-RNN לא רק מסתמכת על הערך הנוכחי, אלא גם על המידע מהשלב הקודם. היא שומרת זיכרון של מה שהיא כבר ראתה.
נוכל לכתוב את עדכון המצב הנסתר כך.

<u>משוואה 2:</u>

$$h_t = g(Wx_t + Uh_{t-1} + b)$$

g - פונקציית האקטיבציה.

h_t - הוא המצב הנוכחי של הנוירון.

x_t - הוא הקלט החדש בזמן t.

W, U – מטריצות משקלים המתעדכנות בזמן האימון.

למרבה הצער, נמצא במחקרים קודמים [1], כי קשה לאמן RNNs לתפוס תלויות לטווח ארוך.
כשהרשת מנסה ללמוד קשרים רחוקים, המידע נשכח בגלל דעיכת הגרדיאנט.
במקרים נדירים, הגרדיאנט נהיה גדול מדי (התפוצצות גרדיאנט), מה שגורם ללמידה להיכשל. זה מקשה לאמן את המודל כמו שצריך, כי המידע מהעבר הרחוק מתפוגג.
כלומר רשתות RNN מתקשות ללמוד קשרים רחוקים בין שלבים מוקדמים לשלבים מאוחרים.
בנוסף לחוסר היכולת ללמוד תלות ארוכת טווח, רשתות RNN מבוססות על חישוב סדרתי, כלומר כל שלב תלוי בקודם לו. תכונה זו מקשה על ביצוע אופטימיזציות חישוביות, ומגבילה את היכולת לנצל חומרה כמו GPU בצורה יעילה.

2.1.2 התפתחויות קודמות: LSTM, GRU, ConvLSTM

התמודדות עם מגבלותיהן של רשתות RNN, הובילה לפיתוח מספר פתרונות המבוססים על שינוי מבנה הנוירונים עצמם כך שיוכלו לשמר מידע לזמן ממושך יותר. אחת התרומות הראשונות והמשמעתיות ביותר בתחום הייתה מודל ה– LSTM (Long Short-Term Memory), שהוצג לראשונה על ידי Hochreiter ו-Schmidhuber. מודל זה כולל מבנה פנימי המורכב מתא זיכרון ומנגנוני בקרת זרימה (שערי קלט, שכחה ופלט), שמאפשרים לו לשלוט על מידע שנשמר לאורך זמן, תוך שמירה על יציבות הגראדיאנטים.

באופן מתמטי, נוכל לתאר את האופן שבו פועלת יחידת LSTM כך:

חישוב הפלט של היחידה:

משוואה 3:

$$h_t^j = o_t^j \tanh(c_t^j)$$

h_t^j - הפלט של יחידת ה- LSTM בזמן t, עבור הנוירון – j.

o_t^j – שער הפלט, קובע כמה מהזיכרון נחשף החוצה.

c_t^j – ערך תא הזיכרון לאחר העדכון.

הפלט הוא שילוב של המידע המאוחסן בתוך תא הזיכרון עם רמת הבקרה של שער הפלט.

חישוב שער הפלט:

משוואה 4:

$$o_t^j = \sigma(W_0 x_t + U_0 h_{t-1} + V_0 c_t)^j$$

x_t: קלט בזמן t.

h_{t-1}: הפלט מהשלב הקודם

c_t: הזיכרון הנוכחי בתא

W_0, U_0, V_0: מטריצות משקלים ללמידה

שער הפלט קובע איזה חלק מהזיכרון יועבר הלאה כפלט. הוא תלוי בקלט הנוכחי, בזיכרון, ובמצב הקודם של היחידה.

עדכון תא הזיכרון:

משוואה 5:

$$c_t^j = f_t^j c_{t-1}^j + i_t^j \tilde{c_t}^j$$

f_t^j: שער השכחה, קובע כמה לשמר מהזיכרון הקודם.

i_t^j : שער הקלט, קובע כמה מהמידע החדש יכנס.

$\tilde{c_t}^j$: הערך החדש של תא הזיכרון, לפי הקלט הנוכחי.

עדכון תא הזיכרון, תא זיכרון משלב בין מה שנשמר מהעבר לבין מידע חדש שמתווסף מהקלט הנוכחי.

יצירת ערך הזיכרון החדש:

<u>משוואה 6:</u>

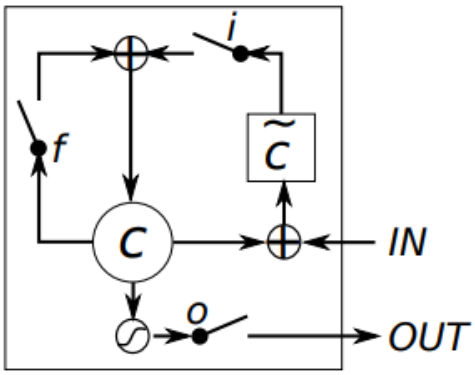
$$c_t^{\sim j} = \tanh(W_c x_t + U_c h_{t-1})^j$$

במשוואה זו ניתן לראות מידע חדש שמועמד להיכנס לתא הזיכרון, ונקבע לפי הקלט והפלט מהשלב הקודם, הוא יכנס בפועל בהתאם לערך של שער הקלט ממשוואה 5.

חישוב שערי השליטה:

$$f_t^j = \sigma\left(W_f x_t + U_f h_{t-1} + V_f c_{t-1}\right)^j$$
$$i_t^j = \sigma(W_i x_t + U_i h_{t-1} + V_i c_{t-1})^j$$

כפי שניתן לראות גם באיור 1 יחידת ה- LSTM שומרת בתוכה זיכרון פנימי שמתעדכן בכל שלב על ידי שילוב שלושה מנגנונים: שער שכחה, שער קלט, שער פלט. השערים האלה נשלטים על ידי פונקציית סיגמואיד, ולכן מתנהגים כמו ברזים שפותחים ונסגרים בין 0 ל-1.

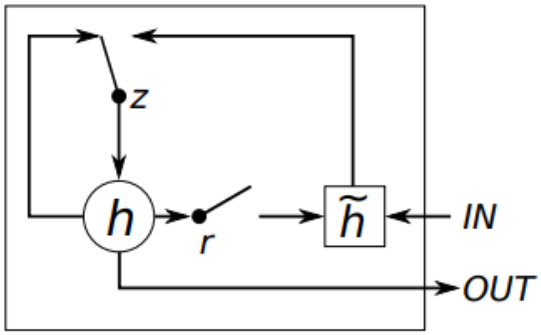


איור 1: Long Short-Term Memory [1]

איור 1: באמצעות תא זיכרון פנימי ומנגנוני שערים הכוללים שער קלט, שער שכחה ושער פלט. מאפשר LSTM שליטה מדויקת על המידע שנשמר ונשכח לאורך הרצף, וכך הוא מצליח ללמוד תלות ארוכת טווח בצורה יציבה. השערים מאפשרים למודל ללמוד אילו נתונים לשמור, אילו לשכוח ואילו להוציא כתפוקה, בכך לשמר מידע רלוונטי לאורך זמן.

בעקבות הצלחתו של LSTM, המודל שימש בסיס לפיתוחים מתקדמים נוספים, כגון GRU (Gated Recurrent Unit) אותו ניתן לראות באיור 2. מודל זה הוצע בשנת 2014 כווריאציה פשוטה וחסכונית יותר ל- LSTM. מודל ה- GRU מאחד את שער השכחה ושער הקלט לשער אחד הנקרא שער עדכון, ובכך מפחית את מספר הפרמטרים הנלמדים ואת המורכבות החישובית. כמו כן, במקום לשמר תא זיכרון פנימי

נפרד, ה- GRU מעדכן ישירות את הווקטור מצב הנסתר h_t בהתאם לשילוב בין המידע החדש והמידע הקודם.



איור 2 : Gated Recurrent Unit [1]

איור 2 : GRU הוא גרסה פשוטה יותר של LSTM המשלב שני שערים בלבד, שער עדכון - z ושער איפוס - r, מה שמפשט את החישובים ומאפשר התכנסות מהירה יותר. שער העדכון שולט על כמות המידע מהמצב הקודם שיש לשמור, ושער האיפוס קובע כמה מהמצב הקודם יש להתעלם. היעדר שער הפלט (הקיים ב LSTM) גורם לכך שכל המצב נחשף במלואו [1].
עם זאת, בהשוואה ל- LSTM חסרונו של GRU הוא בשליטה הפחות גמישה על שימור המידע. מה שעלול להשפיע על יכולתו להתמודד עם תלות ארוכת טווח.

2.1.2.1 למידת תלות מרחבית – זמנית

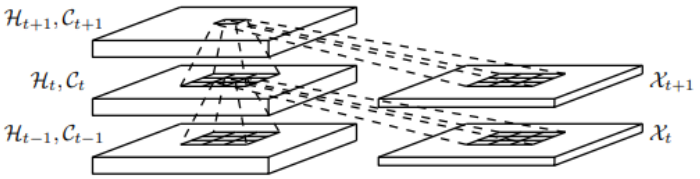
בעוד ש GRU ו- LSTM מתמקדים בשימור תלות סדרתית לאורך זמן, פותחו מודלים נוספים לטיפול במשימות בהן נדרשת למידה של קשרים מרחביים זמניים.
מודל ConvLSTM שהוצע לראשונה בשנת 2015, משלב בין מנגנוני LSTM לשכבות קונבולוציה CNN.
מטרת השילוב היא לאפשר זיהוי תבניות בזמן ובמרחב. בין השימושים הראשונים של המודל היה חיזוי משקעים לטווח קצר [4].

כדי להבין יותר טוב את הבעיה ואת המודל, נפרק לשלבים ונגדיר את בעיית חיזוי המשקעים שהוצגה במאמר. נגדיר את הבעיה כבעיית חיזוי רצפים מרחביים זמנים, שניתן לפתור באמצעות מסגרת למידת רצפים כללית. אפשר להתייחס לזה בתור מערכת דינמית שמשתנה בממד הזמן ובממד המרחב. כלומר, אנחנו מתבוננים בתופעה המתפתחת לאורך הזמן על גבי אזור מרחבי, המחולק לתאים או אזורים קטנים יותר.
ניתן לייצג את האזור הנצפה כרשת דו-ממדית בגודל $N \times M$ שבה N מייצג את מספר העמודות ו-M את מספר השורות. בכל תא ברשת מתבצעות P מדידות שונות המשתנות עם הזמן, כך שניתן לייצג את מצב המערכת בזמן נתון באמצעות טנזור מרחבי זמני: $X_t \in R^{N\times M\times P}$; R- מייצג את תחום הערכים של המדידות.
אם אנחנו אוספים מדידות באופן מחזורי לאורך זמן, כלומר בכל רגע נתון t אנחנו מקבלים טנזור חדש, נקבל רצף של טנזורים המייצג מערכת סדרתית מרחבית זמנית $X_1, X_2, \ldots X_t$.

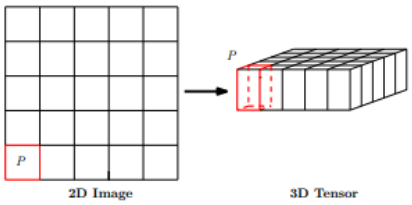
נחזור לדוגמה של חיזוי משקעים, כל תצפית בזמן נתון היא מפה דו ממדית של נתוני מכ״ם, שבה כל פיקסל מכיל מידע על כמות המשקעים באזור מסוים. כדי להתמודד עם הבעיה באופן חישובי, ניתן לחלק את המפה לרשת של תאים ולייצג את הפיקסלים בתוך כל תא כמדידות המתארות את המצב המקומי של אותו אזור. כך, חיזוי המשקעים הופך לבעיה של חיזוי סדרות מרחביות זמניות ,שבה נדרש ללמוד את הדינמיקה של הנתונים גם בממד המרחבי וגם בממד הזמן. בעיות מסוג זה דורשות שימוש במודלים מתקדמים המשלבים עיבוד מרחבי וזמני כאחד.

2.1.2.2 :ConvLSTM

ConvLSTM הוא הרחבה של המודל FC-LSTM על ידי שילוב שכבות קונבולוציה. ConvLSTM מוסיף שכבות קונבולוציוניות שמשפרות את הניתוח המרחבי על ידי עיבוד הנתונים בתבניות דו ממדיות. יש במודל התחשבות בסביבה המקומית של כל פיקסל, הרשת קובעת את מצב העתיד של תא מסוים ברשת על סמך הקלטים והמצבים של שכניו המקומיים [4]. ניתן לראות את מבנה המודל באיורים 3 ו-4. מעברי מצב מהקלט למצב וממצב למצב מבוצעים כעת עם מסננים קונבולוציוניים, המאפשרים למודל לתפוס מידע מקומי חשוב.



איור 3 : מבנה פנימי של ConvLSTM [4]



איור 4 : הפיכת תמונה ממימד D2 לטנזור D3 [4]

איור 3 : מציג כיצד ניתן להמיר תמונת מכ״ם דו מימדית למבנה תלת ממדי. הפיקסלים בתמונה מחולקים לתאים, וכל תא מטופל באופן נפרד, מה שמוסיף מימד נוסף לניתוח המרחבי.

איור 4 : מציג את המבנה הפנימי של ConvLSTM שבו קיים מעבר מידע בין מצבים עוקבים $t, t+1$ השכבות מכילות מצבי תא וזיכרון המיוצגים על ידי H_t, C_t בעוד הנתונים הנוכחיים מוזנים אל המודל, ומאפשרים אינטרקציה טובה יותר בין הזמן והמרחב.

לצד ConvLSTM, פותח גם המודל CRNN , אשר משלב שכבות קונבולוציה עם רשתות נשנות, אך בשונה הוא אינה כולל מנגנוני שערים מתוחכמים אלא מתמקד בשימור מידע ויזואלי לאורך זמן תוך שימוש בשכבות רקורסיביות לחילוץ מאפיינים מחלונות של נתונים סדרתיים. כל חלון נתונים מוזן למודל רקורסיבי, והפלט או המצבים הנסתרים של היחידות הרקורסיביות משמשים לחישוב המאפיינים החילוץ. היתרון העיקרי של CRNN הוא שהוא יכול להפיק מאפיינים טובים יותר מחלונות נתונים על ידי שימוש במידע נוסף על כל חלון, באמצעות המידע הזמני.

מודלים אלה מציגים פתרונות שונים להתמודדות עם האתגרים בעיבוד נתונים תלויי זמן, כאשר LSTM ו- GRU מתמקדים בשיפור שימור המידע לאורך זמן, ConvLSTM ו- CRNN מתמקדים בשילוב תלויות מרחביות וזמניות.

למרות ההתקדמות, מודלים אלה עדיין סובלים ממגבלות מבחינת יעילות חישובית, צריכת אנרגיה ויכולת טיפול בתלויות ארוכות טווח, מה שמחזק את הצורך במודלים חדשים ויעילים יותר כמו טרנספורמרים ו- LMU.

2.2 הצגת הבעיה והמוטיבציה

עיבוד נתונים תלויי זמן מציב אתגר חישובי משמעותי, בעיקר בכל הנוגע לשימור מידע לטווח ארוך, להתמודדות עם סדרות נתונים ארוכות ולשימוש יעיל במשאבים חישוביים. רשתות RNN פותחו כדי להתמודד עם אתגר זה, אך הן סובלות מבעיות כגון הגרדיאנט הנעלם, חישוב סדרתי מוגבל וקושי בלמידת תלות ארוכת טווח ,מה שפוגע ביעילותן במקרים בהם רצף הנתונים משתרע על פני שלבים רבים. במשך השנים פותחו שיפורים שהצגתי קודם לכן, כמו LSTM ו- GRU ורשתות חוזרות מבוססות קונבולוציה שאפשרו למידה יציבה יותר של קשרים לאורך זמן בהשוואה לרשתות הנשנות הפשוטות. אך גם מודלים אלו לא נתנו ביצועים מספיק טובים במשימות מורכבות, והיעילות החישובית שלהם הקשתה על שילובן במערכות זמן אמת או בחומרות בעלות משאבים מוגבלים.

בעשור האחרון, מודל הטרנספורמר חולל מהפכה בתחום, תוך שהוא מציע מנגנון Self-Attention אשר מאפשר למודל ללמוד תלות ארוכת טווח ללא תלות סדרתית ישירה. הטרנספורמרים הציגו ביצועים יוצאי דופן במשימות כמו תרגום מכונה, עיבוד שפה טבעית וזיהוי תמונות, אך גם הם אינם חפים מחסרונות. המורכבות החישובית שלהם גדלה ריבועית עם גודל הקלט, מה שהופך אותם לבלתי יעילים עבור יישומים בזמן אמת או במערכות חסכוניות באנרגיה.
על רקע זה, קיים צורך בגישה חישובית חדשה ,שתשלב בין שימור מידע לאורך זמן, תאפשר למידת תלות ארוכת טווח תוך שמירה על יעילות חישובית ונצילות משאבים אופטימלית. גישות חדשות, כמו LMU מציעות דרך לשלב שמירת מידע יעילה עם צריכת אנרגיה נמוכה ויכולת התאמה למערכות נוירומורפיות. בפרקים הבאים יוצגו המגבלות המרכזיות של רשתות חוזרות, היתרונות והחסרונות של טרנספורמרים, וההצדקה לצורך בפיתוח גישות חישוביות חדשות.

2.2.1 מגבלות חישוביות של רשתות חוזרות

רשתות RNN היוו פריצת דרך בעיבוד נתונים תלויי זמן, בכך שאפשרו למידה של תלות סדרתית ברצפים. עם זאת, יעילותן החישובית מוגבלת בשל מספר בעיות מהותיות, אשר משפיעות על יכולת הלמידה, הביצועים והסקלביליות שלהן. מגבלות אלו הובילו לפיתוחם של מודלים מתקדמים יותר, כמו LSTM GRU, וטרנספורמר אך גם פתרונות אלו אינם חפים מחסרונות.

Vanishing and exploiting Gradients 2.2.1.1

אחת הבעיות המרכזיות של רשתות נשנות היא הקושי בשימור מידע לטווח ארוך ,הנובע מהאופן שבו מתבצע עדכון המשקולות ברשת במהלך הלמידה. בעת שימוש ב Backpropagation Through Time הגרדיאנט מועבר לאורך הרצף. כאשר הסדרה ארוכה, הגרדיאנט נוטה לדועך לערכים קטנים מאוד (Vanishing Gradient) או להפוך לערכים גדולים בצורה לא יציבה (Exploding Gradient). כתוצאה מכך, הרשת מתקשה ללמוד תלויות ארוכות טווח, ומשקולות השכבות המוקדמות ברשת כמעט ואינן מתעדכנות, מה שמונע מהמודל ללמוד מידע קריטי מהשלבים הראשונים של הרצף [13].

2.2.1.2 חישוב סדרתי ואי-ניצול יעיל של חומרה

בניגוד למודלים כמו CNN או טרנספורמר בהם ניתן לבצע חישובים רבים במקביל, רשתות נשנות מבצעות חישוב סדרתי, כל שלב תלוי בתוצאה של השלב הקודם. עובדה זו מגבילה את יעילות החישוב על חומרות מודרניות כגון GPU, אשר מותאמות יותר לחישוב מקבילי. המשמעות היא שאימון רשתות RNN איטית יותר באופן משמעותי בהשוואה למודלים מודרניים, במיוחד כאשר מתמודדים עם רצפים ארוכים מאוד [14].

2.2.1.3 סיבוכיות חישובית גבוהה ודרישות זיכרון

הצורך לשמור את כל המצבים החבויים לאורך ציר הזמן יוצר עומס חישובי כבד. לדוגמה, ברשתות LSTM ו- GRU, נדרשות מספר מטריצות נוספות עבור כל צומת ברשת, מה שמוביל לגידול מעריכי בזיכרון הנדרש לאימון הרשת. בנוסף, ככל שהרצף ארוך יותר, כך הכמות הנדרשת של פעולות כפל מטריציוניות עולה, מה שהופך את RNN לבלתי ישים עבור בעיות עם נתונים ארוכים מאוד [1].

2.2.1.4 קושי בהתמודדות עם תלות ארוכת-טווח

למרות שיפורים כמו LSTM ו- GRU מודלים אלו עדיין אינם אידיאליים כאשר יש תלות במידע מהשלבים המוקדמים של הרצף. לדוגמה, בטקסט ארוך, מודלים אלו מתקשים לזכור את ההקשר מתחילת הרצף. לכן, במקרים רבים בהם נדרשת הבנה של מבנה נתונים רחב מאוד מודלים אלו פשוט אינן מסוגלות לשמר מידע מספיק לאורך זמן. בנוסף מודלים אלו משתמשים בשערים כדי לשלוט על זרימת המידע, אך הם עדיין לא אופטימליים לחלוטין בניהול המידע. הם נדרשים להחליט באופן מפורש כמה מידע לשמור וכמה לשכוח בכל שלב, מה שמקשה על שימור מידע ארוך טווח.

2.2.2 טרנספורמרים כפתרון חלקי

בשנת 2017, נוצג מודל הטרנספורמר (Transformer), כמודל חדשני ללמידה על רצפים המבוסס כולו על מנגנון Attention, וללא שימוש ברשתות חוזרות או קונבולוציוניות בשונה מודלים אחרים עד כה [14]. המודל מאפשר למידה מקבילית על כל איברי הרצף ומאפשר קישור בין פריטים רחוקים באמצעות מנגנון שנקרא Scaled Dot-Product Attention, שהוא הבסיס ל Multi-Head Attention (MHA). אפשר לתאר את פונקציית ה- Attention (תשומת הלב) כמנגנון שמקבל שאילתה (query) וקבוצת זוגות של מפתחות וערכים (key and values), ומחזיר פלט שהוא וקטור. הפלט מחושב כסכום ממוצע משוקלל של הערכים, כלומר כל ערך תורם לפלט לפי משקל. המשקל של כל ערך נקבע לפי מידת הדמיון בין השאילתה למפתח המתאים.

אפשר לתאר את Scaled Dot-Product Attention בצורה פורמלית כך:

<u>משוואה 7:</u>

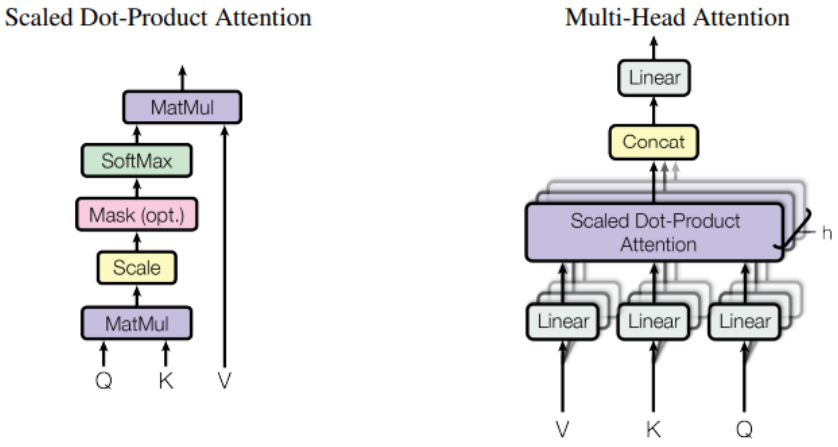
$$Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

כאשר Q, K, V הם מטריצות השאילתה, המפתחות והערכים בהתאמה, ו- d_k הוא מימד המפתח.
הרחבה של מנגנון זה הוא מנגנון MHA, שבו מתבצעים מספר חישובי Attention במקביל, על גרסאות שונות של Q, K, V. לאחר מכן תוצאות כל הראשים מאוחדות ומועברות דרך שכבה לינארית, ראה איור 5. השימוש במספר ראשים מאפשר למודל ללמוד סוגים שונים של קשרים בין אברי הרצף בו זמנית.
אפשר לתאר את זה בצורה פורמלית כך:

<u>משוואה 8:</u>

$$MultiHead(Q,K,V) = Concat(head1,\dots,head_h)W^O \; where \; head_i$$
$$= Attention(QW_i^Q, KW_i^K, VW_i^V)$$

הדבר מאפשר למודל ללמוד משמעויות מרובות לכל יחידת קלט, מה שתורם להבנה עמוקה יותר של תלות גלובלית בין חלקי המידע.



איור 5 : מנגון תשומת הלב בטרנספורמר. מימין מנגנון קשב רב-ראשי שמורכב מכמה שכבות של קשב הפועלות במקביל אותן ניתן לראות בצד שמאל [14]

איור 5 : האיור מציג את מנגנון תשומת הלב והשכבות שלו. בצד שמאל ניתן לראות את שכבת ה- Scaled Dot-Product Attention, המבצעת חישוב משוקלל בין השאילתה Q, המפתחות K והערכים V באמצעות מכפלה מטריציונית נורמליזציה ו- Softmax. בצד ימין ניתן לראות את המנגנון Multi-Head Attention, אשר מפעיל את השכבה שראינו בצד השמאלי במקביל עבור מספר ראשים, כדי ללכוד קשרים מסוגים שונים. לאחר מכן מאחד את הפלטים דרך Linear (מעבירים את הוקטור דרך שכבה לינארית) ו- Concat (מחברים את הפלטים של הוקטורים אחד אחרי השני לוקטור אחד).

מכיוון שבטרנספורמרים אין חישוב סדרתי כמו ב- RNN, יש צורך להוסיף מידע מיקום כדי לשמור על סדר הנתונים. המנגנון של Positional Encoding משתמש בפונקציות סינוס וקוסינוס כדי לקודד את המיקום של כל יחידה ברצף בצורה שמאפשרת למודל להבין את סדר הקלט [14].

למרות יתרונותיו הרבים, יש למודל גם חסרונות, בעיקר מבחינת חישוביות וסיבוכיות. מנגנון תשומת הלב מחייב חישובים רבים עבור כל צמד קלטים, מה שמוביל לעלויות חישוביות גבוהות, במיוחד ברצפי מידע ארוכים. מנגנון זה דורש משאבי חישוב וזיכרון בסדר גודל ריבועי ביחס לאורך הקלט $O(T^2 d)$ כאשר T הוא אורך הרצף ברצף ו- d מימד הייצוג. מה שהופך את האימון וההסקה ליקרים ולא יעילים, במיוחד עבור רצפים ארוכים. כמו כן, שכבות תשומת לב יעילות יותר משכבות RNN כאשר אורך הרצף קטן יותר מהמימד d של הוקטורים.

בנוסף, חשוב לציין כי למודל המקורי שהוצע במאמר [14] הייתה מגבלה מוהקת של טווח זיכרון מוגבל, כיוון שמנגנון תשומת הלב פועל על חלון קלט באורך קבוע ודורש סיבוכיות ריבועית ביחס לאורך הרצף. עם זאת, במציאות המחקרית של השנים האחרוות הטענה שטרנספורמרים אינם מסוגלים לטפל בתלויות ארוכות טווח כבר אינה נכונה. מחקרים עדכניים הציעו מודלים חדשניים אשר מרחיבים את יכולות הזיכרון ועיבוד המידע של טרנספורמרים, תוך שמירה ואף שיפור היעילות החישובית, כדוגמת CCA-Attention ועוד נוספים [28]. למרות ההתקדמות הרבה בשיפור היעילות של טרנספורמרים, הם עדיין נחשבים למודלים עתירי משאבים. הן במובן של עלויות האימון והן בגודל המודל. ככל שמבקשים להרחיב את טווח ההקשר וללכוד תלויות ארוכות טווח, נדרשת הגדלה משמעותית של מספר הפרמטרים, מה שמוביל לעלויות חישוב גבוהות במיוחד, שימוש נרחב בזיכרון, וזמן אימון ארוך.

2.2.3 הצורך בגישה חישובית חדשה

לאור המגבלות של רשתות נוירונים חוזרות RNN ומודל הטרנספורמר, עולה הצורך בגישה חישובית חדשה שתאפשר התמודדות יעילה עם נתונים סדרתיים מורכבים. גישה זו צריכה לתת מענה למספר אתגרים מרכזיים, תוך התמקדות בשיפור היעילות, הגמישות והיכולת ללכוד תלויות ארוכות טווח.

בעיות רבות בעולם האמתי אינן מתבססות על רצפי מידע בדידים, אלא מתאפיינות בתהליכים דינמיים רציפים בזמן, כמו תנועת רובוטים, עיבוד אותות מוחיים (EEG), חיזוי מזג אוויר ועוד. מודלים שראינו כמו LSTM וטרנספורמרים נבנו במקור להתמודדות עם קלט בדיד בדגימה אחידה, ולכן אינם מתאימים באופן טבעי לתהליכים שמתפתחים בצורה רציפה לאורך זמן ובקצבי דגימה משתנים.

על אף שהטרנספורמרים עברו בשנים האחרונות התאמות המאפשרות להם לטפל ברצפים לא אחידים או ארוכים יותר, הם עדיין אינם מייצגים את הדינמיקה של הזמן כישות מתמטית רציפה. במקום זאת, הם מקודדים את המידע הזמני. יש צורך במודלים המסוגלים לייצג את הדינמיקה של מידע מתמשך מבלי להיות מוגבלים לייצוג בדיד [3].

יש צורך במודלים המסוגלים לשמר מידע לאורך רצפים ארוכים מאוד, ללא תלות בחלון זיכרון מוגבל. מודלים אלה צריכים ללכוד תבניות המשתרעות על פני פרקי זמן ארוכים, ולאפשר ניתוח מעמיק של נתונים סדרתיים מורכבים.
יש לשאוף למודלים בעלי מורכבות חישובית נמוכה יותר, המאפשרים אימון והסקה מהירים ויעילים, במיוחד עבור רצפים ארוכים. מודלים אלה צריכים לנצל משאבים בצורה חכמה, ולהימנע מעלויות חישוביות גבוהות.
יש לפתח מודלים התומכים בעיבוד מקבילי של נתונים, כמו הטרנספורמרים, המאפשרים לנצל את מלוא הפוטנציאל של חומרת מחשב מודרנית (GPU). אימון מקבילי יכול להפחית משמעותית את זמן האימון וההסקה, ולאפשר טיפול בנפחי נתונים גדולים.
יש לפתח מודלים גמישים בעלי יכולת הסתגלות, שמסוגלים להתמודד עם דינמיקה משתנה של נתונים, ולתת חשיבות שונה לחלקים שונים בהיסטוריה. ובו זמנית עליהם להיות בעל יכולת להתמודד עם מגוון רחב של משימות ויישומים, ולהציע ביצועים מיטביים בתנאים שונים.
מודלים מבוססי LMU עליהם ארחיב בהמשך, מציעים גישות שונות מבטיחות לטיפול באתגרים אלו. הם משלבים מנגנוני זיכרון יעילים, יכולות למידה מקבילית, יכולת הסתגלות לדינמיקה משתנה של נתונים שמאפשרת להם התעלות על מודלים קודמים.

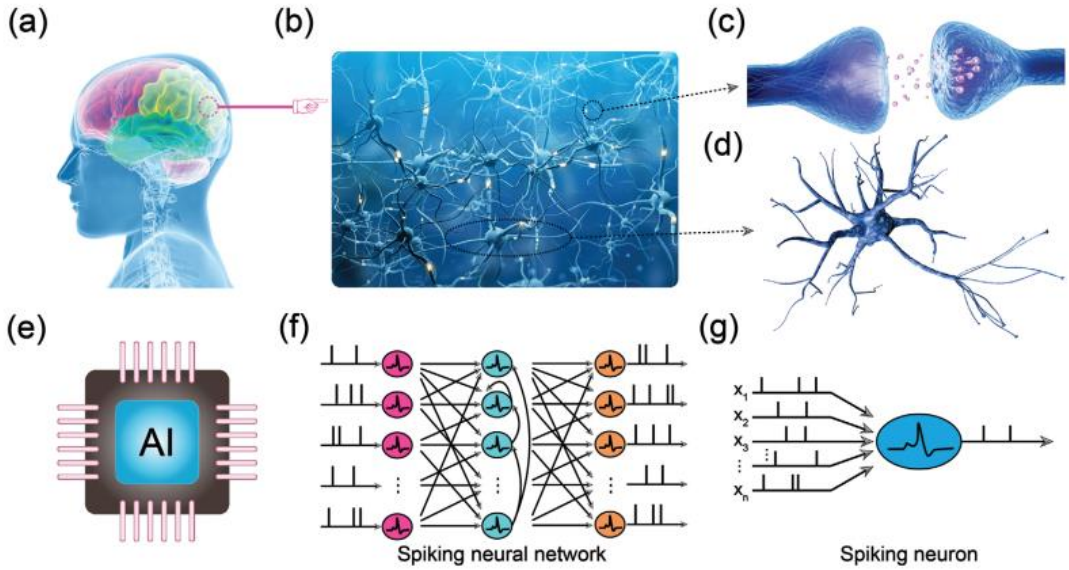
יתר על כן, שילוב טכנולוגיות נוירומורפיות מציע פתרון יעיל במיוחד. טכנולוגיות אלו, המחקות את פעולת המוח הביולוגי, מספקות יתרונות משמעותיים בצריכת אנרגיה וביעילות חישובית. במאמר נחקור מודלים מבוססי LMU, כגון L2MU המתאימים במיוחד ליישום בסביבה נוירומורפית, הודות ליכולתם לפעול עם רשתות עצביות מבוססות ספייקים.

2.3 רקע תיאורטי על חישוב נוירומורפי

כאשר מדענים ומהנדסים מנסים לבנות מערכות חכמות, אחת השאלות שהם שואלים את עצמם היא: "מהו המחשב החכם ביותר שאנחנו מכירים?" והתשובה כמעט תמיד תהיה – המוח האנושי. המוח האנושי הוא מכונה מורכבת להפליא, המסוגלת לבצע עיבוד מידע עצום, להבין שפה, לזהות פנים, לתכנן קדימה, לזכור וללמוד. כל זאת תוך צריכת אנרגיה של בערך נורה קטנה של 20 ואט בלבד.
המכונה הזאת שנקראת המוח האנושי היא הרציונל שעומד מאחורי החישוב הנוירומורפי. מדענים מנסים למצוא דרך לחקות את הדרך שבה המוח עובד, כדי ליצור מערכות מלאכותיות שיעבדו בצורה חכמה, גמישה וחסכונית באנרגיה, בדיוק כמו המוח.

2.3.1 יסודות החישוב הנוירומורפי והשראה ביולוגית

כדי להבין מה מחקים, צריך קודם להבין מה באמת קורה במוח. המוח שלנו מורכב ממיליארדי תאים מיוחדים שנקראים נוירונים. כל נוירון הוא כמו תחנת ממסר קטנה, הוא מקבל מידע מהסביבה שלו, לרוב מנוירונים אחרים דרך מבנים שנקראים דנדריטים ראה איור 6. המידע הזה הוא לא מספרים אלה פולסים חשמליים קטנים מאוד שנקראים "ספייקים".



איור 6 : השוואה סכמטית בין מערכות העצבים הביולוגיות לבין מערכות חישוב מלאכותיות מבוססות רשתות עצביות מתקפיות [12]

איור 6 : האיור מציג השוואה חזותית בין מערכת העצבים הביולוגית (בחלק העליון) לבין מערכות חישוב נויורומורפיות מלאכותיות (בחלק התחתון), תוך הדגשת הקשר הישיר בין המבנים הפיזיים במוח לבין המודלים החישוביים המודרניים.

(a) מתאר את מבנה המוח האנושי.

(b) מציג תמונה מיקרוסקופית של רשתות נוירונים ביולוגיות, מערכת של תאי עצב שמעבירים מידע זה לזה.

(c) האזור שבו הנוירון מעביר מידע לנוירון אחר

(d) מתמקד בתא עצב בודד, עם דנדריטים ואקסון.

(e) מתאר שבב חישובי המייצג את תשתית החומרה של מערכות נוירומורפיות, מעגלים אלקטרונים המדמים את פעולת המוח.

(f) מציג רשת נוירונים מבוססת ספייקים, זה למעשה מודל חישובי שמדמה העברת מידע על ידי ספייקים, באופן דומה להתפרצות חשמלית ביולוגית.

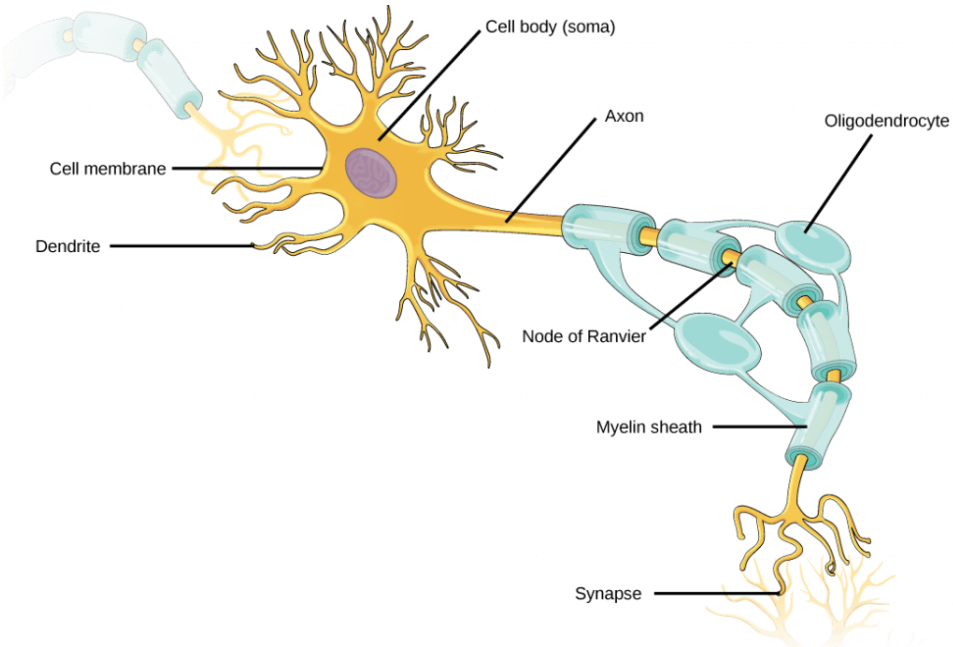
(g) ממחיש נוירון ספייקי בודד, את תגובת הנוירון לקלקטים שונים באמצעות ייצור ספייק רק כאשר מתקיים תנאי מסויים.

2.3.1.1 השראה ביולוגית

לנוירון יש "ענפים" הנקראים דנדריטים, הם מקבלים מסרים (אותות חשמליים קטנים) מנוירונים אחרים. כל אות כזה עובר דרך חיבור מיוחד שנקרא סינפסה.

כפי שניתן לראות באיור 7 הסינפסה היא נקודת המפגש בין שני נוירונים, נוירון אחד משדר אות, והנוירון השני מקבל אותו דרך הסינפסה.

ברגע שמגיעים הרבה הרבה אותות מהסינפסות, הנוירון צובר את המידע. וצובר מתח חשמלי בתוכו, אם המתח החשמלי מגיע לרמה מסוימת הנוירון יורה ספייק לעבר נוירונים אחרים (תהליך זה נקרא דחף עצבי עליו נרחיב בהמשך). הספייק נע לאורך "חוט" דק שנקרא אקסון, עד שהוא מגיע לסינפסות, ומשם ממשיך לדנדריטים של הנוירונים הבאים.

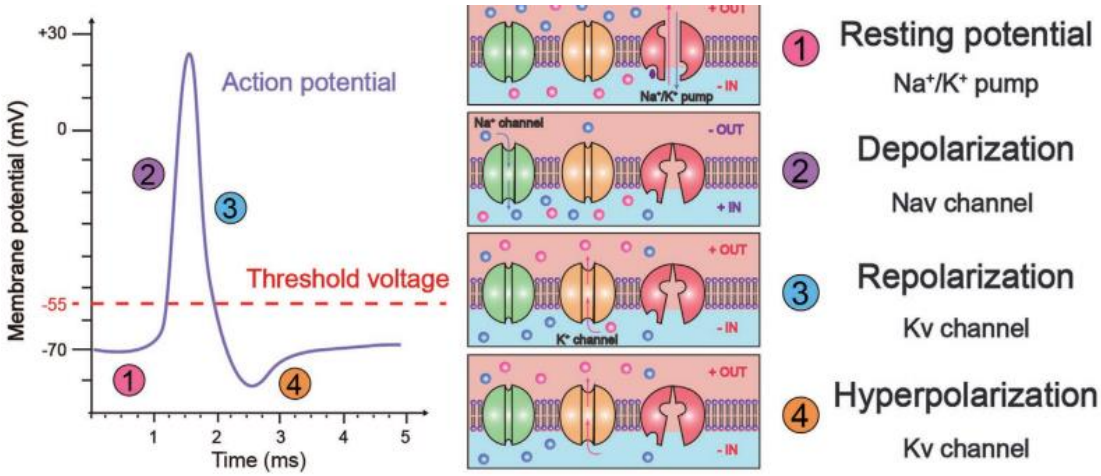


איור 7 : מבנה הנוירון [16]

איור 7 : האיור מציג מבנה של נוירון ביולוגי, כולל הדנדריטים, גוף התא (soma), האקסון שמעביר את המידע, והסינפסה דרכה הנוירון מתקשר עם נוירונים אחרים. ניתן גם לראות את המיאלין, שמאפשר הולכה מהירה של הדחף העצבי לאורך האקסון.

2.3.1.2 דחף עצבי

כפי שניתן לראות באיור 8 שמתאר את פוטנציאל הפעול בנוירון, בכל רגע כל נוירון "צובר" את הקלטים שהוא מקבל. אם הקלטים חזקים או מרובים מספיק, כלומר אם נצבר מתח חשמלי שמגיע לסף מסוים. הנוירון יוצר "קפיצה חשמלית" שנקראת ספייק אותו תיארתי קודם לכן, או במילים אחרות "דחף עצבי", ושולח אותה לתאים אחרים דרך שלוחה שנקראת אקסון. התאים שאליהם הוא מחובר יכולים גם להגיב ולייצר קפיצות משלהם וכך נוצר זרם של עיבוד מידע [12]. המערכת הזאת לא פועלת לפי שעון כמו מחשב רגיל, היא פועלת רק כשקורה משהו שדורש תגובה. זה נקרא עיבוד אסינכרוני, והוא חוסך המון אנרגיה. בנוסף, לא כל ספייק נראה אותו דבר, לעיתוי של כל ספייק, המרחק בין ספייקים, ומי בדיוק יורה את אותו ספייק, יש משמעות שונה, זוהי מערכת גמישה ומורכבת.



איור 8 : דחף עצבי בנוירון [12]

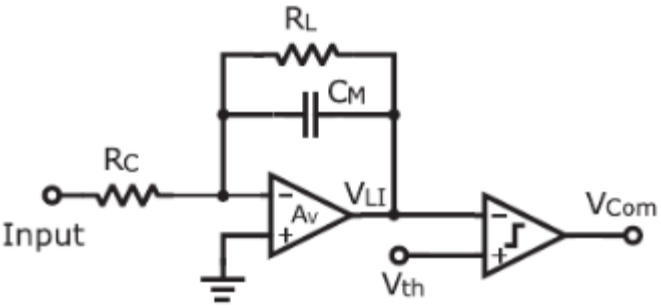
איור 8 : האיור מתאר את השינויים במתח הממברנה של נוירון לאורך זמן, בזמן יצירת הדחף העצבי ואת שלבי התהליך. באיור מוצג גרף של מתח הממברנה לאורך זמן, בו נראים ארבע שלבים עיקריים. השלב הראשון הוא מצב מנוחה, בו המתח שלילי יחסית ונשמר על ידי משאבת הנתרן אשלגן. השלב השני נקרא דה-פולריזציה, שבו תעלות הנתרן נפתחות והמתח עולה. לאחר מכן מגיע שלב הרה פולריזציה, בו תעלות האשלגן נפתחות והמתח יורד. לבסוף בשלב הרביעי מתרחשת היפר-פולריזציה, ירידה זמנית של המתח מתחת לערך הסף, עד שהנוירון חוזר למצב הבסיסי. בתהליך זה מעביר הנוירון מידע חשמלי במערכת העצבים.

LIF 2.3.1.3

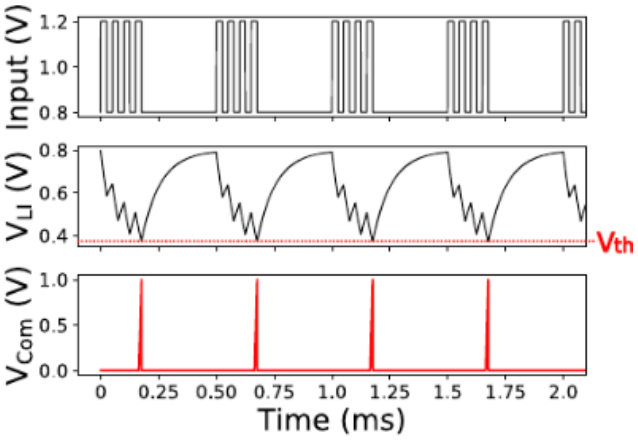
בחישוב נוירומורפי, לא מנסים להעתיק את המוח אחד לאחד, אלא ללמוד מהעקרונות הבסיסיים וליישם אותם בצורה שהמחשב יכול לעבד.

אחד המודלים הנפוצים ביותר לחיקוי נוירון ביולוגי נקרא LIF (Leaky Integrate and Fire). זהו נוירון שמקבל קלט, מצטבר בו מתח, יש "דליפה" טבעית של מתח וכשהמתח חורג מסף מסוים הוא יורה ספייק [11].

במודל LIF, כל נוירון מקבל קלטים מהסביבה, לרוב קלטים אלה הם קפיצות מנוירונים אחרים. כל קלט מעלה את פוטנציאל הממברנה של הנוירון, כלומר את המתח החשמלי שבתוכו. זה נקרא שלב הIntegrate, הנוירון "צובר" את ההשפעות. אבל הנוירון גם "מאבד" מתח כל הזמן, זהו שלב הLeaky, בשלב זה מתבצעת מעין דליפה של מתח שמונעת מהנוירון להגיע לעוררות גבוהה אם לא מתקבל מספיק קלט בפרק זמן קצר. כאשר המתח עובר סף מסוים, הנוירון יורה ספייק, ואז מאפס את הפוטנציאל שלו חזרה לאפס, זהו שלב הfire. על מנת למדל את הנוירון החוקרים בנו מעגל חשמלי המדמה את ההתנהגות הנ"ל, ראה איורים 9 ו-10.



איור 9 : מבנה המעגל החשמלי של נוירון מסוג LIF [11]



איור 10 : ההתנהגות נוירון מסוג LIF כתגובה לקלט [11]

איור 9 : מופיע תרשים המדמה את התנהגות הנוירון, אות הקלט עובר דרך נגדים וקבל אל מגבר שצובר מתח, עד שמתח זה חוצה את סף העירור, מה שמפעיל את השלב הבא במעגל וגורם להפקת "ספייק" ביציאה.

איור 10 : באיור ניתן לראות שלושה גרפים, הגרף העליון מציג אות קלט מחזורי, הגרף האמצעי מראה כיצד המתח מצטבר בכל מחזור עד שהוא חוצה את הסף, והגרף התחתון מראה את תגובת הפלט, פולס חד שנקרא ספייק, בכל פעם שהמתח חוצה את הסף. עיקרון הפעולה של LIF - אגירה, דליפה וירי.

לצד מודל ה- LIF הקלאסי, קיימות גרסאות מורחבות ושונות המאפשרות שליטה טובה יותר בדינמיקת הקלטים, ביניהם מודל Cuba-LIF. בגרסה זו של LIF מתווספת רמת סינפסה, המערכת לא רק בודקת את סך הקלטים, אלא גם מדמה את ההולכה הסינפטית בין נוירונים. זה נעשה באמצעות מצב פנימי נוסף, זרם סינפטי המתעדכן לפני חישוב המתח [23].
וכמובן שקיימים עוד מודלים שונים עם רמות מורכבות והפשטה שונות בהתאם לצרכים השונים.

במערכות נוירומורפיות, כל יחידה מחוברת ליחידות אחרות באמצעות חיבורים שיכולים להתחזק או להיחלש, בדיוק כמו במוח (רכיב זה נקרא סינפסה) . כתוצאה מכך המערכת יכולה ללמוד מדוגמאות.
בנוסף, מנסים לחקות גם את הדרך שבה מידע נשמר לאורך זמן. למשל, כשהמוח האנושי שומע שיר פעם אחת, הוא זוכר אותו. זה בזכות מנגנוני זיכרון ביולוגיים שאוגרים את הרצפים. במודלים מלאכותיים, משתמשים במבנים כמו LMU [3] עליהם נרחיב בהמשך, כדי ליצור ייצוגים כאלה של זיכרון מתמשך, שמבוסס על מתמטיקה של פולינומים אורתוגונליים שמחקים תהליך טבעי של סיכום מידע היסטורי.

2.3.2 חישוב מבוסס אירועים (Event-Based Computation)

אחד ההבדלים המרכזיים בין חישוב קלאסי לחישוב נוירומורפי הוא הדרך שבה מידע מעובד. בחישוב קלאסי, המחשב מעבד את המידע בצורה סדירה וקבועה – עיבוד סנכרוני, הוא עובד לפי קצב אחיד, לא לפי התוכן.
לעומת זאת, בחישוב נוירומורפי, ובמיוחד בגישה של חישוב מבוסס אירועים – המידע מעובד רק כאשר משהו משתנה. כלומר, לא מתבצעת שום פעולה עד שמתרחש אירוע שמצדיק תגובה. אירוע כזה יכול להיות שינוי בתמונה, שינוי בכיוון תנועה, או קלט אחר שחורג מסף מסוים [29].
גם ברמה של רשתות נוירונים, מודלים נוירומורפים פועלים באופן דומה: הם מגיבים רק כשמגיע קלט שמפעיל אותם, במקום לחשב כל הזמן. דבר זה מאפשר להם להיות חסכוניים באנרגיה ובמשאבים.
השלב הבא הוא תרגום האות הרציף לאירועים בודדים, כאן נכנס לתמונה שיטות הקידוד. קיימות מספר דרכים להמיר מידע רציף לסדרת קפיצות:

Rate Coding 2.3.2.1

עוצמת הקלט מתורגמת לקצב הירי של הנוירון. למשל, פיקסל בהיר בתמונה יגרום לנוירון לירות קפיצות בתדירות גבוהה יותר. שיטה זו פשוטה ליישום ומתאימה למערכות הדורשות עיבוד יציב לאורך זמן. למעשה ככל שהקלט חזק יותר, כך יהיו יותר ספייקים בפרק זמן מסוים [18].
עם זאת Rate Coding מצריך מספר רב של קפיצות כדי לייצג מידע, מה שעלול להוביל לצריכת אנרגיה גבוהה יותר ולזמן עיבוד ארוך יותר.

Temporal Coding 2.3.2.2

המידע מקודד לפי התזמון המדויק של הקפיצות. שיטה זו מאפשרת העברת מידע מדויקת ומהירה, אך רגישה לרעש ודורשת תזמון מדויק. בשיטה זו התזמון של כל ספייק חשוב, לדוגמה אם יש שני ספייקים קרובים בזמן, זה מסמן משהו חשוב, ואם יש הפסקה, זה מעיד אחרת.
אחת משיטות ה- Temporal Coding היא TTFS, שבה נוירון יורה קפיצה אחת בלבד, והזמן שבו הוא יורה מייצג את עוצמת הקלט. שיטה זו יעילה מבחינת אנרגיה ומהירה, אך פחות מתאימה למידע מורכב או רועש [17].

Direct Coding 2.3.2.3

היא שיטה שבה הקלט מוזן ישירות לרשת הנוירונים, והרשת עצמה אחראית להמיר את הקלט לקפיצות. שיטה זו מאפשרת לרשת ללמוד את הקידוד האופטימלי עבור המשימה, אך דורשת משאבים חישוביים גבוהים יותר [18].
החדשנות בגישה זו היא שאין בהמרה מוקדמת לספייקים, כיוון שהמודל עצמו דואג להמיר את הקלט לספייקים תוך כדי העיבוד. גישה זו נבחנה במודל L2MU [8] עליו נרחיב בהמשך, בו אותות גולמיים מחיישני תנועה הומרו לספייקים על ידי שכבת עיבוד ראשונית, ללא שלב קידוד נפרד.

2.3.3 ההבדלים בין חישוב נוירומורפי לבין חישוב קלאסי

ההשוואה בין חישוב נוירומורפי לבין חישוב קלאסי משקפת עימות בין שתי פרדיגמות חישוביות: האחת נשענת על עקרונות של עיבוד סדרתי, מדויק וסינכרוני. האחרת שואפת להפיק תובנות מדרך פעולתו של המוח הביולוגי, תוך הסתמכות על עיבוד אסינכרוני, ודינמי. ההבדלים בין שתי הגישות ניכרים בשלושה מישורים מרכזיים: החומרה התומכת בהן, מנגנוני ניהול הזיכרון, ואופן קידוד המידע.

על מנת להבין את היתרונות והחסרונות של חישוב נוירומורפי, חשוב להשוות אותו לחישוב הקלאסי, בו כפי שציינו המידע עובר דרך שרשרת של פעולות מתמטיות, כמו חיבור, כפל, או פונקציות אקטיבציה בצורה סדירה וסינכרונית. בכל רגע נתון, המחשב מבצע חישוב על כל פיסת מידע, גם אם המידע הזה לא השתנה. זה כמו לוח זמנים שבו כל תהליך קורה לפי שעון, בלי קשר אם יש שינוי או לא [15].
לעומת זאת, חישוב נוירומורפי פועל בצורה אסינכרונית ומבוססת אירועים. כלומר, הוא מגיב רק כשקורה משהו. זו גישה שמתאימה במיוחד למערכות שצריכות לעבד מידע בצורה חסכונית ובזמן אמת, כמו חיישנים לבישים, מערכות חכמות ברכבים או רובוטים.

2.3.3.1 צורת הקידוד של המידע

אחד ההבדלים הבולטים הוא בצורת הקידוד של המידע. בחישוב קלאסי, אנחנו מעבירים ערכים רציפים. כל נוירון ברשת מקבל את המספרים שהועברו לרשת ומחשב פונקציות מתמטיות עליהם. בחישוב נוירומורפי, לעומת זאת, המידע מיוצג כסדרת קפיצות כספייקים כמו פולסים חשמליים, שהעיתוי והקצב שלהם נושאים את המשמעות. זו דרך חישוב שמתאימה יותר לתופעות טבעיות, כמו ראייה, שמיעה או תנועה.
בעבודתם של [18] נעשה ניתוח השוואתי של Rate Coding לעומת Direct Coding ברשתות נוירומורפיות, תוך הצבעה על כך ש Direct Coding מאפשר למידה מהירה ומדויקת יותר על חשבון חיסכון אנרגטי. במקביל, במאמר [17] על Time-to-First-Spike הציג כיצד מידע ניתן לייצוג מלא על סמך תזמון הקפיצה הראשונה בלבד, דבר המאפשר חישוב כמעט ללא צורך בקצב חזרתי.

2.3.3.2 ניהול הזיכרון

ניהול זיכרון מהווה מרכיב קריטי בכל מערכת חישובית הפועלת על נתונים רציפים או סדרתיים, במיוחד כאשר יש צורך לשמר הקשרים טמפורליים לאורך זמן. בחישוב הקלאסי, זיכרון מנוהל לרוב באופן דיסקרטי ומרוכז, למשל באמצעות רשומות, מבני נתונים, או שכבות זיכרון מלאכותיות בתוך רשתות עצביות, כגון מצבים פנימיים שמתעדכנים בכל שלב. תהליך זה דורש גישה ישירה לזיכרון ומימוש מפורש של מנגנוני קריאה וכתיבה, ולעיתים קרובות מתבסס על עדכון חזרתי או שימוש בשערים שמבקרים את זרימת המידע פנימה והחוצה [1].
לעומת זאת, בחישוב נוירומורפי ניהול הזיכרון מבוזר, מתמשך ונעשה כחלק אינהרנטי מהדינמיקה של הנוירונים עצמם. במקום לשמור את המידע כמבנה נפרד, הזיכרון מקודד בתוך מצבם הדינמי של הנוירונים והקשרים הסינפטיים ביניהם. הקלטים משפיעים על רמות הפוטנציאל החשמלי של הנוירונים, והתגובה של המערכת תלויה לא רק בקלט העדכני אלא גם במצב המצטבר מהעבר. תהליך זה, הדומה לאופן שבו מידע

נשמר ומעובד במערכת העצבים הביולוגית, מאפשר שמירה טבעית של תלות בטווחי זמן שונים, תוך חסכון במשאבים ואינטגרציה חלקה עם זמן.
הבדל זה מדגיש את השוני התפיסתי בין שני סוגי החישוב: בעוד החישוב הקלאסי שואף לייצוג ברור ומפורש של זיכרון, החישוב הנוירומורפי מטמיע את הזיכרון בתוך התנהגות המערכת עצמה, בתזמון הקפיצות, בעוצמתן ובשינויים מצטברים לאורך זמן. התוצאה היא מנגנון זיכרון גמיש ויעיל, המאפשר תגובה מותאמת הקשר מבלי להסתמך על שמירה אקטיבית או תפעול של מבנה זיכרון מרכזי.

2.3.3.3 חומרה

הבדלים מהותיים קיימים בין החומרה הדרושה למימוש חישוב קלאסי לבין זו המתאימה לחישוב נוירומורפי, והם נובעים מהשוני במודל החישוב הבסיסי. החישוב הקלאסי נשען על ארכיטקטורת פון נוימן, שבה קיימת הפרדה ברורה בין רכיבי עיבוד לזיכרון. מבנה זה דורש מעבדים כמו CPU ו-GPU שמבצעים רצף של פעולות חישוביות על מידע המאוחסן בזיכרון חיצוני [15].
לעומת זאת, חישוב נוירומורפי מתוכנן מתוך השראה ישירה ממבנה המוח, ולכן מיישם עקרונות של חישוב מבוזר, אסינכרוני ומבוסס אירועים. הדבר מתבטא גם ברמת החומרה: רכיבי העיבוד ממוקמים פיזית בצמוד לזיכרון, והם מופעלים רק כאשר מתקבל קלט רלוונטי. בנוסף, בעוד שמערכות קלאסיות מתוכננות לפעול בתדרים קבועים ובשימוש מלא של יחידות עיבוד, חומרה נוירומורפית מאופיינת בפעולה "ספרסית". לפיה מרבית הנוירונים והקשרים אינם פעילים רוב הזמן, אלא "מתעוררים" רק בתגובה לשינויים משמעותיים. עיקרון זה מוביל לא רק לחיסכון אנרגטי אלא גם ליכולת סקלבילית גבוהה, שבה ניתן להרחיב את מספר היחידות הפעילות מבלי להגדיל משמעותית את צריכת המשאבים הכוללת.

LMU .3

בפרק 2 סקרנו מבנה רשתות שונות לעיבוד נתונים תלויי זמן, והצגנו את המגבלות של רשתות נוירונים חוזרות מבוססות RNN ושל הטרנספורמרים, ראינו כי עולה הצורך בגישה חישובית חדשה שתאפשר התמודדות יעילה יותר עם נתונים סדרתיים מורכבים. יחד עם זאת, ראינו כיצד ניתן להשתמש ב-RNN בדרכים מגוונות, על מנת למקסם וליצור מודלים מתקדמים. בין אם על ידי הוספת מנגנוני שערים על מנת לשלוט על זרימת המידע בתא הזיכרון במודלים כמו GRU ו- LSTM. ראינו מודלים שמשלבים שכבות קונבולוציה ומנגנוני Attention, כמו הטרנספורמר, על מנת לתפוס תלות מרחבית ותליות ארוכות טווח. אך גם שיפורים אלו נתקעו במגבלות, ברובם חישוביות שהובילו למציאת מגוון פתרונות פורצי דרך ביניהם החישוב הנוירומורפי ומודלים מבוססים LMU.

באופן כללי, Legendre Memory Unit (LMU) הוא מודל חדשני לרשתות חוזרות, אשר שם דגש על שיפור שימור המידע לטווח הארוך תוך שמירה על יעילות חישובית גבוהה. בניגוד למודלים אחרים שהצגתי קודם לכן, אשר מתמודדים עם בעיות הגראדיאנט הנעלם / המתפוצץ בדרכים שדורשות משאבי חישוב משמעותיים, LMU מציע פתרון חישובי יציב וקל משקל, המבוסס פולינומים אורתוגונליים לשימור מידע לאורך זמן.

3.1 סקירה כללית של פתרונות חישוביים קיימים מבוססי LMU

כפי שראינו בפרק הקודם עיבוד נתונים תלוי זמן הוא תחום מרכזי בלמידת מכונה, העוסק בניתוח סדרות נתונים שבהן הסדר והדינמיקה של המידע קריטיים להבנה ולחיזוי. בניגוד לנתונים סטטיים, בהם כל דגימה נחשבת ללא תלות בקודמותיה, בסדרות נתונים כמו אותות דיבור, מדידות חיישנים, נתונים פיננסיים, ותנועת אובייקטים בזמן דורשות שיטת חישוב מיוחדות המאפשרות שימור מידע היסטורי ושימוש בו להבנת התנהגות עתידית.

לאורך השנים פותחו מספר גישות לעיבוד סדרות נתונים, כאשר כל אחת מהן מתמודדת עם האתגרים של למידת תלות סדרתית, שמירת זיכרון לטווח הארוך, וחישוב יעיל של קשרים בין שלבים שונים ברצף. המודלים המסורתיים, כגון רשתות RNN, סובלות ממגבלות משמעותיות. מודלים מתקדמים יותר [1] פותחו במהלך השנים כדי להתמודד עם חלק מהבעיות, אך הם עדיין מוגבלים ביכולתם ללכוד תבניות מורכבות לאורך זמן. ראינו כיצד מודלים כמו הטרנספורמר [14] הופיעו כתחלופות מבטיחות, אשר הציעו מנגנוני זיכרון יעילים יותר, יכולת למידה מקבילית וגמישות הסתגלותית לדינמיקה משתנה של נתונים, אך הם הביאו איתם אתגרים חישוביים חדשים.

3.1.1 פתרונות חישוביים קיימים

בשנים האחרונות, הופיעו מודלים חדשים המבוססים על גישות חישוביות יעילות יותר כמו LMU, HIPPO, מודלים מבוססי State-Space (SSM) והגרסאות הנוירומורפיות שלהם, אשר מספקים פתרונות אלטרנטיביים לאתגרים אלו.

LMU 3.1.1.1

מודל LMU שפותח בשנת 2019, במטרה לשפר את יכולת שימור המידע לטווח ארוך ברשתות חוזרות תוך מזעור השימוש בזיכרון ומשאבי חישוב [3]. המודל משתמש בפולינומים אורתוגונליים מסוג לג׳נדר כדי לייצג את ההיסטוריה של הקלט בצורה קומפקטית ויעילה יותר בהשוואה ל LSTM ו- GRU. מאז הצגתו הוא הורחב והותאם לשימוש במערכות הנוירומורפיות, שולב עם מודלים מבוססי State-Space כגון Mamba [6] והותאם למשימות זמן אמת ומכשירי קצה [8].

HIPPO 3.1.1.2

High order Polynomial Projection Operator – HIPPO מציע שיטה מתמטית לשימור מידע בצורה אופטימלית, באמצעות היטל של פונקציות על בסיס פולינומים אורתוגונליים [5]. שיטה זו מאפשרת שמירה דינמית של המידע ההיסטורי באופן שמקטין את הצורך באחסון ישיר של כל השלבים הקודמים, ובכך משפרת את היכולת של המודל לזכור הקשרים ארוכי טווח מבלי להכביד על משאבי החישוב. הגישה הכללית של HIPPO מאפשרת שימוש במגוון פולינומים אורתוגונליים, בעוד LMU מתמקד בפולינומים מסוג לג׳נדר.

State Space Models 3.1.1.3

SSM [10] מציעים חלופה למודלים מבוססי Attention כמו טרנספורמרים, בכך שהם מאפשרים חישוב לינארי של קשרי התלות ברצף, במקום חישוב ריבועי. מודל Mamba שהוצג לאחרונה, משתמש במנגנון מבוסס מטריצות מעבר כדי לשמר מידע ארוך טווח תוך צריכת זיכרון יחסית מינימלית [6]. LMU נחשב למעשה מימוש ספציפי של מודל SSM, שבו נעשה שימוש בפולינומים אורתוגונליים מסוג לג׳נדר לניהול הזיכרון. שניהם חולקים את הבסיס התיאורטי של מערכות לינאריות בזמן רציף LTI.

3.1.2 פתרונות חישוביים קיימים – היבט נוירומורפי

החישוב הנוירומורפי מהווה פרדיגמה חדשה בלמידת מכונה, המתבססת על חיקוי המבנה והתפקוד של המוח האנושי לשם פיתוח מערכות חסכוניות באנרגיה ועילות חישובית. גישה זו נשענת על Spiking Neural Networks, אשר מבצעות עיבוד מבוזר ואסינכרוני בדוגמה לרשתות עצביות ביולוגיות, ובכך מאפשרות צריכת אנרגיה מופחתת ושיפור יכולות עיבוד בסביבות זמן אמת.

3.1.2.1 LMU ו- SNN שילוב נוירומורפי טבעי

מחקרים הראו כי ניתן להמיר את LMU למודל מבוסס SNN תוך שימוש בנוירונים מסוג LIF, אשר מדמים את מנגנוני העיבוד העצביים במוח. במחקר שנעשה במסגרת ICANN ב- 2024, הוצג מודל L2MU, אשר מהווה גרסה נוירומורפי טבעית של LMU. המודל פותח עם מנגנון קידוד ייחודי המאפשר לו לעבוד עם נתוני חיישנים בזמן אמת. ללא צורך בהמרת המידע לפורמט מותאם מראש מה שמאפשר אינטגרציה עם מערכות IoT ומחשוב קצה [8].

3.1.2.2 שילוב בין SSM לSNN

בשנים האחרונות הוצעו מספר מודלים חדשנים המשלבים בין מנגנוני SSM ומודלים מבוססי ספייקים, במטרה לשפר את יכולת ניתוח האירועים דלילים בזמן אמת. בין המודלים שפותחו, SpikMamba המתמקד בניתוח נתונים המתקבלים מחיישנים מבוססי אירועים וזיהוי פעילות אנושית. היתרון המרכזי הוא היכולת לנצל את הספייקים לחישוב מקבילי של תלות גלובלית, תוך שמירה על מאפייני זמן אמת. מחקרים מצביעים על כך שSpikMambaw צורך עד 80% פחות אנרגיה בהשוואה למודלים סטנדרטיים כמו טרנספורמר, תוך שמירה על דיוק חיזוי גבוה [9].

השילוב בין LMU, SNN ו- SSM מאפשר פתרונות שלא ניתן היה להשיג בחישוב קלאסי, תוך שמירה על צריכת אנרגיה נמוכה יותר, שימור מידע לטווח ארוך בזמן אמת והתאמה למערכות IoT ומחשוב קצה. מה שלא היה ניתן לבצע עם מודלים כמו Transformer ו- RNN.

3.2 LMU – עקרונות פעולה ויישום

מודל ה- LMU פותח במטרה לשפר את יכולות עיבוד הנתונים הסדרתיים ברשתות חוזרות, תוך שמירה על תלות ארוכת טווח ביעילות חישובית גבוהה. המודל עושה שימוש בפולינומי לג׳נדר כדי לשמר מידע לאורך זמן, מה שמאפשר לו לייצג את ההיסטוריה של קלטים קודמים בצורה קומפקטית ויעילה [3]. ליבת ה- LMU מבוססת מערכת לינארית בלתי משתנה בזמן LTI אשר מקבלת כקלט את האות הרציף וממירה אותו לייצוג תחום תדר באמצעות פולינומים אורתוגונליים. מערכת זו מגדירה מצב דינמי המיוצג על ידי וקטור זיכרון m(t), אשר מתעדכן בהתאם לחוקים שמגדירות מטריצות A ו- B, עליהם ארחיב בהמשך. הוקטור הזה מאפשר לרשת לשמור מידע מרגעי זמן קודמים ללא תלות במרחקם מהנקודה הנוכחית.
בבסיס ה- LMU קיימת נוסחה המבוססת על מטריצות A ו-B אותן הזכרנו קודם לכן, ומטרתה לייצג את ההיסטוריה של האות u(t) בחלון זמן באורך θ באופן קומפקטי ויציב לאורך זמן:
משוואה 9:

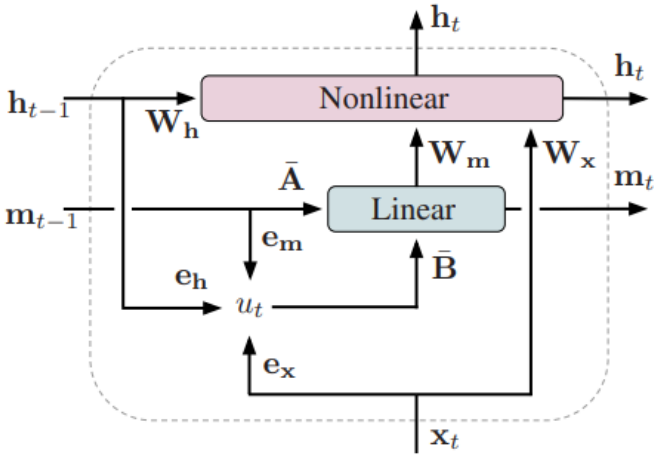
$$\theta m(t) = Am(t) + Bu(t)$$

m(t) – הוא וקטור זיכרון בגודל d, שמכיל היטל של ההיסטוריה של u על בסיס פולינומי לג׳נדר.
θ – מייצג את אורך חלון הזמן לשמירת ההיסטוריה.
מטריצות A ו- B הם מקדם הזמן והקלט של מערכת הזיכרון, שמאפשרות לשמר את ההיסטוריה של הקלט בצורה קומפקטית על בסיס פולינומי אורתוגונלי.

LMU משתלב בקלות במערכות חישוביות נוירומורפיות, שכן הארכיטקטורה שלו ניתנת למימוש באמצעות רשתות מבוססות ספייקים, מה שמקנה לו יתרון משמעותי בפלטפורמות בעלות צריכת חשמל נמוכה. למעשה, גרסאות נוירומורפיות של LMU הוטמעו בהצלחה בחומרה כמו Loihi של אינטל ובמערכות ייעודיות אחרות לאימות יישומים מבוססי עיבוד אותות רציף [3].

3.2.1 ארכיטקטורה בסיסית

אפשר להסתכל על LMU כעל מנגנון זיכרון חדש ברשתות נוירונים מסוג RNN. המטרה העיקרית שלו הוא להתמודד עם בעיות זיכרון לטווח ארוך. הארכיטקטורה המוצעת היא תא זיכרון חדש, שמתחזק בצורה דינמית מידע לאורך חלונות זמן ארוכים. תוך שימוש במבנה מתמטי ייחודי, שכולל פתירת משוואות דיפרנציאליות שמכילות מידע על העבר של הקלט. מבנה המשוואות מייצר בסיס אורתוגונלי של פולינומים בשם פולינומי לג׳נדר שמייצגים את חלון הזמן.
ה- LMU מציעה גישה חלופית המבוססת על מערכת ליניארית בלתי משתנה בזמן LTI בשילוב עם שכבות לא ליניאריות לחישוב תהליכים מורכבים לאורך זמן. יתרון זה מאפשר למודל לשמר מידע רלוונטי למשך פרקי זמן ארוכים מבלי לסבול מהתופעה הידועה כבעיית הגרדיאנט הנעלם.
הארכיטקטורה של LMU כפי שניתן לראות באיור 11, מבוססת על ארבע שכבות עיקריות: שכבת הקלט, שכבת הזיכרון הליניארית, שכבת המצב החבוי, ושכבת הפלט. כל שכבה ממלאת תפקיד ייחודי בתהליך העיבוד, תוך שמירה על איזון בין יעילות חישובית לדיוק חיזוי.



איור 11 : ארכיטקטורת LMU [3]

איור 11 : האיור מציג את מבנה שכבת ה- LMU בזמן t, שבה וקטור המצב החבוי h_t מקיים אינטראקציה עם וקטור הזיכרון m_t דרך שני רכיבים עיקריים : מערכת לינארית (המתוארת על ידי המטריצות $\bar{A}$ ו- $\bar{B}$), ושכבת עיבוד לא לינארית המחוברת דרך המשקלים W_m , W_h , W_x הזיכרון מייצג חלון זמן של הקלט u_t, אשר מוקרן על בסיס הפולינומים של לג'נדר לצורך ייצוג קומפקטי ואפקטיבי של ההיסטוריה של הקלט.

3.2.1.1 שכבת הקלט

שכבת הקלט אחראית על קידוד הנתונים הנכנסים, כך שניתן יהיה לעבד אותם בזיכרון ובמצב החבוי של המודל. כל קלט x_t מתקבל בצעד זמן t ועובר דרך מנגנון קידוד, המשלב את הקלט עם המידע ההיסטורי של המערכת :

משוואה 11 :

$$u_t = e_x^T x_t + e_h^T h_{t-1} + e_m^T m_{t-1}$$

כאשר e_x, e_h, e_m הם וקטורי קידוד נלמדים, אשר תפקידם להקרין את הקלט למרחב הזיכרון ולשמר תכונות רלוונטיות לאורך זמן.

3.2.1.2 שכבת הזיכרון הלינארית

שכבה זו היא אחת התכונות המרכזיות של LMU, והיא משמשת לאגירת מידע לאורך sliding window. הייחודיות שלה טמונה בכך שהיא אינה מבוססת על מנגנוני שערים, אלא על משוואה דינמית לינארית המגדירה את עדכון המידע בזיכרון :

משוואה 12 :

$$m_t = \bar{A} m_{t-1} + \bar{B} u_t$$

כאשר $\bar{A}$ ו- $\bar{B}$ הן מטריצות קבועות אשר מבטיחות אופטימיזציה של הייצוג הפולינומי של המידע הנכנס. מטריצות אלו מבוססות על פולינומים של לג׳נדר, שמאפשרים ייצוג קומפקטי של ההיסטוריה של הקלט תוך שמירה על אורתוגונליות המידע המאוחסן.
השימוש בזיכרון לינארי מעניק למודל יכולת להתמודד עם תלות ארוכת טווח בצורה חסכונית במשאבים חישוביים, זאת בניגוד למודלים מסורתיים אשר דורשים חישוב שערים מורכב ומתמודדים עם דעיכת מידע לאורך זמן.

3.2.1.3 שכבת המצב החבוי

שכבת המצב החבוי אחראית על חישוב פונקציות לא לינאריות באמצעות שילוב של המידע מהזיכרון עם מידע נוסף שהתקבל מהקלט ומהמצב הקודם של הרשת. הפעולה בשכבה זו מוגדרת על ידי המשוואה:
משוואה 13:

$$h_t = f(W_x x_t + W_h h_{t-1} + W_m m_t)$$

כאשר f היא פונקציה לא לינארית כגון tanh או ReLU ו- W_m , W_x , W_h הן מטריצות למידה האחראיות על שילוב הקלטים השונים למצב חבוי חדש.
באמצעות גישה זו LMU מצליחה להפריד בין גודל הזיכרון d לבין גודל המצב החבוי n, בניגוד למודלים מסורתיים שבהם שני הגדלים הללו קשורים זה לזה. הפרדה זו מאפשרת גמישות גדולה יותר בהתאמת המודל לבעיות שונות, שכן ניתן להגדיל את קיבולת הזיכרון מבלי להגדיל את כמות החישוב הלא לינארי הנדרש.

3.2.1.4 שכבת הפלט

שכבת הפלט ממירה את המידע המעובד לתוצאה הסופית המתאימה למשימה המבוקשת. בדומה למודלים אחרים של למידת מכונה, הפלט מתקבל באמצעות שילוב לינארי של המצב החבוי. לקיחת המצב החבוי h_t ושימוש במיפוי לינארי פשוט כדי להפסיק את הפלט הסופי:

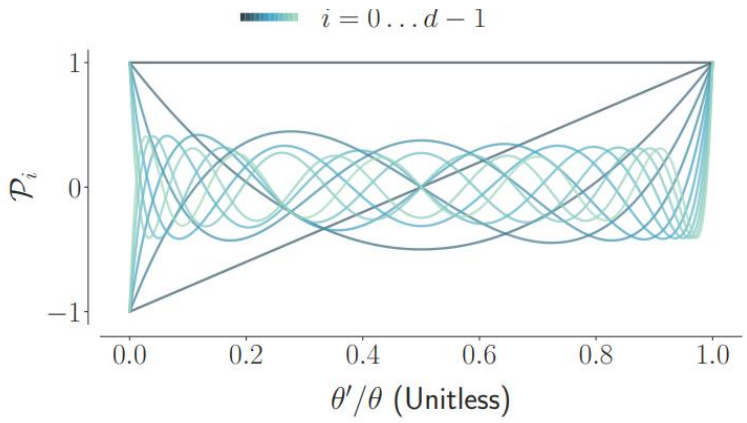
$$W_o h_t = y_t$$

כאשר W_o היא מטריצת משקולות נלמדת, שמבצעת את ההתאמה בין המצב החבוי לפלט הרצוי. שכבת הפלט של LMU מתפקדת בדומה לשכבת Softmax במודלים סטנדרטיים של עיבוד סדרות זמן למטרת סיווג.

3.2.1.5 וקטור הזיכרון m_t

הפרמטר d ב- LMU מייצג את גודל הזיכרון של הרשת, כלומר את מספר הממדים שבהם נשמרת ההיסטוריה של הקלט. כפי שניתן לראות באיור 12, LMU עושה שימוש ב- d להשלכת ההיסטוריה של הקלט על בסיס פולינומי של לג׳נדר, אשר מאפשר שימור מידע לאורך זמן בצורה קומפקטית ויציבה חישובית.
בניגוד למודלים שראינו קודם לכן כמו LSTM ו- GRU, שבהם הזיכרון הוא חלק בלתי נפרד מהמצב החבוי, דבר שמוביל לכך שגידול בזיכרון מחייב הגדלת כמות החישוב הלא לינארי.

ניתן לראות ש- LMU מפרידה בין גודל הזיכרון d לבין גודל המצב החבוי, כך שניתן להגדיל את קיבולת הזיכרון מבלי להגדיל את מורכבות החישוב הלא לינארי.



איור 12 : פולינומים של לג'נדר עבור d=12 [3]

איור 12 : האיור מציג פולינומים של לג׳נדר עבור d=12 פולינומים אלו מהווים את הבסיס שעליו מתבצע ייצוג ההיסטוריה של הקלט. התאמת דרגת הפולינומים d מגדירה עד כמה ניתן לשמר תדרים גבוהים יותר בקלט.

הפרמטר d מייצג את מספר הממדים המשמשים לייצוג ההיסטוריה של הקלט. כל ממד ב- m_t, משמש לשימור רכיב אורתוגונלי של הקלטים הקודמים על פני חלון זמן מסוים.

ניתן לכתוב את הקלט $u(t)$ כסכום משוקלל של d הפולינומים הראשונים בסדרה:

<u>משוואה 14:</u>

$$u(t) \approx \sum_{i=0}^{d-1} c_i P_i(t)$$

$P_i(t)$ – פולינומי לג׳נדר מסדר i.

c_i – מקדמים

השגיאה בקירוב המידע של הזיכרון ב- LMU תלויה ישירות בפרמטר d, כאשר d=1 הזיכרון של LMU יכול לתפקד כמו יחידת LSTM פשוטה ללא מנגנוני שערים. משמר מידע אך גם שוכח אותו בהדרגה בקצב תלוי בפרמטר הזמן θ.

ככל ש d גדול יותר, כך ניתן לייצג את היסטוריית הקלט במדויק יותר ולהימנע מאיבוד מידע לאורך זמן. אם d=1 המערכת יכולה לשמר רק ממוצע של הקלטים הקודמים, עבור d=2 נוכל לשמר מידע על מגמות לינאריות, ככל ש-d גדל, כך המודל שומר היסטוריה עמוקה ומדויקת יותר של הקלטים הקודמים.

לכן ביישומים כמו NLP וחיזוי סדרות זמן, LMU יכול לספק ביצועים טובים יותר מרשתות חוזרות מסורתיות תוך הפחתה משמעותית של העומס החישובי, הודות להפרדה בין אחסון מידע לבין חישוב לא ליניארי.

3.2.2 HIPPO כמסגרת תיאורטית

כמו שראינו עד עכשיו, אחד הקשיים העיקריים בלמידה ממידע שמגיע לפי סדר הוא איך לשמור מידע מהעבר בצורה מצטברת, ובו בזמן להמשיך לעבד גם את המידע החדש שמגיע. מסגרת HIPPO (High order Polynomial Projection Operators), מספקת גישה עקרונית לבעיה זו באמצעות היטל פונקציות היסטוריות על בסיסים פולינומים אורתוגונליים.
LMU נבנה ישירות על העקרונות של HIPPO, תוך שימוש בפולינומים של לג׳נדר כבסיס ייצוג הזיכרון.
בפרק זה אתאר את מסגרת HIPPO, את העקרונות הבסיסיים, את הקשר ל- LMU ואת השיפורים שהובילו ליעילות חישובית גבוהה יותר.

3.2.2.1 עקרונות היסוד של HIPPO

למידת מכונה על נתונים סדרתיים מחייבת שימור מידע על הקלט הקודם על מנת לבצע תחזיות או להבין תבניות לאורך זמן. עם זאת, כיוון שההיסטוריה של הקלט אינה ניתנת לשמירה במלואה עקב מגבלות זיכרון וחישוב, יש צורך במנגנון דחיסה אופטימלי. HIPPO פותר בעיה זו על ידי שימוש בהיטלים אורתוגונליים של פונקציות קלט על בסיס פולינומים כפי שניתן לראות באיור 13.
מהות של HIPPO טמונה בפתרון אופטימלי לבעיה טבעית של קירוב פונקציות באופן מקוון, בהינתן מדד הקובע את חשיבותו של כל צעד זמן בעבר.

מספר עקרונות מפתח מגדירים את HIPPO:

מדד משתנה בזמן $\boldsymbol{\mu(t)}$: בבסיס עומד הרעיון של מדד $\mu(t)$ המגדיר את חשיבותו של כל רגע זמן בעבר. מכיוון שמרחב הפונקציות הוא עצום ולא ניתן לשמור אותו במלואו יש לדחוס את ההיסטוריה. על מנת לדעת אם הקירוב טוב, מודדים את ההבדל בין הפונקציה המקורית לפונקציה המקוצרת. המדד μ הוא מדד הסתברותי שאומר לנו איזה חלקים של הפונקציה חשובים יותר לשימור בקירוב, המדד קובע כיצד לשקלל את החלקים השונים של הפונקציה. בחירה מושכלת של μ מובילה להתנהגויות שונות מבחינה איכותית של מנגנון הזיכרון, לדוגמה: אם μ נותן משקל גבוה לזמנים מוקדמים, זה אומר שאנחנו רוצים לזכור אותם יותר.

הטלה אופטימלית $\boldsymbol{proj_t}$: הרעיון הוא בכל רגע זמן t, לקחת את הנתונים שהתקבלו עד כה ולחפש ייצוג פשוט שלהם בעזרת פולינומים.
בוחרים בסיס של פונקציות פולינומיות, שהן פולינומים שמייצגים מידע בצורה קומפקטית כמו פולינומים של לג׳נדר ב- LMU [3]. לאחר מכן, מטילים את הפונקציה f(t) על בסיס הפולינומים הנבחר, על יד חישוב המכפלה הפנימית ומקבלים את המקדמים שמייצגים את המידע.
אנחנו מודדים את איכות הקירוב באמצעות מכפלה פנימית, שמודדת כמה שני ייצוגים דומים אחד לשני. הקירוב המקוון מחפש פונקציה פשוטה g(t) מתוך מרחב מסוים G שתגרום לשגיאה להיות מינימלית.

משוואה 15:

$$\left\| f_{\leq t} - g^{(t)} \right\|_{\mathrm{L2}\left(\mu^{(t)}\right)}$$

$f_{\leq t}$ – הנתונים שהתקבלו עד זמן t.

$g^{(t)}$ – פונקציה פשוטה שמנסה לייצג את הנתונים.

$\mathrm{L2}\left(\mu^{(t)}\right)$ – מדד השגיאה שמתחשב במשקל של הנתונים לאורך זמן.

המטרה היא למצוא את $g^{(t)}$ כך שההבדל בין המידע האמתי יהיה קטן ככל האפשר. כלומר אנחנו צריכים לבחור פונקציה פשוטה g שמייצגת את הנתונים שהצטברו בצורה טובה.

התמרת הפונקציה למקדמים $coef_t$: במקום לשמור את כל הנתונים, אנחנו מיצגים אותם באמצעות מקדמים שמתארים עד כמה כל פולינום מתאים להיסטוריה. אפשר לייצג את ההיסטוריה בתור:

משוואה 16:

$$f_{\leq t} \approx g^{(t)} = \sum_{n=0}^{N-1} c_n(t) g_n^{(t)}$$

ואז במקום לשמור את כל הנתונים, אנו שומרים רק את המקדמים: $c(t) = [c_0^{(t)}, c_1^{(t)}, c_2^{(t)}, \ldots , c_{N-1}^{(t)}]$

עדכון מקדמים ו- Discretization: לאורך הזמן, הנתונים מתעדכנים וצריך לעדכן את המקדמים באופן דינמי. במקום לחשב הכל מחדש, משתמשים במשוואה דיפרנציאלית המתארת איך המקדמים משתנים עם הזמן:

משוואה 17:

$$\frac{d}{dt} c(t) = A(t)c(t) + B(t)f(t)$$

או בצורה רקורסיבית:

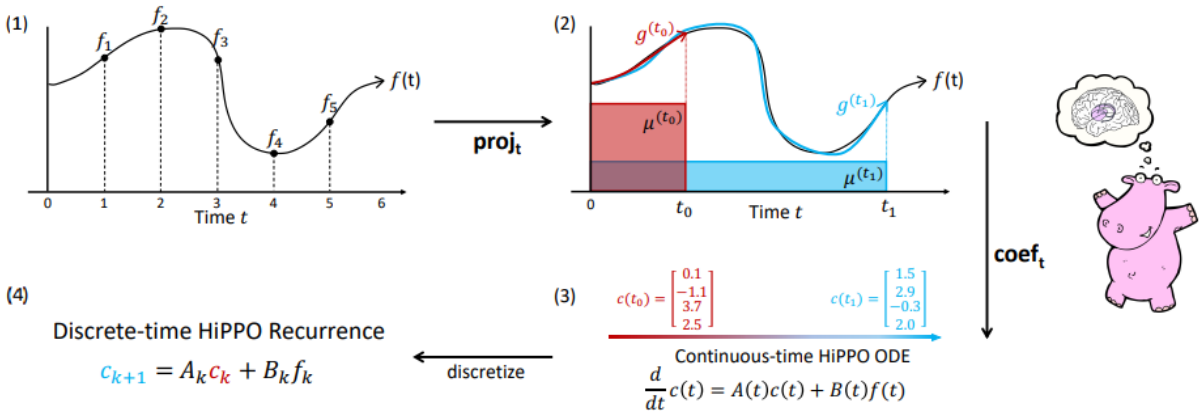
$$c_t = A_t c_{t-1} + B_t f_t$$

A(t) – מטריצה המתארת את שינוי המקדמים עם הזמן.

B(t) – מטריצה שמתארת איך נתונים חדשים נכנסים לחישוב.

f(t) – הנתון החדש שמתווסף.

בפועל, אנחנו לא עובדים עם נתונים רציפים, אלא עם נקודות מדידה בזמן. נדרש במודל יכולת לעדכן את הייצוג בצורה מהירה ויעילה, בלי לשמור את כל הנתונים הקודמים. במקרה הזה אנחנו צריכים לחשב את המקדמים בצורה חוזרת לפי משוואה 17.



איור 13 : שלבים במסגרת HiPPO [5]

איור 13 : האיור מציג חלוקה את ההתמרה ל4 שלבים מרכזיים, בשלב הראשון, לקיחת הנתונים שהתקבלו עד כה והשלכה שלהם על בסיס פולינומים נבחר (למצוא התאמה לפונקציות פשוטות יותר). בשלב השני, שמירה רק של מקדמי ההתאמה במקום את כל הנתונים. בשלב השלישי, עדכון המקדמים בכל פעם שמתווסף מידע חדש באמצעות משוואות דינמיות. בשלב הרביעי, שימוש בגרסה בדידה על מנת לנהל את הזיכרון ולבצע חישובים בזמן אמת.

3.2.2.2 הקשר בין RNN ל- HiPPO

כאשר אנחנו משתמשים בפולינומים מסדר נמוך, ומסתפקים בהיטל פשוט מאוד למשל של הסדר הנמוך ביותר N=1. משוואת הדיסקרטיזציה [4] הופכת להיות פשוטה מאוד.

מאחר ש A=B=1 נקבל:

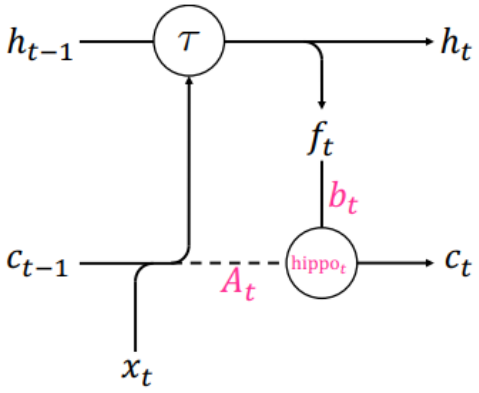
<u>משוואה 18:</u>

$$c(t+\Delta t) = c(t) + \Delta t\big(-Ac(t) + Bf(t)\big) = (1-\Delta t)c(t) + \Delta t f(t)$$

פירוש הדבר הוא שהמצב החדש תלוי ישירות בקלט החדש בשילוב עם המצב הקודם עם משקלים פשוטים. רשתות RNN עם שערים כמו GRU שראינו קודם לכן [1] מתנהגות באופן דומה למקרה זה. ברשתות GRU העדכון מתבצע בשילוב לינארי של מצב קודם וכניסה חדשה, בדיוק כמו במשוואה לעיל. ההבדל המרכזי הוא שרשתות GRU משתמשות במנגנוני שערים שמאפשרים לרשת לבחור איזה חלקים של המידע לשמור. לכן, במקרה של HiPPO עם N=1 אנו מקבלים עדכון פשוט כמו GRU, אך ללא צורך בשערים מתוחכמים. HiPPO פועלת באופן טבעי יותר ומשיגה תוצאה דומה באמצעות קריטריונים מתמטיים מבוססי פולינומים.

<u>אינטגרציה של HiPPO בתוך מודל RNN בסיסי:</u>

HiPPO יכול לשמש גם כמפעיל זיכרון דינמי בתוך מודל RNN בסיסי כפי שניתן לראות באיור 14. השילוב מתבצע על ידי החלפת מנגנון הזיכרון המסורתי (כגון מצב חבוי רגיל או מנגנוני שערים כמו ב- LSTM) במערכת דינמית לינארית לשימור מידע. במקום לשמור את המצב החבוי הישן משתמשים באופרטור $hippo_t$ שמטיל את ההיסטוריה של f_t על בסיס המדד שנבחר.



איור 14 : אינטגרציה של HiPPO בתוך מודל RNN בסיסי [5]

איור 14 : התרשים מתאר אינטגרציה של HiPPO בתוך מודל RNN בסיסי, שבו HiPPO משמש כמפעיל זיכרון דינמי המשפר את יכולת הרשת לשמר מידע לאורך זמן.

$h_{t-1} \rightarrow h_t$: מסלול המציין את זרימת המידע של המצב החבוי הקלאסי ב- RNN. המעבר דרך τ מסמן עיבוד לא לינארי סטנדרטי, כמו פונקציית אקטיבציה או משקלים נלמדים.

$c_{t-1} \rightarrow c_t$: במסלול זה $hippo_t$ משמש כמפעיל זיכרון, אשר מקרין את המידע ההיסטורי של f_t על סמך מדד הזיכרון שנבחר.

מטריצת A_t קובעת את המעבר של המידע בזיכרון, ו- b_t מייצג עדכון שמקורו בקלט החדש.

HiPPO מציע מנגנון זיכרון רציף שבו c_t מתעדכן על בסיס פולינומים אורתוגונליים, מה שמאפשר שמירת מידע לזמן ארוך יותר ללא דעיכת גראדיאנט. בנוסף ההחלפה של מנגנוני השערים במערכת לינארית, המתעדכנת באמצעות מטריצות A_t , B הופכת את הזיכרון למדויק ויציב יותר.

3.2.2.3 יישום HiPPO ברשתות נוירונים מודרניות

לאחר פיתוחה, נמצא כי HiPPO מספקת תשתית מתמטית חזקה לשיפור מנגנוני הזיכרון ברשתות נוירונים מודרניות, תוך הפחתת בעיות כמו דעיכת גרדיאנט ואיבוד מידע. יישום HiPPO ברשתות נוירונים מתמקד בעיקר באינטגרציה בתוך רשתות חוזרות (RNN) ובהרחבת היכולת של LMU. בפרק זה אסקור כיצד HiPPO ממומש ברשתות נוירונים, ואתמקד בשלושה מדדי היישום המרכזיים : HiPPO-LegT, HiPPO-LagT ו- HiPPO-LegS. ואבחן את ההשפעה של מדדים שלו על ביצועי הרשת.

HiPPO ממומש ברשתות נוירונים חוזרות על ידי החלפת מנגנון הזיכרון המסורתי במערכת לינארית דינמית המתעדכנת לפי אורתוגונליזציה של פולינומים. למשל ב- GRU ו- LSTM במקום להשתמש בשערים לשליטה על זרימת המידע [1], המודל פועל כמערכת לינארית מהצורה שמתוארת במשוואה 17. שיטה זו מאפשרת עדכון יעיל יותר של הזיכרון בהשוואה לשיטות מסורתיות, ללא הצורך בשימוש בשערים לינאריים.

HiPPO-LegT : מדד המשתמש בפולינומים של לג'נדר על מנת לייצג חלון קבוע של הקלטים האחרונים. הוא פועל על פי הכלל :

משוואה 19:

$$\mu^{(t)}(x) = \frac{1}{\theta}\mathbb{I}_{[t-\theta,t]}(x)$$

המדד מחלק משקל שווה לכל נקודות הזמן בתוך חלון הזמן האחרון. חלון הזמן מיוצג על ידי הפרמטר θ. שמציין, כמה זמן אחורה אנחנו מסתכלים או במילים אחרות את גודל חלון הזמן הנדרש. כל הנקודות בטווח הזמן $[t, t-\theta]$ מקבלות משקל אחיד. כל האירועים בטווח הזה חשובים באותו המידה.
מדד זה יעיל במשימות בהן יש מחזוריות או מבנה נתונים קבוע בזמן.

HiPPO-LagT: מדד המבוסס פולינומי לגואר, המאפשרים עדכון מידע עם משקל עדיפות לקלטים האחרונים, תוך הפחתה הדרגתית של תרומת המידע הישן.
הוא פועל לפי הכלל:
משוואה 20:

$$\mu^{(t)}(x) = e^{-(t-x)}\mathbb{I}_{(-\infty,t]}(x) = \begin{cases} e^{x-t}, & x \le t \\ 0, & x > t \end{cases}$$

המדד המשתמש בפונקציה מערכית מתפרקת, ככל שנקודת זמן קרובה יותר להווה, היא מקבלת משקל גבוה יותר, אירועים רחוקים בזמן יקבלו משקל נמוך יותר. המדד מתאים למידע שמשתנה לאורך זמן ולא דורש חלון זמן קבוע.
מדד זה יעיל במשימות בהם תלות המידע נחלשת באופן טבעי ככל שהזמן מתקדם.

כזכור במשוואה 9 הראנו את היחס הישיר בין A, B ומקדמי הפולינומים c.
הערכים של A ו- B לכל מדד:
משוואה 21:

עבור LegT:

$$A_{nk} = \frac{1}{\theta}\begin{cases} (-1)^{n-k}(2n+1), & n \ge k \\ 2n+1, & n \le k \end{cases}, \qquad B_n = \frac{1}{\theta}(2n+1)(-1)^n$$

עבור LagT:

$$A_{nk} = \begin{cases} 1, & n \ge k \\ 0, & n < k \end{cases}, \qquad B_n = 1$$

עבור מדד LegT שמייצג חלון זמן אחיד, המשוואות מספקות עדכון זיכרון תוך הנחה שהנתונים בטווח θ נשמרים במלואם. זה דומה לדרך שבה פועל עדכון LMU [3].
עבור מדד LagT שמייצג דעיכה מערכית, במקום לשמור את כל ההיסטוריה משקל הנתונים פוחת עם הזמן והנתונים האחרונים משפיעים יותר על ההחלטות.

HiPPO-Legs: במקום להשתמש במדדים סטטיים כמו שראינו עד כה, LegT שמשקלל אחיד לזמן מוגדר או LagT שמתעדף זמן אחרון. המדד החדש שמציעים LegS מעניק משקל אחיד לכל ההיסטוריה מההתחלה עד לזמן נוכחי t .

המדד פועל לפי הכלל:

משוואה 22:

$$\mu^{(t)}(x) = \frac{1}{t}\mathbb{I}_{[0,t]}(x)$$

זה אומר שכל הנתונים מהעבר מקבלים חשיבות יחסית שווה, אך ההשפעה מתפשטת על טווח הזמן שהולך וגדל. כתוצאה מכך ככל שהזמן מתקדם, **המדד מתאים את עצמו אוטומטית**, וכך מונע שכחת נתונים מוקדמים מידי.

כזכור במשוואת הדיסקרטיזציה 17 הראנו את הקשר בין A ל- B שמתאר איך המקדמים משתנים עם הזמן. במדד החדש נעשה שינוי במשוואות:

משוואה 23:

$$\frac{d}{dt}c(t) = -\frac{1}{t}Ac(t) + \frac{1}{t}Bf(t)$$

הערכים של A ו-B למדד הם:

$$A_{nk} = \begin{cases} (2n+1)^{\frac{1}{2}}(2k+1)^{\frac{1}{2}}, & n > k \\ n+1, & n = k \\ 0, & n < k \end{cases} \qquad \mathrm{B}_n = (2n+1)^{\frac{1}{2}}$$

עדכון הזיכרון מתבצע לפי משוואה 23, התלות ב $\frac{1}{t}$ מראה שהעדכון נעשה באופן יחסי לזמן. כך שהמידע החדש משתלב נכון מבלי לשכוח מידע ישן.

כאשר מבצעים דיסקרטיזציה של משוואה 23 נקבל עדכון בדיד:

משוואה 24:

$$c_{k+1} = \left(1 - \frac{A}{k}\right)c_k + \frac{1}{k}Bf_k$$

החלוקה ב– k מבטיחה שההשפעה על נתונים ישנים לא נעלמת, אלא מדוללת עם הזמן. יש פה אי תלות בסקאלת הזמן, אם נמתח או נדחוס את קלט הנתונים, הקירוב של המודל יישאר מדויק והפלט יותאם בהתאם.

3.2.2.4 השוואות למודלים אחרים

HiPPO פורץ דרך במשימות עיבוד רצפים ארוכי טווח. השוואת הביצועים נעשתה על המאגר psMNIST המשמש למשימת סיווג הספרות במערך MNIST בגרסה מאתגרת שלו. גרסה זו נעדה לבחון את היכולת של מודלים נשנים להתמודד עם תלויות טמפורליות מורכבות לאורך זמן [19]. בנוסף נמצא כי שיטת LegS הראתה את הביצועים הטובים ביותר מכל השיטות.

שיטת LegT משיגה ביצועים קרובים, אך ביצועיה נופלים כאשר הפרמטר θ אינו נכון.

שיטת LagT הצליחה אבל לא מספקת ביצועים מיטביים בכל התנאים. שיטת RNN אחרות כמו GRU ו-LSTM התמודדו פחות טוב עם המשימה, מכיוון שהן מתקשות להתמודד עם תלות ארוכת טווח בסדר מעורבב.

המודל השיג דיוק של 98.3%. מה שממקם אותו מעל המודלים מסורתיים כמו GRU שהשיג 93% , LSTM עם 95.1% ואפילו התעלה מעל הטרנספורמר הבסיסי עם 97.9%.

יתרון נוסף של HiPPO הוא יעילותו החישובית. בעוד שמודלים מסורתיים שהצגנו קודם לכן דורשים חישובים מורכבים עם סיבוכיות של $O(N^2)$ או יותר, HiPPO-Legs מבצע עדכון זיכרון עם סיבוכיות $O(N)$ בלבד, דבר המאפשר עיבוד רצפים ארוכים פי 10 מהר יותר ממודלים חוזרים מסורתיים.

לסיכום HiPPO ,מספק פתרון תאורטי ומעשי רב עוצמה לניהול זיכרון ברשתות נוירונים, הוא משלב דיוק גבוה יותר, גמישות טובה יותר, והתאמה רחבה למשימות למידת מכונה.

3.2.3 SSM – ניהול דינמי של מידע בסדרות זמן

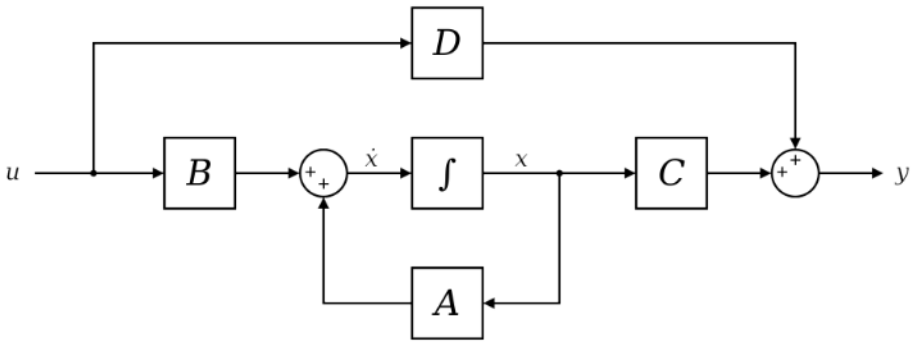
הצורך בהבנה ובניתוח של מידע המגיע לאורך ציר הזמן הוא אחת הבעיות המרכזיות במדעי המחשב ובתחום הלמידה העמוקה בפרט. בפרקים הקודמים בעבודה סקרתי את מודל הLMU, שפותח על מנת לייצג מידע רציף לאורך זמן בצורה מדויקת ויעילה, תוך התבססות על פולינומים אורתוגונליים. ראינו כי LMU בנוי ישירות על העקרונות של HiPPO, תוך שימוש בפולינומים של לג׳נדר כבסיס ייצוג הזיכרון.
בהמשך ישיר לכך, בפרק הנוכחי נעסוק במודלים מבוססי מרחב-מצב (State Space Models) שהתפתחו מתוך עולם תורת הבקרה ומשיקים באופן טבעי ל- LMU. בדומה ל- LMU גם מודלי ה- SSM עוסקים בייצוג דינמי של מידע זמני, אך מרחיבים את ההסתכלות למגוון רחב יותר של ייצוגים ושיטות [10]. בין המודלים הבולטים בתחום זה בשנים האחרונות נמצא מודל ה- S4, המשלב בין הגישות השונות, ובכך מאפשר להתמודד עם מידע טמפורלי ארוך במיוחד בצורה חסכונית יעילה.

בפרק נתמקד ביסודות התיאורטיים של מודלי SSM, נסקור את ההתפתחויות המרכזיות בתחום זה, ונעמיק באופן שבו הם מאפשרים ייצוג דינמי יעיל תוך השוואה והקבלה למודל ה- LMU שהוצג בפרקים הקודמים.

3.2.3.1 רקע תיאורטי על מודלי SSM

מודל SSM הוא גישה מתמטית לתיאור מערכות שמתפתחות לאורך זמן, והוא נמצא בשימוש נרחב בתחומים כמו בקרת מערכות, חיזוי סדרות זמן, עיבוד אותות וניווט. המטרה המרכזית של SSM היא לתאר כיצד מערכת משתנה, כך שנוכל לחזות את העתיד שלה, לשלוט בה ולסנן רעשים מנתונים חיצוניים.
מודלים אלה מבוססים על מצב פנימי שמכיל את המידע ההכרחי על המערכת, כך שאין צורך לזכור את כל העבר שלה. במקום זאת, הם משתמשים במערכת של משוואות מתמטיות שמתעדכנות בכל רגע בזמן.

כפי שניתן לראות באיור 15, SSM משתמש במערכת משוואות כדי לתאר איך מצב פנימי של מערכת משתנה עם הזמן ואיך ניתן למדוד אותו. המשוואות המרכזיות במודל הן: משוואת המצב שמתארת את השינוי הפנימי ומשוואות התצפית שמתארת איך המערכת מייצרת פלטים חיצוניים.



איור 15 : הגדרת SSM [27]

איור 15: באיור מוצג תרשים בלוקים של מודל SSM, המתאר את זרימת המידע במערכת דינמית לאורך זמן האיור מדגים כיצד הקלט u עובר דרך מטריצות A, B, C ו- D תוך שילוב אינטגרציה של מצב פנימי x, ליצירת הפלט y בהתאם להתנהגות המערכת.

משוואת המצב: $x'(t) = Bu(t) + Ax(t)$ (משוואה 23)

$x(t)$ מתאר את המצב הפנימי של המערכת ברגע נתון t. זהו הזיכרון של המערכת שמתעדכן כל הזמן בהתאם לקלט.

$x'(t)$ – קצב השינוי של מצב המערכת.

A – מטריצה שמתארת איך מצב המערכת משתנה באופן טבעי ללא השפעות חיצוניות.

B – מטריצה שמתארת איך קלט חיצוני משפיע על המערכת.

$u(t)$ – קלט חיצוני שמשפיע על המערכת.

אם הייתי לוקח את המשוואה ומביא אותה לידי ביטוי במצב שבו המערכת SSM שלי שולטת למשל ברחפן, המשתנה $x(t)$ היה מתאר את המיקום והמהירות של הרחפן, $Ax(t)$ מתאר איך החרפן ינוע באופן טבעי אם לא ניגע בו. $Bu(t)$ מייצג איך ההנחיות שאנחנו נותנים למנועים ישפיעו על הרחפן. המשוואה הזו מאפשרת לנו לחזות איך הרחפן יתנהג גם אם יש שיבושים, כי היא לוקחת בחשבון גם את התנועה הטבעית שלו וגם את הפקודות שאנחנו שולחים.

משוואת התצפית $y(t) = Du(t) + Cx(t)$

$y(t)$ – הפלט הנמדד של המערכת, מה שרואים בפועל.

C – מטריצה שמתארת איך המצב הפנימי של המערכת משפיע על הקלט.

D – מטריצה שמתארת השפעה ישירה של הלקט על הפלט.

במציאות, אנחנו אף פעם לא יכולים למדוד ישירות את כל מצב המערכת. אנחנו תמיד מודדים רק חלק מהמידע דרך חיישנים חיצוניים, והמדידות הללו עלולות להיות שגויות או רועשות. משוואת התצפית מסייעת לחבר בין המצב האמתי של המערכת לבין מה שאנחנו יכולים למדוד בפועל.

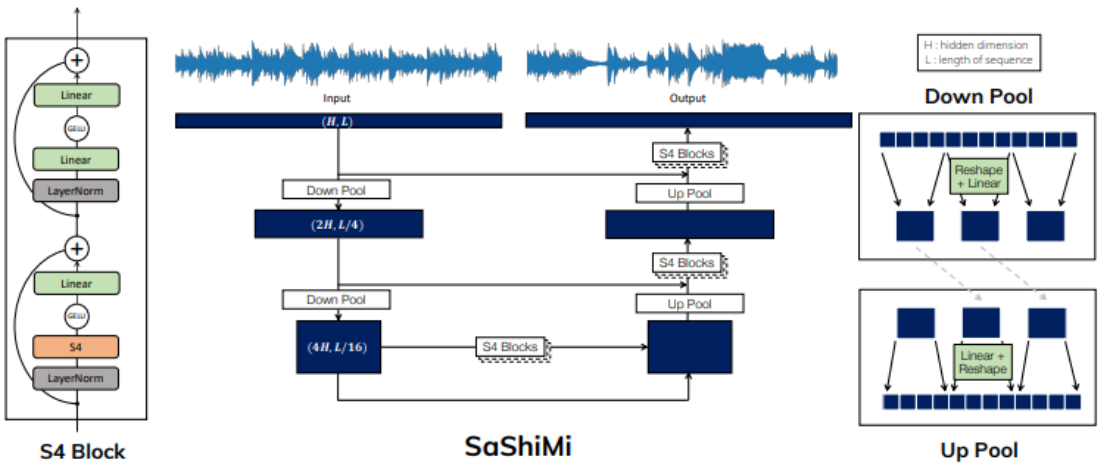
משוואת המצב עוזרת לנו להבין איך המערכת מתפתחת לאורך זמן, ומשוואת התצפית עוזרת לנו להבין איך ניתן למדוד את ההתפתחות הזו. כלומר נשתמש במשוואת המצב כדי לחשב את השינוי אותו אנחנו רוצים לחזות ולאחר מכן נשתמש במשוואת התצפית כדי לבדוק איזה מדידות נקבל.

במילים פשוטות SSM, מנסה להעריך איך המערכת תתנהג בעתיד בהתבסס על המידע שהיא קיבלה עד כה.

3.2.3.2 יישומי SSM

אחד השימושים המרכזיים של SSM הוא חיזוי וחישוב של סדרות זמן. מאחר ו- SSM מבוסס על משוואות דיפרנציאליות לינאריות, הוא מסוגל להתמודד בצורה טבעית עם סדרות זמן ארוכות, וזאת מבלי להיתקל בבעיות גרדיאנט שהצגתי קודם לכן, המאפיינות רשתות חוזרות [1]. מודלים מבוססי SSM כמו S4, מצליחים להציג ביצועים טובים משמעותית בחיזוי נתונים בזמן אמת, תוך שמירה על יציבות חישובית ושימוש מופחת במשאבי חישוב.

בנוסף כפי שניתן לראות באיור 16, SSM נמצאים בשימוש נרחב גם בתחום עיבוד אותות שמע ודיבור, הודות ליכולתם לנתח הקשרים מורכבים בין תדרים שונים לאורך זמן. מודלים כמו S4 הצליחו להתעלות על מודלים מבוססי CNN ו- RNN בעיבוד אותות דיבור, בכך שהם שומרים על הקשרים הפנימיים של האותות לאורך טווחי זמן ארוכים יותר.



איור 16 : מודל SaShiMi מבוסס S4 לעיבוד אודיו וסינתזת גלי קול גולמיים [10]

איור 16 : SaShiMi הוא דוגמה מתקדמת ליישום של מודל SSM בתחום עיבוד אודיו. בתרשים תיאור הארכיטקטורה המשלבת בלוקי S4, עם מנגנוני pooling מרובי רמות כדי לאפשר למודל לעבד מידע ברזולוציות שונות של האות תוך שמירה על תלות ארוכת טווח. המודל מקבל קלט של גל קול ומבצע עיבוד היררכי עם כמה רמות, הקלט מצומצם באופן הדרגתי דרך שכבת Down Pooling שבהן הרצף מתקצר אך גדל ברוחב הייצוג (מאבד פירוט זמנים אך שומר מידע עקרוני). בכל רמה מבוצע עיבוד נוסף באמצעות בלוקים של S4, המאפשרים למודל ללמוד דפוסים בטווחי זמן משתנים. בסוף התהליך, השחזור נעשה דרך שכבות Up pooling, אשר מגדילות מחדש את אורך הסיגנל ומשחזרות את הגל המלא.

3.2.3.3 SSM אורתוגונליים O-SSM

ראינו כיצד מודלי SSM מסורתיים מהווים מסגרת מתמטית עוצמתית ללמידת סדרות זמן, אך הם סובלים ממגבלות בכל הקשור לשימור מידע לאורך זמן ארוך. בעיה זו נובעת מכך שמודלי SSM מסורתיים מסתמכים על משוואות לינאריות פשוטות, אשר מתקשות להתמודד עם זיכרון ארוך טווח ועם יציבות חישובית לאורך אלפי צעדים בזמן. המאמר [10] מציע פתרון לבעיה זו באמצעות שילוב של גישת מרחב-מצב עם שיטות אורתוגונליות, אשר מאפשרת יצירת SSM אורתוגונליים.
באופן כללי, אורתוגונליות משמעותה ששני וקטורים או פונקציות לא משפיעים זה על זה, כלומר הם עצמאיים לחלוטין מבחינה מתמטית. כאשר משתמשים בזה במודלים כמו SSM, זה מאפשר להם לארגן מידע בצורה חכמה ולמנוע ערבוב של תכונות שונות, מה שעוזר בלמידת מידע לטווח ארוך בלי לאבד דיוק.

השימוש בשיטות אורתוגונליות משפר משמעותית את SSM בכך שהוא מאפשר לייצג את המצב הפנימי של המערכת באמצעות פולינומים אורתוגונליים כמו פולינומי Legendre או Fourier. בעוד שב- SSM מסורתי המצב הפנימי $x(t)$ מתעדכן בהתאם לקלט בצורה לינארית פשוטה, ב- SSM אורתוגונלי המצב הפנימי נבנה על בסיס היטל על פולינומים אורתוגונליים, מה שמאפשר ייצוג קומפקטי ויציב של המידע.
מבנה זה מאפשר למערכת להתמודד עם סדרות ארוכות במיוחד, תוך שמירה על יציבות חישובית גבוהה.
HiPPO-Legs שהצגנו קודם לכן [5], הוא SSM מיוחד שבו המטריצות נבחרות כך שהן מאפשרות זיכרון ארוך טווח דרך היטל פולינומי. הוא פותח מתוך מטרה להגדיר מנגנון SSM אורתוגונלי, שבו המטריצה A שראינו במשוואה 23 מבוססת על פולינומי Legendre.
בניגוד ל- SSM סטנדרטיים שהוא LTI, בלתי משתנה בזמן כי המקדמים A, B, C, D אינם משתנים בזמן.
HiPPO-Legs הוא מערכת שמשתנה בזמן כיוון שיש תלות ב- t בכל איבר במטריצות. הדבר נובע מהצורך להתאים את סולם הזמן של ההשלכה הפולינומית כך שהמערכת תוכל להתמודד עם קלטים המשתנים בצורה דינמית, דבר המשפר את יכולת ניהול הזכירון של המערכת.
האורתוגונליות מאפשרת ל SSM להתמודד עם תלות ארוכת טווח בצורה חסכונית ויעילה, בלי מנגנונים מסובכים כמו LSTM או טרנספורמרים.

3.3 הרחבות ושיפורים למודלים מבוססי LMU

בשנים האחרונות חלה התקדמות משמעותית בפיתוח של רשתות עצביות מבוססות מודלי SSM, כאשר LMU נחשב לאחת ההצעות החדשניות ביותר לשימור מידע באופן אופטימלי. עם זאת למרות היתרונות הרבים של LMU על פני מודלים מסורתיים כמו LSTM, קיימות מספר הגבלות שמקשות על שימושו בקנה מידה רחב, במיוחד כשמדובר בבעיות למידת מכונה מודרניות הדורשות חישוב מקבילי, מהירות חישוב גבוהה ויעילות משאבי זיכרון. בפרק זה נעמוד על ההרחבות והשיפורים למודל LMU, אשר נועדו להתגבר על המגבלות הללו ולהחריב את היישומים בהם ניתן להשתמש בגישה זו.

אחת ההרחבות המרכזיות שאציג היא FF-LMU, גרסה של LMU שמטרתה להאיץ חישובים ולהתאים את המודל לעבודה מקבילית. בעוד LMU המקורי בנוי כרשת סדרתית עם תלות בין צעדים עוקבים, FF-LMU משתמש בגישת Feedforward, שינוי זה מאפשר למודל להפיק תובנות מבלי להסתמך על עדכון סדרתי של מצב פנימי, ובכך מאפשר אימון מהיר יותר על חומרות מודרניות כמו GPU. בנוסף, אציג בפרק זה את השיפורים החישוביים של LMU, הכוללים האצת חישוב באמצעות שיטות מטריציוניות מתקדמות, מקבול של עדכון מצבי המערכת, ושימוש באופטימיזציות יעילות יותר למודל מרחב-מצב .שיפורים אלו הופכים את LMU לכלי שניתן להפעיל בסביבות עם כמויות גדולות של נתונים, כמו עיבוד דיבור, חיזוי סדרות זמן, וניתוח נתוני חיישנים.
כמו כן נדון במודל Mamba, מודל חדשני המשלב את העקרונות של LMU יחד עם טכניקות מ- SSM ועם עקרונות מבוססי טרנספורמרים, ובכך מאפשר ביצועים דומים לאלה של מודלים מבוססי Attention אך עם סיבוכיות נמוכה בהרבה. השיפורים הללו לא רק משפרים את היכולת חישוב של LMU, אלה גם פותחים אפשרות לשימוש מודלים אלו בסביבות נוירומורפיות. בהן חיקוי מבנה הרשתות של המוח האנושי מאפשר לממש פתרונות למידה יעילים וצריכת אנרגיה נמוכה יותר, על זה נדון בהרחבה בפרקים הבאים בעבודה.

3.3.1 חישוב מקבילי של LMU

בשנים האחרונות, רשתות נוירונים חוזרות (RNNs) הוכיחו את יעילותן במגוון רחב של משימות עיבוד נתונים סדרתיים. עם זאת, אימון RNNs, המיוחד על רצפים ארוכים, נותר מאתגר בשל בעיות כמו גרדיאנטים נעלמים או מתפוצצים ומגבלות במקביליות על חומרת ה- GPU, הנובעת מזה שכל מצב תלוי בקודמו. מודל ה- LMU שהצגתי בפרקים הקודמים, הוצג כארכיטקטורת RNN חדשנית המציגה שיפורים משמעותיים בקיבולת הזיכרון הזמני. למרות הצלחתו, גם ה- LMU המקורי סבל מחלק מאתגרי האימון של RNNs. כדי לפתור בעיה זו, הוצעה גרסה משופרת של LMU בשם Feedforward Legendre Memory Unit. ה- FF-LMU [4] מהווה וריאציה מעניינת של הארכיטקטורה המקורית בכך שהוא מאפשר אימון במתכונת של רשת feedforward תוך שמירה על יכולת פעולה כרשת חוזרת בזמן ההיסק. גישה זו מאיצה את תהליך האימון באופן משמעותי ומראה ביצועים מרשימים במגוון משימות תוך שימוש במספר פרמטרים קטן משמעותית בהשוואה למודלים אחרים.

3.3.1.1 המעבר מ-LMU סדרתי ל-FF-LMU

המעבר מ- LMU סדרתי ל- FF-LMU מבוסס על שתי התאמות מרכזיות. ראשית, ב- FF-LMU אין שימוש במערכת דינמית לא לינארית המשמשת לחישוב המצבים הפנימיים. במקום זאת, החישוב נעשה באופן feedforward טהור, כאשר כל שלב מחושב בו זמנית במקום תלות סדרתית בין צעדים עוקבים. שנית, מצב עדכון המערכת, שב- LMU מתבצע בצורה נשנית על פני זמן, הומר לפעולה שמבוססת על חישוב מטריציוני שניתן להפעיל ביעילות גבוהה יותר על חומרות כמו GPU.

כזכור המודל המקורי של LMU השתמש במערכת LTI בה הייצוג של המידע לאורך זמן הוגדר על ידי שלוש משוואות המייצגות את הקלט למערכת u_t, את המצב החבוי h_t ומצב הזיכרון m_t משוואות [11][12][13].

הבעיה המרכזית בגישה זו היא שהעדכון של m_t מחייב חישוב סדרתי לאורך הצעדים בזמן, מה שגורם לתלות שדיברנו עליה קודם בין שלבים עוקבים ומונע ריצה מקבילית מלאה. המודל החדש מציע פתרון על ידי שינוי במבני משוואות המצב של LMU כך שניתן לחשב כל צעד באופן בלתי תלוי ולבצע את החישובים במקביל.

השינויים במשוואות המצב:

<u>משוואה 25:</u> $u_t = g(W_u x_t + b_u)$

<u>משוואה 26:</u> $m_t = Am_{t-1} + Bu_t$

<u>משוואה 27:</u> $o_t = f(W_o m_t + b_o)$

משוואה (25) מציגה את הקלט למערכת u_t, העובר דרך שכבה לינארית עם bias ולאחר מכן פונקציה לא לינארית g.

משוואה (26) מציגה את מצב הזיכרון שנשאר מבוסס על מערכת ה- LTI

משוואה (27) מציגה את הפלט o_t, המחושב דרך שכבה לינארית עם bias ולאחר מכן פונקציה לא לינארית f.

ההבדלים במשוואות באים לידי ביטוי:

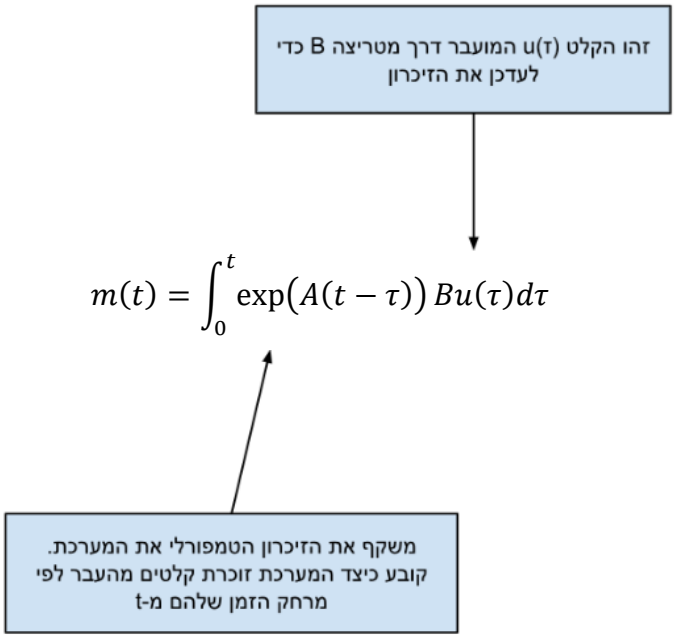
רכיב	LMU	FF-LMU
קלט למערכת u_t	הקלט מחושב על ידי שילוב של הקלט הנוכחי x_t, המצב הנסתר הקודם h_{t-1} ומצב הזיכרון הקודם m_{t-1} עם וקטורי משקל e, e_h, e_m. הקלט הוא סקלר שנוצר על ידי היטל של מספר רכיבים.	הקלט u_t מחושב ישירות כוקטור, על ידי טרנספורמציה לינארית של x_t באמצעות מטריצה W_u ו bias b_u. אחר כך הפעלת פונקציה לא לינארית g. אין תלות במצב הקודם או בזיכרון הקלט הקודם.

חישוב המצב הנסתר h_t	המצב הנסתר מחושב תוך שימוש במצב הקודם h_{t-1} , הזיכרון הנוכחי m_t והקלט הנוכחי x_t . כל אלה משתלבים עם מטריצות משקל W_x, W_h, W_m ולאחר מכן מופעלת פונקציה לא לינארית f.	המצב הנסתר מוחלף בפלט o_t שמחושב בצורה ישירה מהזיכרון m_t על ידי טרנספורמציה לינארית והפעלת פונקציה לא לינארית f. אין חזרה למצב הקודם h_{t-1}
שימוש במערכת לא לינארית	יש שילוב מורכב בין רכיבים לינאריים ולא לינאריים, כאשר ה- LTI פועלת כזיכרון והחישוב הלא לינארי מבוצע עם הזיכרון, הקלט והמצב הקודם.	כל המורכבות הלא לינארית מצטמצמת לחישוב הפלט בלבד o_t , וה- LTI נשארת אחראית היחידה לזיכרון.

ביטול התלות בh_{t-1} מאפשר לעבד את כל הצעדים במקביל, שכן אין צורך לחכות לחישוב של הצעד הקודם.
במערכת LTI הזיכרון (המצב הפנימי) m_t הוא וקטור שמייצג את ההיסטוריה של הקלטים הקודמים בצורה דחוסה. במערכת רגילה, כמו RNN או LMU נדרש לחשב את m_t באופן רקורסיבי, כלומר:

$$m_t = f(m_{t-1}, x_t)$$

משמעות הדבר היא שחישוב של m_t תלוי בm_{t-1} ו- x_t (הקלט הנוכחי). גישה זו מחייבת מעבר סדרתי על הנתונים לאורך הזמן. במילים פשוטות כדי לחשב את m_t עבור כל שלב בזמן, יש לחשב תחילה את השלבים הקודמים, ולכן נדרש חישוב סדרתי שאינו מאפשר עבודה מקבילית.
לעומת זאת ב FF-LMU המבנה המיוחד, המשתמש בייצוג קונבולוציוני של עדכון המצב (משוואה 26). דבר ההופך את הזיכרון למשהו שניתן לחשב בבת אחת לכל הזמנים במקום בכל שלב באופן סדרתי, באמצעות אינטגרל. <u>משוואה 28</u>:



האינטגרל מבטא את המצב של מערכת בצורה ישירה עבור כל נקודה בזמן t, מבלי לחשב את m_{t-1} או לבצע חישוב סדרתי.

בעצם מה שביצעו פה זה טרנספורמציה של כל הסדרה בזמן באמצעות קונבולוציה:

משוואה 29:

$$m(t) = \int_0^t K(t-\tau)u(\tau)d\tau$$

כאן, $K(t)$ הוא גרעין קונבולוציה, שדרכו אנחנו מחשבים את המצב עבור כל שלב מבלי לעבור על השלבים הקודמים אחד-אחד. במקרה שלנו גרעין הקונבולוציה הוא: $e^{A(t-\tau)}B$

Parallelizing LMU 3.3.1.2

בפרק הקודם עסקתי במעבר מ- LMU סדרתי ל- FF-LMU, שבו שונתה ארכיטקטורת המודל כך שתאפשר חישוב מקבילי של כל שלבי העיבוד במקום חישוב סדרתי. עם זאת FF-LMU משיג מקביליות בעיקר בכך שהוא משנה את מבנה הרשת, וממיר את האופן שבו מתבצע העדכון של מצב הזיכרון למבנה המבוסס טרנספורמציות לינאריות בלבד. פתרון נוסף שנחקר כדי להתגבר על בעיה זו הוא אימון מקבילי של LMU, אשר מאפשר לבצע אופטימיזציה למודל תוך שמירה על יתרונותיו הייחודיים [20].
ראינו כי באימון סטנדרטי של LMU המשוואות הבסיסיות מתארות עדכון מצב פנימי באופן סדרתי, לפי (משוואה 13). אימון מקבילי של LMU מציע שיטה שונה לניהול הזיכרון ועדכון המצב, כך שהוא משתמש בטרנספורמציה מטריציונית של כל הרצף במקום לעדכן כל שלב בפני עצמו. במקום לחשב כל m_t לפי m_{t-1}, אפשר להשתמש בגישה מטריציונית המאפשרת חישוב מקבילי:

משוואה 30:

$$m_t = \sum_{j=1}^{t} \bar{A}^{t-j} B u_j$$

זוהי משוואה קונבולוציונית לינארית שמחליפה את הצורך בעדכון סדרתי של m בכך שהיא מאפשרת חישוב ישיר של כל שלבי המצב על ידי סיכום השפעת הקלטים הקודמים.
כעת, כדי להפוך את תהליך האימון למקבילי המאמר מגדיר מטריצה H המכילה את תגובות האימפולס של המערכת הלינארית, הוא פועל כמסנן קונבולוציוני המתאר איך כל שלב הושפע מהשלבים הקודמים:

$$\mathrm{H} = [\overline{A^0}\bar{B}\ \overline{A^1}\bar{B}\ \dots\ ]$$

כך שניתן להגדיר את כל המצבים m עבור כל צעדי הזמן באמצעות המכפלה:

משוואה 31:

$$m_{1:n} = HU$$

את החישוב שמוצע (במשוואה 31) ניתן לייעל עוד יותר, על ידי יישום משפט הקונבולוציה, המאפשר לבצע את החישוב באמצעות התמרת פורייה. בתחום זה פעולת הכפל מתבצעת ישירות על ספקטרום הנתונים, דבר שמפחית את מורכבות החישוב [נספח 1].

היתרון של שיטה זו הוא שהיא מאפשרת אימון של LMU כולו במקביל, ללא צורך בהמתנה לסיום של כל שלב. בנוסף, כאשר המודל פועל על חומרה מבוזרת, ניתן להקצות חישובים שונים לחלקים שונים של הרשת, ולנצל אופטימיזציות כמו חישוב מטריציוני מהיר כדי להאיץ את התהליך. בעוד שב FF-LMU שהצגתי בפרק הקודם, שונו המשוואות עצמן כדי לבטל את החזרתיות של הרשת, באימון מקבילי של LMU הרעיון המרכזי הוא לחשב את כל הערכים במקביל על ידי שימוש במטריצות, תוך שימור המודל הבסיסי של LMU.
האימון המקבילי שומר על המבנה של LMU המקורי, כולל השימוש במערכת לינארית דינמית לזיכרון פנימי. יחד עם זאת, יש לכך גם חסרונות, גישה זו דורש יותר זיכרון, מכיוון שהמערכת צריכה לאחסן בו זמנית את כל מצבי המערכת, במקום לעדכן אותם צעד אחר צעד.
הבחירה בין שתי השיטות תלויה בסוג היישום. כאשר המטרה היא לשלב LMU במערכות שבהן Feedforward עדיף, FF-LMU מספק פתרון מתאים יותר. לעומת זאת, כאשר יש צורך לשמר את המבנה הסדרתי אך להאיץ את האימון, ניתן להשתמש בשיטה של אימון מקבילי.

3.3.1.3 שימוש ב Delay Network

המרכיב המרכזי של הרשת הוא רשת הדילייי. DN משמש כרכיב הליבה של ה- LMU ותפקידה להעניק למערכת יכולת שימור מידע לאורך זמן ארוך תוך שימוש בתכונות מתמטיות של פולינומים אורתוגונליים.
DN נבנתה כמודל לינארי לזיכרון טמפורלי. כך היא מיישמת פעולה של עיכוב מדויק של קלטים. באופן אידיאלי מערכת דילייי מקבלת קלט $u(t)$ ומוציאה פלט $y(t)$ שהוא פשוט גרסה מושהית של הקלט בזמן θ. באופן מתמטי אפשר לנסח את זה:
<u>משוואה 32:</u>

$$y(t) = D[u(t)] = \begin{cases} 0, & t < \theta \\ u(t-\theta), & t \geq \theta \end{cases}$$

מדוע יש צורך ב- DN?
אחד האתגרים המרכזיים בעיבוד סדרות זמן הוא הצורך לאגור מידע היסטורי מבלי שהוא יתפוגג או יתערבב עם מידע חדש. רשת ה DN משמשת כבסיס לזיכרון, אשר מאפשר ייצוג של היסטוריית הקלטים הקודמים באופן קומפקטי ודינמי. DN מתפקד כמו פילטר אופטימלי, אשר משמר את המידע בצורה מדורגת בהתאם לחשיבות שלו.
הרעיון המרכזי כאן הוא שה DN משמש כמערכת זיכרון נפרדת מהחלק הלא ליניארי של הרשת, מה שמאפשר שימור מידע לאורך זמן ללא הצטברות של שגיאות או גרדיאנטים נעלמים.
מכיוון שה DN מבוססת על מערכת לינארית, ניתן לבצע חישובים במקביל (ולא בצורה סדרתית), מה שמאיץ את תהליך האימון עד פי 200.

3.3.2 Mamba - שילוב עם SSM

מודלים מבוססי SSM הפכו בשנים האחרונות לאחד הכלים המרכזיים לשיפור עיבוד רצפים, במיוחד בתחומים בהם יש צורך בלמידת תלות ארוכת טווח ובזיכרון אפקטיבי. אחד המודלים החדשניים שניצלו את היתרונות של SSM הוא LMU, שהוצג כפתרון המשלב פולינומים של לג׳נדר עם דינמיקות מצב מרחב כדי ליצור זיכרון חזק לאורך זמן עם שימוש יעיל במשאבים חישוביים.
LMU מיישם את עקרונות SSM באמצעות שימוש במערכת דינמית לינארית רציפה בזמן שמקרינה את ההיסטוריה של הקלט על פולינומים אורתוגונליים. ראינו כי קיימות מספר מגבלות ל- LMU כמו חוסר יכולת להתמודד עם בחירה דינמית של מידע רלוונטי, הגבלה על אינטראקציות בלתי לינאריות בין מצבים שונים, והצורך באופטימיזציות מתקדמות עבור יישומים מודרניים.
כאן נכנס Mamba, שהוא חידוש בתחום הSSM, אשר משפר ומרחיב את היכולת של LMU. Mamba עושה זאת על ידי שילוב של מנגנון בחירה סלקטיבית המאפשר למודל לפעול באופן דינמי על פי הנתונים הנכנסים. כלומר, לבחור אילו מצבים לשמור ואילו ניתן להזניח. באופן זה, Mamba מחזק את התכונות הבסיסיות של LMU ומשפר את יכולותיו לעיבוד מידע רציף ומורכב [6].

3.3.2.1 מערכת SSM דינאמית

בליבת Mamba נמצאת מערכת SSM אשר מיישמת משוואות לינאריות של דינמיקה רציפה כדי לקודד מידע על פני רצפים ארוכים. בניגוד ל- SSM רגיל, שבו הפרמטרים (המטריצות A, B, C) קבועים, Mamba משתמש בפרמטרים שתלויים בקלט, כלומר הם מחושבים באופן דינמי בהתאם למידע שהתקבל.
החישוב העיקרי במערכת זו מבוסס על המשוואה:
<u>משוואה 33:</u>

$$h'(t) = Ah(t) + Bx(t)$$

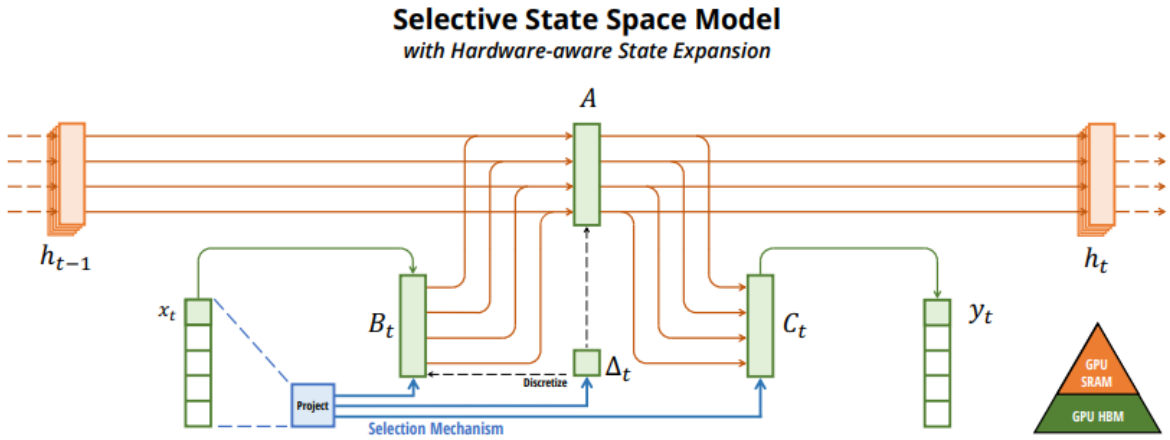
כאשר המצב הפנימי $h(t)$ משתנה בהתאם למצבו הקודם $Ah(t)$ ולתרומת הקלט החדש $Bx(t)$. המערכת פותרת את המשוואה הזאת בזמן אמת, תוך התאמה של B ו-A לכל שלב בקלט, מה שמאפשר למודל לבחור אילו פרטים במידע לשמור ואילו לדכא.

3.3.2.2 מנגנון בחירה סלקטיבית

כעת, כדי לאפשר למודל להתאים את עצמו לנתונים בצורה יעילה Mamba משתמש במנגנון בחירה סלקטיבי כמתואר באיור 17. שכבה זו מוסיפה שערים דינמיים, אשר הקובעים כמה מידע מהמצבים הקודמים יש להעביר הלאה וכמה מידע להזניח. השער פועל על פי הנוסחה:
<u>משוואה 34:</u>

$$g_t = \sigma(Linear(x_t))$$

כאשר g_t הוא שער הבחירה, Linear היא מטריצת משקלים הלומדת איזה מאפיינים חשובים. ו- σ היא פונקציה לא לינארית כמו סיגמואיד או ReLU. באמצעות מנגנון זה המודל מחליט איזה מאפיינים חשובים ללמידת התלות ברצף. זה מאפשר סינון רעשים, שיפור יעילות חישובית והפחתת הצורך בזיכרון נוסף.



איור 17 : Selective SSM במודל Mamba [6]

איור 17 : האיור מציג את הארכיטקטורה של מודל Selective SSM, עם הרחבת מצב מודעת חומרה. התרשים מדגים את הזרימה של המידע במודל, כולל השפעת מנגנון הבחירה, הרכיבים המרכזיים במערכת המצב מרחב (A, B_t, C_t, Δ_t), ואת הדרך שבה הנתונים מעובדים ממצב קודם h_{t-1} למצב הנוכחי h_t ולפלט הסופי y_t.

נציין כי מטריצה A שאנו רואים בתרשים, אחראית על עדכון המצב הפנימי של המערכת, היא קובעת איך הזיכרון מתעדכן בכל שלב. ומחליטה האם המידע חזק מספיק להישאר או יוחלף במידע חדש.

מטריצה B_t, מטריצה המשמשת כדי להמיר את הקלט x_t לתוך המערכת. זה השער שמכניס את המידע החיצוני לתוך המצב הפנימי.

מטריצה C_t משמשת להמיר את המצב הפנימי לפלט. כשהמודל רוצה להוציא תשובה, מטריצה זו ממירה את המידע בזיכרון לפלט שימושי.

אחד הגורמים הכי חשובים בתרשים הוא Δ_t, שמסומן כשלב הדיסקרטיזציה, אשר משמש לנרמל ולעדכן את שלבי המעבר בהתאם למשאבי החומרה. בעולם האמתי, הנתונים מגיעים בערכים בדידים, כדי שנוכל להשתמש במודל זה במחשבים, עלינו להמיר את המשוואות הרציפות למשוואות המתאימות לערכים בדידים. מבצעים לצורך זה דיסקרטיזציה על ידי המרת הפרמטרים A ו- B לערכים דיסקרטיים.

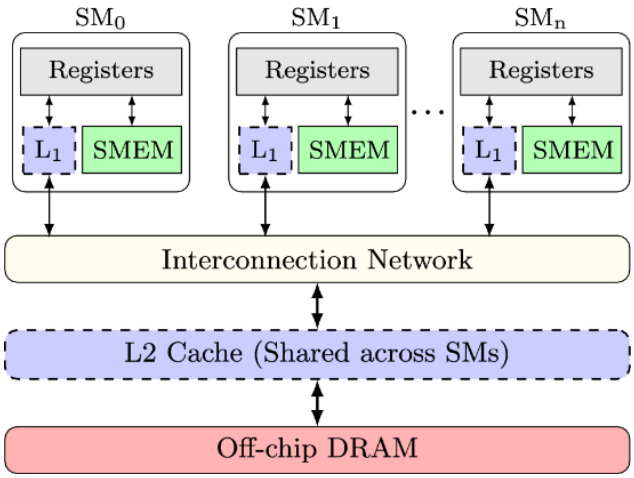
לאחר המרת הפרמטרים, המודל יכול להתבצע בשתי דרכים.

חישוב כרצף לינארי, המודל עובר על כל קלט אחד אחרי השני. ממש כמו שרשרת. החיסרון זה שאי אפשר לבצע חישוב במקביל. דרך שנייה, חישוב באמצעות קונבולוציה, לא מחכים לשלב הקודם אלא מחשבים את כל הרצף במכה.

3.3.2.3 ארכיטקטורת המודל

כפי שראינו בפרק 3.2.3, מודלי SSM מופיעים כארכיטקטורה מבטיחה לעיבוד סדרתי. הרעיון במודלים אלו הוא שהמידע נשמר בצורה רציפה ולכן המודל לא זקוק לחלון הקשר מוגבל כמו הטרנספורמרים. מודל Mamba לוקח את SSM ומשפר אותו, בכך שהוא מאפשר למודל לבחור מידע רלוונטי ולדחות מידע לא חשוב. Mamba פותר על הבעיה של שמירת מידע לא רלוונטי, על ידי מנגנון בחירה דינמי שמחליט איזה מידע חשוב ואיזה מידע ניתן להזניח. המודל יכול לשמור מידע ללא הגבלת זמן, ולכן מסוגל לעבוד עם רצפים ארוכים.

על מנת להבין את החידוש שהביאה ארכיטקטורת המודל החדש, נתמקד קודם לכן במבנה של SSM מסורתי. כזכור, מודלי SSM מסורתיים משתמשים בקונבולוציות, הדורשות גישה רבה לזיכרון בזמן החישוב. המעבר בין רמות הזיכרון השונות ב- GPU (L1, L2, Global Memory) מאט את הביצועים.
איך Mamba פותר את זה? במקום להשתמש בקונבולוציות, הוא משתמש בסריקה, כלומר הוא מחזיק מידע בזיכרון הפנימי של הליבה L1 Cache וכך נמנע מגישות יקרות לזיכרון הראשי כפי שניתן לראות באיור 18.
סדר הגודל הוא לינארי, ככל שאורך הקלט גדל, זמן החישוב גדל לינארית ולא בריבוע כמו בטרנספורמרים.



איור 18 : היררכית הזיכרון של GPU [6]

איור 18 : האיור מתאר את הארכיטקטורה ההיררכית של הזיכרון ב- GPU. תוך התמקדות במבנה הזיכרון של SM. כל SM כולל רשומות המשמשות לאחסון מהיר של נתונים, L1 Cache, המספק אחסון נתונים מהיר לכל SM וזיכרון משותף המשמש לשיתוף נתונים בין הליבות בתוך אותו SM. כל ה SMs מחוברים דרך רשת תקשורת המאפשרת גישה למטמון L2 Cache המשותף לכל ה SMs ומשפר את יעילות הגישה לזיכרון. מתחת למבנה זה נמצא זיכרון DRAM חיצוני, שהוא הזיכרון המרכזי של ה- GPU אך הגישה אליו איטית יותר בהשוואה לזיכרונות הקרובים לליבות העיבוד. הארכיטקטורה ממחישה את חשיבות שימוש יעיל בהיררכיית הזיכרון, כך שהגישה לנתונים תהיה מהירה ויעילה ככל האפשר, מה שמשפר את ביצועי ה GPU במשימות חישוביות מקבילות.

Mamba שייך למחלקה חדשה של מודלים המכונים מודלים מבוססי מרחב מצב סלקטיבים. מודלים אלה משפרים את יכולותיהם של SSM מסורתיים ושל RNNs על ידי הוספת מנגנון סלקטיביות.

במודלים כמו RNN וטרנספורמרים (בגרסה הבסיסית שלו), המידע נשמר באופן זהה בכל השלבים. הם לא יכולים להתאים את עצמם להקשר שונה. במודלים מבוססי קונבולוציה יש בעיה אחרת, החישוב מהיר אבל חסרים להם יכולות של למידה מבוססת הקשר, כי הפילטרים הקונבולוציונים לא יודעים לבחור מידע מסוים. הפתרון הוא סלקטיביות, ראינו כי Mamba פותר בעיה זו באמצעות מנגנון בחירה דינאמי, המאפשר למודל לשמור מידע חשוב ולהזניח מידע לא רלוונטי, ללא מגבלת אורך זמן.

כדי להבין את המנגנון בצורה טובה יותר, אציג 2 אלגוריתמים של SSM, אלגוריתם S4, המציג גרסה לינארית וקבועה בזמן. אלגוריתם S6, המציג גרסה שמשתנה בזמן עם מנגנון בחירה.

Algorithm 1 SSM (S4)

Input: $x : (B, L, D)$
Output: $y : (B, L, D)$
$\boldsymbol{A} : (D, N) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
$\boldsymbol{B} : (D, N) \leftarrow$ Parameter
$\boldsymbol{C} : (D, N) \leftarrow$ Parameter
$\Delta : (D) \leftarrow \tau_\Delta(\text{Parameter})$
$\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (D, N) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
$y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-invariant: recurrence or convolution
return y

אלגוריתם 1 [6], מקבל כקלט X בגודל (B, L, D) כאשר B מייצג את ה- batch size, L מייצג את אורך הרצף, D מספר הממדים של הנתונים. מגדירים את A שמייצגת את הדינמיקה של המערכת, B הקובעת איך הקלט משפיע ואת C שקובעת איך הפלט מחושב מהמצב הנסתר. מבצעים דיסקרטיזציה של A ו- B ומפעילים את המודל ומקבלים את הפלט. אפשר לשים לב שהפרמטרים A, B, C קבועים לאורך כל הרצף.

Algorithm 2 SSM + Selection (S6)

Input: $x : (B, L, D)$
Output: $y : (B, L, D)$
$\boldsymbol{A} : (D, N) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
$\boldsymbol{B} : (B, L, N) \leftarrow s_B(x)$
$\boldsymbol{C} : (B, L, N) \leftarrow s_C(x)$
$\Delta : (B, L, D) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
$\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (B, L, D, N) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
$y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-varying: recurrence (*scan*) only
return y

אלגוריתם 2 [6], הוא הרחבה של S4 שמוסיפה מנגנון בחירה, השינויים העיקריים הם ש- B ,C תלויים בקלט X והפרמטר A מקבל תיקון נוסף שמאפשר לו לשנות את עצמו בהתאם לקלט. Δ מחושב עם תיקון דינאמי, המודל מתאים את עצמו לכל שלב בזמן. המודל כבר לא LTI הוא משתנה בזמן.

ראינו כי מודלים מבוססי LTI כמו S4 יכלו להמיר את כל החישובים שלהם לקונבולוציות, מה שאפשר להם להיות מאוד מהירים. אבל Mamba הוא מודל שמשתנה בזמן, ולכן הוא לא יכול להשתמש בקונבולוציות קבועות. במודל כמו S6, המצב הפנימי משתנה באופן דינמי, ולכן אי אפשר "לקבע" את החוקים של המודל כמו ב LTI. אם החוקים משתנים, אי אפשר לבצע את כל החישוב במקביל כמו בקונבולוציות, כי כל שלב צריך להיות מחושב לפי התוצאה של השלבים הקודמים.

אז איך פותרים את זה ? זו הסיבה שMamba משתמש ב- selective scan, האלגוריתם מתבסס על שלוש טכניקות עיקריות :

Kernel Fusion, טכניקה הכוללת מיזוג חישובים להפחתת מספר הפניות לזיכרון.

Parallel Scan, חישוב מבוסס Scan המבוצע בצורה מקבילית.

Re-computation חישוב מחדש של שלבי ביניים כדי לחסוך בזיכרון, בנוסף טכניקה זו מבצעת חישובים באופן חכם על פי היררכיית הזיכרון של ה- GPU. ניצול של חומרת ה- GPU בצורה חכמה. במקום לשמור את כל המצב הפנימי h בבת אחת, אנחנו משתמשים ברמות שונות של זיכרון ב- GPU.

הפעולות הכבדות ביותר מתבצעות בזיכרון מהיר SRAM, במקום בזיכרון איטי HBM כך, אפשר לחשב את הנתונים בלי להחזיק הכל בזיכרון בבת אחת. המודל טוען את A, B, C לSRAM שהוא זיכרון מהיר יותר, מבצע את החישובים הרלוונטים על הפרמטרים האלה שם, כולל דיסקרטיזציה ולבסוף כותב חזרה את הפלט ל- HBM (ראה איור 18).

3.3.2.4 הקשר לLMU

ראינו בפרקים קודמים כי LMU הם מימוש ספציפי של SSM ללמידת רצפים. שבו פונקציות לגנדר משמשות כבסיס לאגירת זיכרון יעיל. שניהם משתמשים בווקטור מצב h(t) כדי לשמור מידע על רצפים ארוכים, ובמשוואות לינאריות כדי לעדכן את מצבם הפנימי. ב- LMU ראינו כי המצב הפנימי לא רק מתעדכן עם המטריצה A אלא גם עובר דרך בסיס לג׳נדר, מה שמאפשר שימור מידע חכם יותר.

שני המודלים מבוססים על עקרונות המשלימים אחד את השני, על ידי שילוב הרעיונות משני המודלים, ניתן לפתח מערכות היברידיות המשלבות ייצוג מתמטי חזק של LMU עם הדינמיות והגמישות של Mamba. לדוגמה ניתן להשתמש בפולינומי לג׳נדר כבסיס לייצוג זיכרון יעיל בתוך ארכיטקטורה של selective SSM, וכך לאפשר למודל להבין רצפים מורכבים תוך צמצום צריכת המשאבים. שילוב הכלים של שני המודלים יכול להוביל לפריצת דרך בפיתוח רשתות נוירונים רציפות, המסוגלות לשמור תלות ארוכת טווח באופן חסכוני יותר ולבצע חישובים בקנה מידה רחב עם יעילות חישובית גבוהה.

3.3.3 השוואה בין HiPPO, FF-LMU וMamba

מודלים לעיבוד רצפים ארוכי טווח חייבים לאזן בין שמירה על מידע היסטורי, יעילות חישובית ודיוק חיזוי HiPPO , LMU , FF-LMU ו- Mamba מייצגים גישות שונות להתמודד עם אתגר זה. בפרק זה נשווה את הביצועים של המודלים בהיבטים שונים: דיוק חיזוי, מהירות אימון, יעילות זיכרון ויכולת הכללה. תוך התבססות על ניסויים ותוצאות שהוצגו במאמרים השונים.

3.3.3.1 זיכרון ותלות ארוכת טווח

פרק זה יבחן וישווה את יכולות הזיכרון ארוך הטווח והטיפול בתלויות ארוכות טווח, תוך התבססות על ניסויים ותוצאות שהוצגו במאמרים השונים.

<u>HiPPO: שימור אופטימלי של היסטוריה באמצעות הטלות פולינומיות</u>

מסגרת HiPPO פותחה במטרה לספק דרך אופטימלית לשמר מידע היסטורי ברשתות נוירונים חוזרות. היא עושה זאת על ידי שימוש בהיטל של הקלט על בסיסים פולינומים אורתוגונליים, כאשר כל מדד כגון ,LegT LagT , LegS מגדיר כיצד יש לשקלל את חשיבותם של חלקי עבר שונים. היתרון המרכזי של HiPPO טמון ביכולתו לשמור על מידע לאורך זמן רב ללא צורך בהגדרת חלונות זמן מראש, והוא עמיד בפני שינויים בסולם הזמן של הנתונים. ראינו כי אחד היישומים המבטיחים הוא HiPPO-Legs, אשר הודגם אמפירית כי הוא מסוגל להתרחב ולטפל במערכי נתונים גדולים מאוד וללמוד תלויות לאורך מיליוני צעדי זמן מבלי לאבד ביצועים. יתר על כן HiPPO-LegS, הראה יכולת לשמר מידע לאורך זמן מבלי לאבד דיוק, במקרים בהם מודלים אחרים מתקשים. בניסויים השיג דיוק גבוה במיוחד 98.3% במשימת psMNIST [19], תוצאה שהייתה מתקדמת ביותר עבור רשתות נוירונים חוזרות באותה תקופה. הצלחה זו מעידה על יכולתו של HiPPO-LegS ללכוד ולשמר תלויות טמפורליות מורכבות לאורך רצפים ארוכים.

<u>FF-LMU: שיפור יעילות הזיכרון ארוך הטווח באמצעות מבנה feedforward</u>

FF-LMU מייצג שיפור משמעותי ביחס למודל LMU המקורי, תוך שמירה על יכולות הזיכרון ארוך הטווח שלו. אחד היתרונות המרכזיים של FF-LMU הוא היכולת להתאמן במהירות רבה יותר על GPU שכן הוא משתמש במבנה feedforward במהלך ההסקה, מה שמבטל את התלות הסדרתית הקיימת ברשתות חוזרות רגילות. למרות זאת, הוא עדיין שומר על מצב זיכרון המבוסס על מערכת ליניארית בלתי משתנה בזמן LTI המאפשר לו לשמר מידע לאורך זמן. בניסויים FF-LMU, הראה ביצועים מרשימים במשימות שונות הדורשות טיפול בתלויות ארוכות טווח. לדוגמה, במשימת סיווג סנטימנטים על מאגר הנתונים IMDb [21], השיג דיוק מרבי של 89.10% תוך שימוש בפחות מ-500 פעמים פחות פרמטרים בהשוואה ל- LSTM. תוצאות דומות נצפו במשימת זיהוי דמיון סמנטי על מאגר הנתונים Quora [22] שם FF-LMU השיג ביצועים טובים יותר מ LSTM עם כ-650 פעמים פחות פרמטרים. יתר על כן FF-LMU, השיג את התוצאה

הגבוהה ביותר (98.49%) על מבחן psMNIST תוך שהוא מתעלה אפילו על HiPPO-Legs. הצלחה זו מדגימה כי FF-LMU מסוגל לשמר מידע רלוונטי לאורך זמן רב וללמוד תלויות מורכבות ביעילות רבה.

Model	Accuracy
RNN	67.4 ± 7.7
LSTM	87.3 ± 0.4
LMU	87.7 ± 0.1
HiPPO-LagT	88.0 ± 0.2
LSTM (our impl.)	87.29
ff-LMU$_m$	**89.10**

טבלה 1: IMDB מבחן דיוק [2]

	Parameters		Accuracy	
Model	split$_1$	split$_2$	split$_1$	split$_2$
LSTM	-	800k	82.58	81.4
ff-LMU$_m$	1201	1201	**86.95**	**85.36**

טבלה 2: QQP מבחן דימיון סמנטי של שאלות [2]

Model	Accuracy
RNN-orth	89.26
RNN-id	86.13
LSTM	89.86
LSTM-chrono	88.43
GRU	92.39
JANET	91.94
SRU	92.49
GORU	87.00
NRU	95.38
Phased LSTM	89.61
LMU	97.15
HiPPO-LegS	98.3
ff-LMU$_o$	**98.49**

טבלה 3: psMNIST מבחן [2]

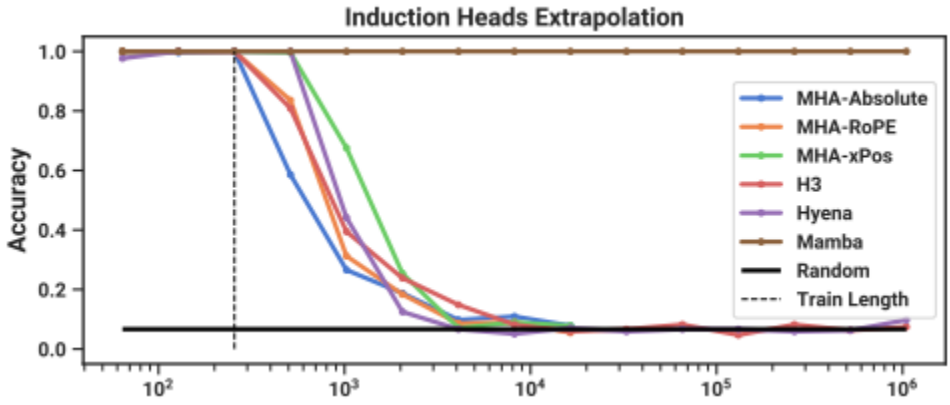
מעניין לציין ש- FF-LMU, למרות פשטותו ביחס ל- LMU המקורי, מתעלה עליו במבחנים. הדבר מצביע על כך שהיתרון המרכזי של LMU הוא איכות הזיכרון הטמפורלי שלו, המיושמת באמצעות מערכת ה- LTI האופטימלית בחלק החוזר של המודל.

<u>Mamba: מידול רצפים בזמן לינארי עם מנגנון מרחב-מצב סלקטיבי</u>

מודל Mamba מייצג גישה חדשנית למידול רצפים ארוכים על ידי שימוש במודל מרחב מצב סלקטיבי. בניגוד למודלים חוזרים או מבוססי תשומת לב Mamba, משתמש במנגנון בחירה המאפשר לו לסנן מידע לא רלוונטי ולשמר רק את המידע החשוב לאורך הרצף. סקיילביליות החישוב הליניארית של Mamba ביחס לאורך הרצף הופכת אותו ליעיל במיוחד עבור טיפול ברצפים ארוכים מאוד. בניסויים אמפיריים Mamba הראה יכולת יוצאת דופן בשימור מידע לאורך טווחים ארוכים (ראה טבלה 4 ואיור 19). במשימת Induction Heads Mamba הצליח לשמר מידע ולהכליל על רצפים באורך של עד מיליון טוקנים, בעוד שמודלים אחרים כמו Hyena וטרנספורמרים נכשלו לחלוטין כאשר אורך הרצפים התארך [6]. יתרה מכך, במשימת Selective Copying השיג דיוק של 99.8%, ביצועים גבוהים משמעותית ממודלים מבוססי Hyena או S4. תוצאות אלה מדגימות את יכולתו של Mamba להתמודד עם תלויות ארוכות טווח בצורה יעילה בהרבה ממודלים קיימים, תוך שמירה על מידע רלוונטי והתעלמות מרעש.

Model	Arch.	Layer	Acc.
S4	No gate	S4	18.3
-	No gate	S6	**97.0**
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	**99.7**
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	**99.8**

טבלה 4 : בדיוק במבחן העתקה סלקטיבית [6]



איור 19 : גרף דיוק במבחן אורך טקסט עבור מודלים שונים [6]

איור 19 : גרף המציג את ההשוואה בין דיוק המודלים השונים אשר אומנו על רצף באורך 256 ונבחנו על רצפים גדולים בהרבה ממה שהם התאמנו.

שלושת המודלים HiPPO, FF-LMU ו- Mamba מציעים גישות מתקדמות לטיפול בזיכרון ארוך טווח ובתלויות ארוכות טווח, כל אחד עם יתרונות ייחודיים. HiPPO מספק מסגרת תיאורטית חזקה ומוכיח יכולת שימור היסטוריה מצטברת לאורך פרקי זמן ארוכים במיוחד. FF-LMU משפר את יעילות החישוב תוך שמירה על יכולות זיכרון ארוך טווח מרשימות, והוא מתאים במיוחד ליישומים הדורשים מהירות ויעילות על GPU. Mamba מציג מנגנון סלקטיבי חדשני המאפשר לו להתמודד ביעילות עם רצפים ארוכים מאוד, לעתים קרובות תוך שהוא מתעלה על מודלים מבוססי טרנספורמרים. הניסויים השונים המוצגים בטבלאות 1-4 ובאיור 19 מספקים הוכחות משמעותיות ליכולות הזיכרון ארוך הטווח והטיפול בתלויות ארוכות טווח של מודלים אלה.

3.3.3.2 יעילות ומספר פרמטרים

בפרק זה נשווה את היעילות המתייחסת הן למהירות החישוב והאימון והן לעלות החישובית הכוללת, וכן את מספר הפרמטרים הנדרשים במודלים. תוך התבססות על תוצאות ניסויים ונתונים מהמאמרים.

מסגרת HiPPO, ובפרט היישום שלה HiPPO-LegS, הדגימה יעילות מרשימה בטיפול ברצפים ארוכים מאוד. המודל יכול לעבד עד 470,000 צעדי זמן בשנייה על ליבה אחת של מעבד (CPU). שהוא פי 10 מהיר יותר ממודלים כמו LMU ו- LSTM. עדות זו מצביעה על יעילות חישובית גבוהה בעדכון הזיכרון.

מודל FF-LMU, הציג שיפור משמעותי ביעילות האימון וההסקה, במיוחד על GPU. הוא מתאמן בערך פי 80 מהר יותר מ- LSTM בגודל דומה על גבי GPU. זאת הודות למבנה המיוחד שלו במהלך ההסקה. יתרה מכך, המודל השיג ביצועים מתקדמים על משימות שונות תוך שימוש בפחות פרמטרים משמעותית. לדוגמה במבחן psMNIST ו- QQP השיג המודל תוצאות שיא תוך שימוש בעד פי פחות 1000 פרמטרים. ובניסוי על מאגר הנתונים IMDb השיג דיוק מרבי תוך שימוש בפחות מפי 500 פרמטרים בהשוואה לLSTM, תוצאות דומות נצפו במבחן על Quora. נתונים אלה מדגימים את היתרון של המודל ביעילות החישובית ובדרישות הפרמטרים הנמוכות [2].

מודל Mamba מציג יעילות יוצאת דופן בטיפול ברצפים ארוכים, עם סקלאבליות לינארית ביחס לאורך הרצף. דבר זה נובע משימוש במנגנון סריקה סלקטיבי במקום קונבולוציות, מה שמפחית את עלויות הגישה לזיכרון. המודל עם 74K פרמטרים השיג תוצאות מצוינות במגוון משימות סינתטיות. בהשוואה, מודלים אחרים, במבחנים דומים כמו H3 דורשים יותר פרמטרים 153K כדי להשיג ביצועים דומים או נחותים יותר בחלק מהמקרים. יכולתו של Mamba לעבד רצפים ארוכים במיוחד במשימות שונות, ולשמור על ביצועים טובים יותר עם הקשר מתארך בהשוואה לטרנספורמרים, מעידה על יעילותו בטיפול בתלויות ארוכות טווח ללא עלות חישובית מופרזת או מספר פרמטרים גדול באופן ניכר [6].

השוואת מספר הפרמטרים:

מודל	מספר פרמטרים
HiPPO-LegS	512 יחידות חבויות (מספר פרמטרים לא ידוע)
LMU	102K
FF-LMU	165K
Mamba	משתנה, נע בין 125M ל-6.9B בהתאם ליישום.

מכאן ניתן לראות כי FF-LMU דורש יותר פרמטרים מ- LMU, אך מספק יעילות גבוהה יותר.
HiPPO מחזיק זיכרון קומפקטי, אך אינו מספק יתרון חישובי ברור במודל מסובך. לעומת זאת Mamba, משתמש במספר פרמטרים גבוה משמעותית אך מצליח לספק ביצועים טובים יותר עם פחות משאבי חישוב יחסית למודלים גדולים כמו טרנספורמרים.

4. LMU בעולם החישוב הניומורפי

מודל ה- LMU פותח ככלי עוצמתי ללמידת רצפים ארוכי טווח, תוך שימוש בייצוגים קומפקטיים ודינמיקה לינארית מבוססת פולינומים אורתוגונליים. בזכות מבנהו המתמטי והקשר הישיר שלו למערכות מצב מרחב LMU, מתגלה כמודל אידיאלי לשילוב בעולם החישוב הנוירומורפי. תחום המתבסס על עקרונות ביולוגיים של המוח, במיוחד על חישוב מבוסס ספייקים. חישוב נוירומורפי מתמקד בשמירה על יעילות אנרגטית גבוהה ועיבוד מקבילי, ומודלים כמו LMU, שמאפשרים ניהול זיכרון מדויק בזמן רציף, משתלבים בו באופן טבעי.

בפרק זה אסקור כיצד ניתן להטמיע את LMU בתוך פלטפורמות נוירומורפיות, תוך התייחסות להבדלים המהותיים בין רשתות נוירומורפיות לרשתות סטנדרטיות וליתרונות של גישות מבוססות ספייקים. לאחר מכן נתמקד בשני כיוונים עיקריים של שילוב LMU במרחב זה: L2MU, המממש את LMU על גבי נוירונים ספייקיים באופן נאמן למודל המקורי, ו- LMUFormer, המציג מבנה ארכיטקטוני חדשני המשלב בין LMU לבין רעיונות של טרנספורמרים תוך התאמה לעיבוד מקבילי. לבסוף, נסקור את SpikMamba מודל המשלב SSM דינמי עם ספייקים, כהרחבה ישירה של עקרונות LMU ו- Mamba אל תוך הסביבה הנוירומורפית.

4.1 שילוב LMU בחישוב נוירומורפי

יחידת הזיכרון LMU תוכננה מלכתחילה כיחידת זיכרון חישובית רציפה, הממפה את ההיסטוריה של הקלט על פני חלון זמן באמצעות הטלה על בסיס של פולינומים אורתוגונליים. תכונה זו, יחד עם הדינמיקה הלינארית והמבנה המבוסס משוואות דיפרנציאליות הופכת את LMU למועמד טבעי להטמעה במערכות נוירומורפיות. תחום שבו השילוב בין יעילות חישובית, זיכרון לטווח ארוך, והתאמה לעיבוד בזמן אמת הוא הכרחי. המחקר המקורי על LMU [3] הדגים כיצד ניתן לממש LMU בעזרת נוירונים מסוג Poisson spiking, תוך שמירה על קנה מידה לינארי בזמן ובזיכרון, ודיוק הולך ומשתפר ככל שמספר הנוירונים גדל. פיתוחים נוספים, כמו L2MU [8] הראו שמימוש נאמן של LMU באמצעות נוירוני LIF סטנדרטיים מאפשר לממש את יכולות הזיכרון של LMU גם על גבי חומרות קצה מסחריות, מבלי להזדקק למערכות קידוד חיצוניות.

כדי להבין את התאמת ה LMU למערכות נוירומורפיות, חשוב לעמוד על ההבדלים העקרוניים בין רשתות נוירומורפיות לרשתות עצביות סטנדרטיות, ובפרט על עקרונות הפעולה של רשתות מבוססות ספייקים.

בניגוד לרשתות רגילות הפועלות במבנה דיסקרטי ובקצב אחיד, רשתות נוירומורפיות פועלות באופן אסינכרוני ומבוסס אירועים, תוך חסכון משמעותי בצריכת אנרגיה ובדרישות החישוב. בעבור מודלים כמו LMU, המתבססים על דינמיקה בזמן רציף וניחנים ביכולת שמירה של זיכרון לאורך זמן, המעבר למימוש מבוסס ספייקים אינו רק אפשרי אלא אף טבעי, הן מבחינה מתמטית והן מבחינה חישובית. תתי הפרקים הבאים יעסקו בהבחנה זו, ויציגו את היתרונות של גישות מבוססות ספייקים ככלי משלים ואף מחזק ליכולות של LMU בסביבות נוירומורפיות.

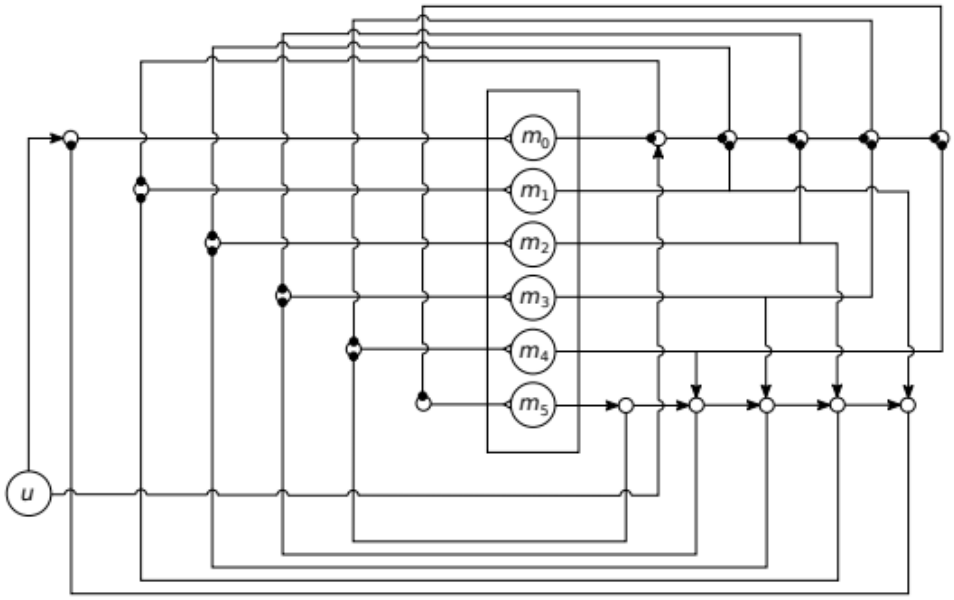
4.1.1 התאמת הדנמיקה של LMU לארכיטקטורה מבוססת ספייקים

כפי שראינו, אחד המאפיינים המרכזיים של מודל ה- LMU הוא הדינמיקה הליניארית הרציפה שלו, המנוסחת כמערכת דיפרנציאלית ממעלה ראשונה, המאפשרת לו לקודד ולהחזיק מידע היסטורי של האות בצורה קומפקטית ויעילה. דינמיקה זו מבוססת על הטלת אות הקלט על בסיס אורתוגונלי של פולינומים, אשר יוצג ייצוג פנימי של חלון זמן מתמשך של הקלט, תוך שימוש במטריצות מצב קבועות, A, B, ותלות ישירה בקלט u(t). מבנה זה מעניק ל- LMU יתרון ברור במעקב אחר מידע לאורך זמן, ויכולת לשמר תלות טמפורלית באופן יציב גם לאורך עשרות אלפי צעדים.
עם זאת, יישום של דינמיקה זו בתוך מערכת נוירומורפית, המבוססת על נוירונים מקודדי ספייקים כמו LIF, אינו טריוויאלי. הסיבה לכך נעוצה בעובדה, שבעוד LMU פועל על אות רציף בזמן ובערכים ממשיים, רשתות נוירומורפיות פועלות בד״כ על אותות בינריים, דיסקרטיים בזמן ובערך, כלומר סף ירי פעיל או לא פעיל בלבד. תהליך ההתאמה של LMU לארכיטקטורה כזאת מחייב תרגום של המשוואות הדיפרנציאליות הלינאריות למבנים דינמיים שיכולים להתבצע באמצעות נוירונים יורים ספייקים, תוך שמירה על אותה התנהגות גלובלית של זיכרון.

4.1.1.1 יישום בסיסי של LMU בעזרת ספייקים

במאמר [3] הוצג לראשונה כיצד ניתן לבצע מימוש עקרוני של LMU באמצעות נוירונים יורים ספייקים, דרך שימוש ברשת נוירונים מסוג Poisson Spiking עם חיבורים מותאמים המייצרים את הדינמיקה הליניארית הרצויה (ראה איור 20). במימוש של LMU שפועלת על ספייקים, יש לבחור את המשקלים של הקשרים (נקרא גם Synaptic Weights) בין נוירונים בצורה מדויקת. המטרה היא לגרום לכך שכאשר הנוירונים יורים לאורך זמן, הם בעצם ״יצברו״ מידע חדש על הקלט, כלומר יבצעו אינטגרציה של האותות שנכנסו לאורך זמן. הבעיה, שלא מדובר בסכימה פשוטה, הם צריכים לצבור מידע בתצורה שתואמת לפולינומי לג׳נדר, שהם הבסיס המתמטי ש- LMU משתמש בו כדי לייצג את הזיכרון. במילים אחרות, הרשת לומדת איך לפרק את הקלט לגורמים שמיוצגים על ידי פולינומים, והספייקים מייצגים את התרומה של כל רכיב כזה.

התוצאה היא שהרשת מבוססת ספייקים, לא מחשבת בדיוק את הפתרון הרציף של LMU, אבל היא כן מתקרבת אליו, כמו חישוב מדויק אך בדיד. ככל שמשתמשים יותר ביותר נוירונים ברשת, היכולת לדמות את ההתנהגות של ה- LMU הרציף משתפרת – בדיוק כמו שיותר פיקסלים בתמונה נותנים יותר פירוט.
כתוצאה מכך, מושגת דינמיקה שמהווה קירוב חישובי לא רציף אך מדויק לפתרון משוואות ה- LMU.



איור 20 : מבנה חיבורים של LMU המיושם בעזרת נוירונים מבוססי ספייקים [3]

איור 20 : באיור מוצג מבנה החיבורים של מודול הזיכרון במודל LMU, כאשר הוא מיושם באמצעות נוירונים ספייקים בסביבה נוירומורפית. התרשים מציג כיצד כל אחד ממשתני המצב $m_0, m_1, m_2, m_3, m_4, m_5$ משמש כאינטגרטור שמקבל קלט מאות הכניסה u(t) וגם מהמשתנים הקודמים לו בשרשרת. החצים הישירים מייצגים פעולות חיבור, ואילו החצים עם ראש עגול מציינים חיסור. מבנה זה מממש את המשוואות של LMU באמצעות תבנית חיבורים מחזורית, שבה כל משתנה זוכר את ההיסטוריה של הקלט בקצב שונה, בהתאם למקדם $(2i+1)\theta^{-1}$. תצורה זו מנהלת את הדינמיקה של נוירוני ספייקים ליצירת ייצוג קומפקטי ומתמשך על מידע לאורל זמן, ומהווה בסיס למימוש חסכוני באנרגיה של LMU על גבי חומרה נוירומורפית.

4.1.1.2 יישום מתקדם של LMU בעזרת ספייקים

פיתוח מתקדם יותר ניתן לראות במאמר [8] שבו הוצג מדל L2MU, מימוש של LMU בעזרת נוירונים מסוג LIF, ללא צורך בהמרת קלט לקידוד ספייקים חיצוני. במקום לבנות את LMU בעזרת רשת רגילה ואז לנסות להתאים אותה לרשת נוירומורפית שמבוססת על ספייקים, המודל הזה נבנה מראש כך שיפעל ישירות עם נוירונים מסוג LIF. היתרון המרכזי במודל זה הוא שאין צורך להמיר את הקלט לפורמט מיוחד של ספייקים, המערכת יודעת לקבל קלט רציף, ולטפל בו באמצעות נוירונים שפועלים באופן מדמה פעולה ביולוגית. במודל L2MU, כל אחד מחלקי הLMU – החלק שקולט את המידע, הזיכרון הפנימי, והפלט, ממומש על ידי שכבות של נוירונים שפועלים לפי חוקי LIF. כלומר הם דולפים עם הזמן – שוכחים באופן הדרגתי, ויורים כשעוברים סף מסוים.

כדי לגרום לזה לעבוד, תיאמו בין הפרמטרים של LMU המקורי, לבין ההתנהגות של נוירוני LIF. כך מתקבל מימוש טבעי של LMU שמתאים לריצה גם על חומרות פשוטות וזולות כמו Raspberry Pi. ועדיין שומר על ביצועים טובים במשימות זמן רציף כמו זיהוי תנועות ממידע חיישני.

נקודה חשובה נוספת היא שהעקרונות הדינמיים של LMU, המבוססים על שמירה קומפקטית של מידע לאורך זמן באמצעות מערכת מצב ליניארית, יכולים להשתלב בתוך מודלים מתקדמים יותר ממשפחת SSM. במאמרים כמו SpikMamba ו- LMUFormer [9][7], נעשה שימוש בעקרונות דינמיים דומים לאלו של LMU בתוך מודלים מתקדמים ממשפחת SSM כמו S4 ו- Mamba שמותאמים לעבודה בסביבה נוירומורפית. מודלים אלו שומרים על התכונות המרכזיות של LMU, כמו יכולת זיכרון לאורך זמן ודינמיקה רציפה, אך מוסיפים יכולות נוספות כמו התאמה לארכיטקטורות מבוזרות, תמיכה ב attention או קידוד מבוסס ספייקים.
כך למשל, ב- SpikMamba שולבו בלוקים של SSM הפועלים עם דינמיקה דמוית LMU בתוך שכבות מבוססות ספייקים, כולל Spiking Attention ומודולים של עיבוד קדמי FFN הפועלים אף הם באמצעות ספייקים בלבד. בצורה זו, מתקבלת מערכת שלמה שבה העיבוד הטמפורלי והמרחבי מתבצע בצורה חסכונית, מדויקת ואפקטיבית וכל זאת בסביבה ניורומורפית בלבד, ללא הסתמכות על עיבוד רציף רגיל.

מכל האמור לעיל עולה כי הדינמיקה של LMU למרות מוצאה הרציף והליניארי, ניתנת להתאמה ישירה לעקרונות פעולה של מערכות ניורומורפיות. היכולת של נוירונים ספייקיים לממש משוואות דיפרנציאליות באמצעות סכימה אסינכרונית של ספייקים, והעובדה ש- LMU נבנה על מודל של מערכת מצב ליניארית, מאפשרות תרגום טבעי של מנגנון הזיכרון של LMU למרחב פעולה של SNN. שילוב זה יוצר תשתית עקרונית למימושים יעילים, גמישים וביולוגית עקבית של מודלים מתקדמים בעולם החישוב הנוירומורפי.

4.1.2 יתרונות ייחודיים של LMU בסביבה נוירומורפית

בעוד הפרק הקודם התמקד באתגרים בהטמעת LMU ברשתות מבוססות ספייקים, בפרק זה נדון בפוטנציאל הגלום בגישה זו. למעשה מעבר להתאמה המבנית בין הדינמיקה של LMU לבין מנגנונים נוירומורפים, קיימים גם יתרונות ממשיים שמתקבלים כאשר משלבים את העקרונות המבססים את LMU בתוך סביבה שפועלת לפי עקרונות ביולוגיים. ברשתות עצביות אסינכרוניות, חסכוניות באנרגיה, עם ייצוג מבוסס אירועים. יתרונות אלו נוגעים לשני תחומים מרכזיים: יעילות חישובית וצריכת אנרגיה נמוכה מצד אחד, ויכולת שמירה על תלות טמפורלית וזיכרון לטווח ארוך מאידך. שני נושאים אלו מהווים את מוקד הדיון בפרק זה.

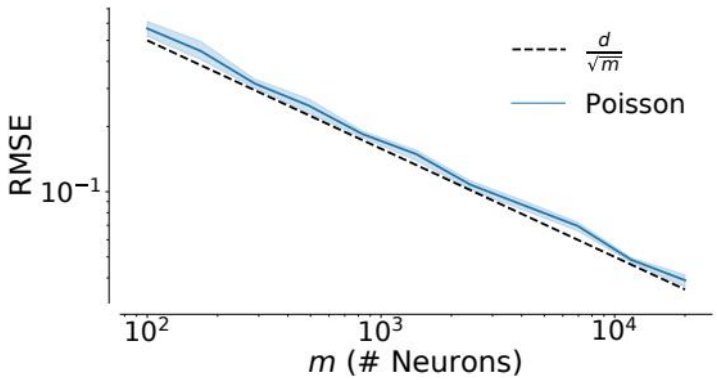
4.1.2.1 יעילות חישובית וצריכת אנרגיה נמוכה

אחד היתרונות המרכזיים בשילוב LMU עם רשתות נוירומורפיות הוא היכולת לצמצם באופן משמעותי את עלות החישוב והאנרגיה, תוך שמירה על ביצועים תחרותיים. LMU בשונה ממודלים כמו LSTM או טרנספורמר (בגרסתו הבסיסית), פועל באמצעות דינמיקה ליניארית פשוטה ולא זקוק למנגנוני gating מורכבים. לכן, כבר ברשת רגילה הוא מציע יעילות. אך כאשר LMU מיושם על גבי רשת מבוססת ספייקים, כמו ב Spiking LMU-Former [7] (אותו נציג בהמשך) החיסכון הופך למשמעותי במיוחד. במודל זה החוקרים הראו כי ניתן לממש את יחידת הזיכרון של LMU יחד עם שכבות עיבוד מבוססות ספייקים כמו Spiking Attention ו- Spiking FFN. תוצאה זו הובילה להפחתה של פי 27.2 מצריכת האנרגיה לעומת LMU רגיל, שכבר היה חסכוני האנרגיה. ההישג המרשים הזה התאפשר בעיקר בזכות שתי סיבות: Sparse Activation לפיה, רק מעט מאוד נוירונים יורים בכל זמן נתון, ולכן מתבצע פחות חישוב בפועל. והסיבה השנייה היא שימוש בפעולות בינאריות במקום מכפלות יקרות.

מיותר לציין שגם מבחינת זיכרון LMU מתאים במיוחד לחומרות קצה. ב- L2MU למשל, יושם LMU בעזרת נוירוני LIF בלבד, על גבי Raspberry Pi ומיקרו בקרים, והצליח להריץ משימות כמו סיווג תנועה בזמן אמת תוך שימוש בכמות זיכרון קטנה מאוד, וללא צורך ביחידת GPU חיצונית. המאפיינים הייחודים של LMU מובילים לכך שהוא לא דורש שכבות רבות או מבנה ארכיטקטוני עמוק, בזכות יכולתו לייצג את רצף הזמן כולו במבנה קומפקטי. זהו יתרון מובהק בעולם נוירומורפי, שבו כל שכבה נוספת מתורגמת לעלייה בצריכת חשמל ובמשאבים חומרתיים.

4.1.2.2 שמירה על זיכרון לטווח ארוך ועמידות טמפורלית

שמירה על מידע לאורך זמן, או במילים אחרות, זיכרון טמפורלי, היא אתגר מרכזי ברשתות עצביות, ובמיוחד ברשתות נוירומורפיות. רשתות מבוססות ספייקים מתקשות לעיתים לייצג הקשרים ארוכי טווח, שכן המידע מיוצג כסדרה של אירועים בינריים (ספייקים) שמאבדים את ההיסטוריה מהר מאוד אלא אם נבנה מנגנון ייעודי לכך. כאן בדיוק נכנס היתרון הייחודי של LMU מודל שפותח מראש לשם שמירה מדויקת, קומפקטית ובת שליטה של היסטוריית הקלט. בעוד ישנם מודלים נוירומורפים הנשענים על דינמיקות פנימיות אקראיות, שאותן יש ללמוד או לאפיין דרך אימון ממושך. LMU, מציע מנגנון זיכרון מובנה, מתמטי ואורתוגונלי, שבו כל רכיב במערכת אחראי על ייצוג תחום זמן שונה, במדויק. כאשר LMU מיושם בסביבה נוירומורפית בין אם על ידי קבוצת נוירונים מסוג Poisson spiking [2] או על ידי נוירוני LIF, הוא שומר על תכונות הזיכרון הללו, תוך התאמה מושלמת לעקרונות פעולה אסינכרוניים, חסכוניים ודינמיים. שכן ככל שיש יותר נוירונים ניתן לייצג את האות בדיוק יותר גבוה (ראה איור 21)



איור 21 : הקשר בין מספר הנוירונים במודל לבין שגיאת השחזור במימוש של LMU באמצעות Poisson neurons [2]

איור 21 : ניתן לשמר את איכות הזיכרון של LMU גם כאשר כל רכיב מיוצג על ידי קבוצה של נוירונים יורים, תוך שמירה על יחס שגיאה של $O\left(\frac{d}{\sqrt{m}}\right)$. משמעות הדבר היא שככל שמגדילים את מספר הנוירונים, מתקבל זיכרון מדויק יותר. תכונה קריטית כאשר פועלים בסביבות נוירומורפיות שבהן ניתן לשכפל קבוצות נוירונים באופן מקבילי וזול אנרגטית.

יתרה מזאת, ראינו כיצד ניתן ליישם את מנגנון הזיכרון של LMU כולו באמצעות נוירונים LIF בלבד, כולל קידוד הקלט, הזיכרון הפנימי והפלט, וכל זאת ללא צורך בהמרת הקלט לספייקים מראש .

הייחוד של LMU בסביבה נוירומורפית טמון בכך שהוא משלים את החולשות של SNN טיפוסיים. הוא מספק מנגנון זיכרון מדויק, עם שליטה על עומק הזיכרון כפי שניתן לראות באיור 21 (באמצעות פרמטרים כמו d ו- θ), והוא ניתן להטמעה מלאה על נוירונים מבוססי ספייקים, מבלי להקריב לא את היעילות האנרגטית ולא את רציפות העיבוד. בכך LMU, הופך לאחד המודלים הבודדים שמסוגלים להציע גם עמידות טמפורלית וגם התאמה מבנית טבעית לעקרונות נוירומורפים.

4.2 הרחבות ושיפורים ל-LMU בחישוב נוירומורפי

לאחר שעמדנו על ההתאמה המבנית והתיאורטית של LMU לעקרונות החישוב הנוירומורפי, ועל היתרונות הייחודיים שהוא מציע בסביבה מבוססת ספייקים, פרק זה מתמקד בהרחבות ובשיפורים בחישוב נוירומורפי עבור מודלים המבוססים LMU. מטרתן לממש את הפוטנציאל הגלום בו בצורה יעילה, מדויקת ורחבת היקף. השיפורים הללו נוגעים הן לאופן שבו המודל ממומש בפועל על גבי נוירונים יורים, והן לדרכי שילובו בארכיטקטורות עיבוד מתקדמות, שמסוגלות להתמודד עם משימות מורכבות בזמן אמת.

נבחן שלושה כיוונים עיקריים שבהם התפתח ה- LMU בהקשרים נוירומורפים: תחילה נסקור את L2MU, מימוש ישיר ויעיל של LMU על גבי נוירוני LIF, שמבצע את כל שלבי העיבוד, קידוד, זיכרון ופלט בתוך רשת מבוססת ספייקים מלאה. בהמשך נעסוק במודל LMUFormer, אשר משלב את מנגנון הזיכרון של LMU בתוך ארכיטקטורת טרנספורמר מותאמת לחישוב מבוסס ספייקים. לבסוף, נבחן את SpikMamba, מודל מתקדם המבוסס על שילוב של דינמיקת Mamba עם יחידות מבוססות ספייקים, כהרחבה טבעית של עקרונות LMU לעיבוד טמפורלי דינמי ברזולוציה גבוהה ובחומרה חסכונית.

4.2.1 L2MU: יישום נוירומורפי טבעי של LMU באמצעות נוירוני LIF

אחד האתגרים המרכזיים בהטמעת מודלים כמו LMU בסביבה נוירומורפית הוא מציאת דרך לשמר את הדינמיקה הלינארית והרציפה של המודל, שמהווה את ליבת הזיכרון שלו. המודל L2MU מהווה מענה ישיר לאתגר זה, בכך שהוא מציע יישום למודל של LMU, אך תוך שימוש בלב הארכיטקטורה הנוירומורפית, באמצעות נוירונים מסוג LIF. זהו לא רק פתרון טכני, אלא קפיצת מדרגה, ליצירת LMU שבנוי כולו מיחידות זיכרון ביולוגיות מדמות. המעבר לייצוג נוירומורפי טבעי נובע מהרצון לנצל את העקרונות של חישוב עצבי ביולוגי, המאופיין בצריכת אנרגיה נמוכה ועיבוד אירועי ודליל, תוך שמירה על יכולות שימור הזיכרון ארוך הטווח של LMU.

ב- L2MU כל אחד מהמרכיבים של LMU: ממיר הקלט, יחידת הזיכרון, והפלט, מתורגמים ליחידות שמבוססות על נוירוני LIF. נוירונים אלו, שמדמים תהליך של אינטגרציה, דליפה, וסף ירי, מאפשרים לדמות את הדינמיקה של LMU מבלי להזדקק לחישובים רציפים או לפעולות לא ביולוגיות. ההתאמה הזו איננה מקרית, אלה מבוססת על תכנון מתמטי מדויק, שבו קבועי הזמן, משקלי החיבורים והמבנה של הרשת מתואמים עם משוואות המצב המקוריות של LMU.

4.2.1.1 הליבה של המודל: מימוש נוירומורפי של מרכבי LMU באמצעות LIF

המודל מציג שיטה לזיהוי פעילות אנושית HAR, תחום שעוסק בזיהוי אוטומטי של פעולות אנושיות על בסיס מידע הנאסף מחיישנים. בניית מודל נוירומורפי פירושה שאיבה של השראה מחישוב מבוסס אירועים ודליל. עם זאת, בדיוק כפי שהמוח האנושי חייב להתמודד עם אותות מהעולם האמתי, הנחשבים לרציפים, כך גם כל רשת עצבית SNN חייבת להיות מסוגלת לתקשר עם חיישנים.
מודלים בזמן אמת לא יכולים לעבוד עם נתונים אינסופיים, ולכן חייבים לחלק את האותות לחלונות זמן קצרים. החוקרים במאמר השתמשו בחלונות זמן של 2 שניות, מה שמתאים לתפיסת פעילות אנושית. כאשר עבור כל חיישן יש 40 מדידות שמרכיבות חלון זמן בודד.

L2MU ממומש באמצעות אוסף של נוירונים המקיימים אינטראקציה באמצעות ספייקים, בדומה למערכות עצביות ביולוגיות. בליבת הארכיטקטורה של המודל קיימות שלוש אוכלוסיות של נוירונים עיקריים המממשים את מרכיבי הLMU המקורי (מצב הזיכרון m, המצב החבוי h, וייצוג הקלט u). כל אחת מהאוכלוסיות האלה מבוססת על נוירונים מסוג LIF או CuBa-LIF [23], כאשר כל נוירון באוכלוסייה מקבל קלט סינפטי, צובר מתח בממברנה בהתאם לדינמיקה הפנימית שלו, ויוצר ירי כאשר המתח חוצב סף מוגדר מראש.

<u>אוכלוסייה U</u>: אחראית על עיבוד ראשוני של הקלט. היא אחראית על ייצוג הקלט u לליבת ה- LMU הנוירומורפי. כזכור ב- LMU המקורי, הקלט u_t מחושב כשילוב לינארי של הקלט החיצוני x_t, המצב החבוי מהצעד הקודם h_{t-1} ומצב זיכרון מהצעד הקודם m_{t-1}. ב- L2MU, שלב זה ממומש באמצעות אוכלוסיית נוירונים המקבלת קלט משלוש אוכלוסיות נוירונים אחרות המייצגות מרכיבים אלו (או מהקלט החיצוני ישירות לאחר מודול הקידוד, ראה איור 22). האינטראקציה בין אוכלוסיות אלו מתווכת על ידי ספייקים, והקלט הנוכחי לאוכלוסיית U מחושב באופן המקביל למשוואה 3:

<u>משוואה 35:</u>

$$u_{t,curr} = e_x^T \cdot x_{t,spk} + e_y^T \cdot h_{t-1,spk} + e_m^T \cdot m_{t-1,spk}$$

כאשר e_x^T, e_m^T, e_y^T מייצגים קידוד משקלים, והסיומת spk מציינת את פעילות הספייקים של האוכלוסייה המתאימה. פעילות הספייקים מופקת כאשר המתח הממברנלי של הנוירונים באוכלוסייה זו חוצה סף מסוים: $u_{t,mem} > \theta^u$.

<u>אוכלוסייה M</u>: אוכלוסייה זו מממשת את מצב הזיכרון m של ה-LMU. האוכלוסייה מנהלת את הקשרים בין שלבים קודמים, ומטפלת בקשרים לטווח ארוך. מבנה זה מאפשר לרשת ללמוד את הדינמיקה של התנועה ולזהות דפוסים על ציר הזמן, תוך שימוש בחישוב מבוסס ספייקים, שחוסך באנרגיה ומשפר את הדיוק. ב- LMU המקורי, מצב הזיכרון m_t מתעדכן בהתבסס על מצב הזיכרון הקודם m_{t-1} ועל הקלט u_t בהתאם למשוואה 12. ב- L2MU, עדכון מצב הזיכרון מתבצע באמצעות אוכלוסיית נוירונים המקבלת כקלט מאוכלוסיית M מהצעד הקודם ומאוכלוסיית U.

הקלט הנוכחי ל- M מחושב באופן מקבילי למשוואה 12:

משוואה 36:

$$m_{t,curr} = \bar{A} \cdot m_{t-1,spk} + \bar{B} \cdot u_{t,spk}$$

כאן כל מרכיב בזיכרון m ממומש על ידי קבוצה של נוירונים, והחיבורים ביניהם (שנושאים את ערכי A ו- B) מווסתים את אופן הצבירה והדליפה של מידע. זה מאפשר למודל לייצג זיכרון דינמי רציף בתוך מערכת ספייקית לא רציפה, בכך מביאים את הדינמיקה הלינארית של LMU לסביבה ביולוגית.
A, B הן מטריצות המצב מרחב הדיסקרטיות של ה- LMU. פעילות הספייקים של אוכלוסייה M מופקת כאשר המתח הממברנלי של הנוירונים באוכלוסייה הזו חוצה רף: $m_{t,mem} > \theta^m$

אוכלוסייה H: אוכלוסייה זו מממשת את המצב החבוי h של ה-LMU. במודל המקורי, המצב החבוי h_t מחושב כפונקציה לא לינארית של הקלטx_t, המצב החבוי הקודם h_{t-1} ומצב הזיכרון הנוכחי m_t, כפי שמתואר במשוואה 13. בL2MU, עדכון המצב החבוי מבוצע באמצעות אוכלוסיית נוירונים המקבלת קלט מהקלט החיצוני $x_{t,spk}$, מאוכלוסיית H מהצעד הקודם $h_{t-1,spk}$ ומאוכלוסיית M של הצעד הנוכחי $m_{t,spk}$. הקלט הנוכחי לאוכלוסייה H מחושב באופן מקבילי למשוואה 13:

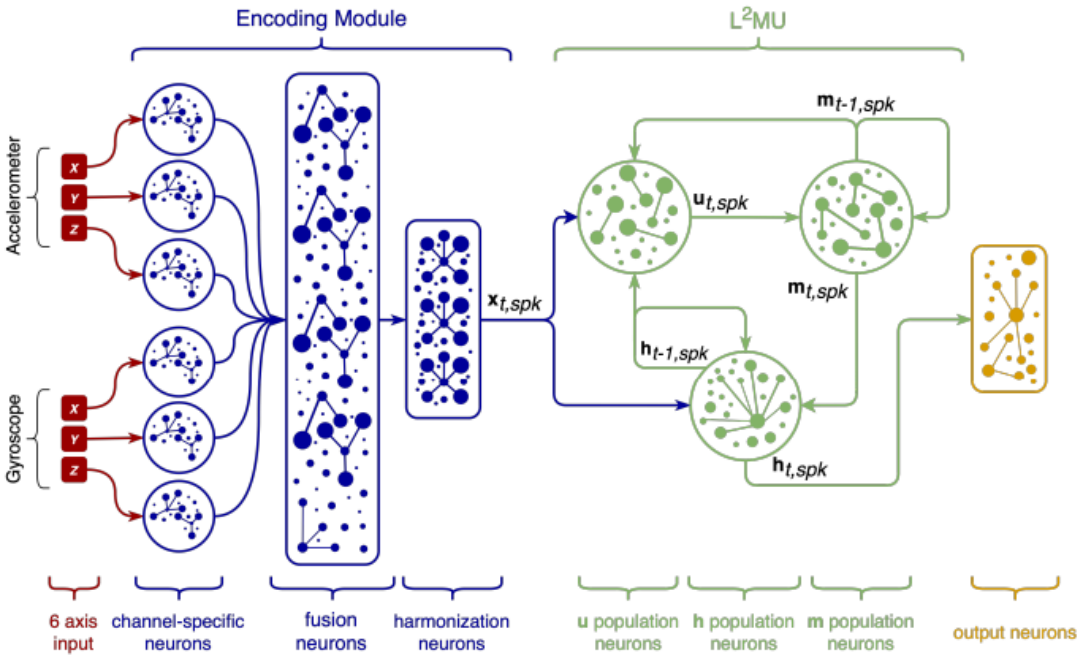
משוואה 29:

$$h_{t,curr} = W_x \cdot x_{t,spk} + W_h \cdot h_{t-1,spk} + W_m \cdot m_{t,spk}$$

כאשר W_h, W_x, W_m מייצגים משקלים סינפטיים נלמדים בין האוכלוסיות השונות. פעילות הספייקים של אוכלוסייה H מופקת כאשר המתח הממברנלי של הנוירונים באוכלוסייה זו חוצה רף: $h_{t,mem} > \theta^h$
המרכיב הזה מעבד את המידע שמצטבר, שולח אותו לאוכלוסיות אחרות ומשפיע על הקלט הבא, ממש כמו ברשת עצבית רגילה, אך כל זאת בצורה ביולוגית שמבוססת נוירונים בלבד. אוכלוסייה h מעבדת מידע קצר טווח וקשורה לקשרים בין שלבים עוקבים. הפעילות של האוכלוסייה היא כמו "מחשבות" או "מסקנות ביניים" שהרשת הגיעה אליהן.

הספייקים מאוכלוסייה H מועברים לשכבת הפלט, אשר מבצעת את הסיווג או את המשימה הרלוונטית.
השימוש באוכלוסיות נוירונים עם מודלי LIF ו- CuBa-LIF [23] מאפשר לממש את הפונקציונליות של LMU תוך ניצול המאפיינים הנוירומורפים של חישוב בדיד ודליל, ובכך לספק פתרון יעיל אנרגטית לעיבוד נתונים תלויי זמן על חומרה קונבנציונלית.

כדי להבין את הקשר בין הרכיבים השונים במודל כפי שמופיע באיור 22, אפשר להסתכל על המידע הנכנס לרשת (הקלט הגולמי, האותות מהחיישנים) הוא כמו מידע שמגיע מהעולם האמתי, המספר הזה הוא בד"כ ערכים רציפים. המידע הזה עובר ישר למודול הקידוד עליו נרחיב בהמשך, שמתפקד כמתרגם, הוא מקבל את המספרים מהחיישנים והופך אותם לשפה חדשה שהרשת L2MU יכולה להבין, השפה החדשה הזאת היא ספייקים.
ממודול הקידוד המידע מועבר לאוכלוסיות הנוירונים U, M ו-H, כל אוכלוסייה היא קבוצה של נוירונים, הם מקבלים את הספייקים ממודול הקידוד או מאוכלוסיות אחרות ומעבדות אותם. את המסקנות שלהן מעבירות לנוירוני הפלט, נוירוני הפלט הם השלב הסופי של הרשת, והם אלה שנותנים את התשובה הסופית של המודל.



איור 22 : ארכיטקטורת מודל L2MU [8]

איור 22 : התרשים מציג את המבנה הכולל של מודל L2MU, המאפשר לעבד מידע מחיישנים בצורה נוירומורפית. המודל עושה זאת באמצעות נוירונים שפועלים כמו תאי עצב במוח. מצד שמאל רואים את הקלט : שישה ערוצים שמייצגים נתונים גולמיים מאקסלרומטר וג׳ירוסקופ. המידע הזה נכנס למודל קידוד שמורכב משלוש שכבות הגנה של נויורונים : שכבה שמטפלת בכל ערוץ בנפרד, שכבה שממזגת את המידע בין הערוצים, ושכבה שמבצעת התאמה והרמוניזציה של הקלט. התוצאה היא אות מקודד בצורת ספייקים, אות בינארי שמייצג את המידע בצורה שמתאימה לעיבוד ביולוגי.

אות זה מועבר אל החלק המרכזי של התרשים - מודל ה- L2MU, המורכב משלוש אוכלוסיות נוירונים עיקריות : אחת מייצגת את הקלט המעובד u, השנייה שומרת את הזיכרון הפנימי של המערכת m, והשלישית מפיקה את הפלט החבוי h. כל אחת מהאוכלוסיות האלו בנויה מנוירונים מסוג LIF, המדמים תהליך של הצטברות ידע, דליפה וירי כשהסף נחצה. הפלט שמתקבל מהאוכלוסייה החבויה h מוזן לשכבת הפלט הסופי, שהיא זאת שאחראית על ביצוע המשימה.

4.2.1.2 מודול הקידוד: עיבוד נתוני חיישנים גולמיים לזיהוי פעילות אנושית

אחת התרומות המרכזיות של המודל היא היכולת לעבד נתוני חיישנים גולמיים ישירות, מבלי להזדקק לשלב מקדים של המרת הקלט לפורמט של ספייקים, תהליך שמקובל ברוב הרשתות הנוירומורפיות. בשיטות אחרות, הקלט עובר תחילה דרך מנגנון קידוד חיצוני שמתרגם אותו לסדרת אירועי ספייקים מלאכותיים, תהליך שלעיתים דורש כוונון ידני ומוסיף עיכוב או מורכבות. לעומת זאת, ב- L2MU ניתן להזין את הנתונים כפי שהם בצורתם הרציפה או הדגומה, אל מודול פנימי שמעבד אותם לירי של נוירונים בצורה טבעית ויעילה. זהו שלב חשוב בדרך להפוך רשת נוירומורפית ליישומית באמת, במיוחד בסביבות זמן אמת ומכשירי קצה.

ליבת התהליך הזה הוא מודול הקידוד, שמופיע בצדו השמאלי של התרשים (איור 22). מודול זה אחראי על קבלת הקלט מהחיישנים, המגיע משישה ערוצים שונים, שלושה צירים X, Y, Z מכל חיישן, המוזנים ליחידת נוירונים ייעודיות לכל ערוץ. מידע זה עובר למודול הקידוד, המורכב משלוש שכבות מרכזיות:

שכבת נוירונים ייעודית לכל ערוץ – כל אחד מהם מתמחה בעיבוד של אות ממימד מסוים.
זוהי השכבה הראשונה במודול הקידוד, מורכת מנוירוני LIF. כל ערוץ קלט מהחיישן, למשל, ציר X של מד התאוצה, מחובר לאוכלוסיית נוירונים נפרדת. אוכלוסייה זו מגדילה את ממדיות הנתונים, היא יוצרת דפוס פעילות שבא לידי ביטוי בסדרה של ספייקים המייצגים את הערך הקלט.

שכבת איחוד מידע – זוהי השכבה השנייה במודול הקידוד, היא מכילה אוכלוסיית נוירונים ספייקיים נוספת. נוירונים אלה מאחדים את המידע המגיע מאוכלוסיות הנוירונים הספציפיות מערוץ הקלט מהשכבה הראשונה. זה מאפשר לשלב מידע ממקורות שונים (למשל, נתוני תאוצה משלושת הצירים). האותות מהשכבה הקודמת מאוחדים, ליצירת ייצוג מרחבי זמני עשיר.

שכבת הרמוניזציה – ממירה את המידע לקידוד שמתאים לתוך הזנה ל- L2MU Core. כלומר היא מתרגמת את המידע לפורמט שנוירון LIF יכול להבין.

השכבות השונות מאפשרות לרשת ללמוד את הקשרים בין צירי התנועה ולשפר את ייצוג הנתונים. הן מאפשרות שילוב אופטימלי של המידע מכל החיישנים ללא צורך בקידוד חיצוני.
נוירוני הקידוד לומדים בעצמם לייצר סדרת ספייקים שמייצגת את הנתון בצורה מיטבית, על סמך מאפיינים כמו עוצמה, קצב שינוי ומבנה טמפורלי. בדיוק כמו שמערכת עצבים ביולוגית מגיבה לגירוי פיזי. כך נוצר תהליך end-to-end שבו המערכת כולה מרגע קבלת האות החיישני ועד להחלטה על הפעולה, פועלת בסביבה נוירומורפית מלאה, עם שקיפות חישובית גבוהה ובלי שכבות תיווך מיותרות.

יישום הגישה הזו הודגם במשימת זיהוי פעילות אנושית שבה נדרש המודל לזהות תנועות אנושיות שונות כמו הליכה, ריצה ועמידה על בסיס סדרות של נתוני תאוצה. בעזרת מודול הקידוד ו- L2MU הצליח המודל לעבד את הקלט בזמן אמת, לזהות תבניות תנועה מתוך הנתונים הגולמיים, ולהגיע לרמת דיוק גבוהה, וכל זאת ללא צורך בהמרת נתונים לפורמט ספייקים חיצוני או בשכבות עיבוד עמוקות. זהו הישג שמצביע על יכולת אמתית לשלב עיבוד נוירומורפי במשימות חישה חכמות, תוך שמירה על פשטות, יעילות ומהירות תגובה.

<u>השוואה לשיטות קידוד אחרות:</u>

ברוב הרשתות הנוירומורפיות המסורתיות, נתונים רציפים כמו ערכי תאוצה, קול או וידאו, נדרשים לעבור שלב קידוד לפורמט ספייקים לפני שניתן יהיה להזין אותם לרשת. ישנן כמה שיטות קידוד נפוצות שסקרנו קודם לכן: Rate Coding, Temporal Coding ועוד.
שיטות אלו אמנם שימושיות, אך הן כוללות חסרונות בולטים: הן מוסיפות מורכבות חישובית, דורשות תכנון ידני או עיבוד מקדים, ולעיתים גורמות לאיבוד מידע עדין, שכן ייצוג מבוסס ספייקים הוא תמיד

קירוב לדגם המקורי. מעבר לכך, תהליך הקידוד החיצוני אינו ביולוגי באופיו ,והוא חורג מהרעיון של רשתות end-to-end כמו הרשת שהוצגה.
לעומתן הגישה שמציג L2MU, שתומכת ב"ללא קידוד", אין צורך במתורגמן באמצע, הקלט הרציף מוזן ישירות לרשת, והנוירונים עצמם במודל הקידוד לומדים כיצד להגיב אליו ולייצר ספייקים מתאימים. המשמעות היא תהליך עיבוד שהוא גם פשוט יותר, גמיש יותר, מדמה טוב יותר את המוח, וגם חוסך באנרגיה ובזמן, מכיוון שאין צורך בשלב נוסף של תרגום או חישוב קצב ירי.

4.2.1.3 פריסה של L2M על חומרת קצה

כדי לממש את הפוטנציאל של מודלים נוירומורפים כמו L2MU ביישומי IoT וחישוב קצה, חיוני לבחון את היכולת לפרוס אותם ביעילות על פלטפורמות חומרה בעלות משאבים מוגבלים. במאמר [8] התמקדו בהערכה של פריסת L2MU על גבי מכשירי קצה מסחריים, תוך ניתוח ביצועי הדיוק בזמן אמת, צריכת הזיכרון וההספק, ובחינת טכניקות אופטימיזציה להפחתת דרישות המודל.

כפי שאפשר לראות בטבלה 5, המודלים הדחוסים של L2MU נפרסו על שלוש פלטפורמות חומרה מסחריות שונות: Raspberry Pi 3B, Raspberry Pi 4B ו- STM32MP157F. מכשירים אלו מייצגים קשת של יכולות עיבוד ומשאבים המצויות במכשירי קצה טיפוסיים. תוצאות הניסויים האמפיריים הדגימו כי L2MU, בגרסת נוירוני Leaky והן בגרסת נוירוני Synaptic, ניתנת להרצה על חומרה קונבנציונלית עם תוצאות טובות. מבחינת דיוק בזמן אמת, המודל עם נוירוני Leaky הגיע לדיוק של 93.91% עם זמן תגובה ממוצע של 0.13 שניות לכל היותר, בעוד שהמודל עם נוירוני Synaptic הגיע לדיוק של 93.84% עם זמני תגובה מעט גבוהים יותר, שנעו בין 0.07 ל-0.22 שניות. זמני התגובה הללו מצביעים על יכולת תגובה של זמן אמת עבור יישומי IoT. נסכם כי ברמת הביצועים, המודל הגיע לדיוק של מעל 93% בזיהוי הפעילויות השונות, עם זמן תגובה ממוצע של 130ms בלבד וצריכת זיכרון של פחות מ 200MB תוצאה יוצאת דופן בהשוואה למודלים נוירונים רגילים הנדרשים למשימות דומות.

על מנת לעמוד ביעדים אלו, נעשה שימוש בשתי טכניקות אופטימיזציה מרכזיות:
Pruning – תהליך שבו מסירים קשרים או נוירונים בעלי השפעה נמוכה על הפלט הסופי, מבלי לפגוע משמעותית בדיוק. הדבר הוביל לירידה בגודל הרשת ובדרישות העיבוד.
Quantization – המרה של משקלים וערכים מייצוג floating point לייצוגים מדויקים פחות, תוך איזון בין דיוק חישובי לבין חיסכון בזיכרון ואנרגיה.

Neuron model	Device	Used RAM	Mean inference time	Mean energy per inference	Accuracy (*)
Leaky	STM32MP1	65.7 MB	0.13 s	215.1 mJ	93.91%
	Raspberry Pi 3B+	77.8 MB	0.06 s	268.8 mJ	
	Raspberry Pi 4B	77.4 MB	0.03 s	153.9 mJ	
Synaptic	STM32MP1	167.9 MB	0.22 s	383.4 mJ	93.84%
	Raspberry Pi 3B+	187.5 MB	0.15 s	727.5 mJ	
	Raspberry Pi 4B	187.4 MB	0.07 s	348.9 mJ	

טבלה 5 : תוצאות הפריסה של המודל המכורץ על גבי מכשירי קצה [8]

שילוב של טכניקות אלו מאפשר לפרוס מודלים נוירומורפים מורכבים כמו L2MU על מגוון רחב יותר של מכשירי קצה מסחריים, ובכך מגשר על הפער בין מחקר נוירומורפי ליישומי עולם אמתי.
התוצאות הראו כי מודל Leaky היה מדויק יותר מהמודל המבוסס על Synaptic, בנוסף התוצאות הראו כי L2MU ניצח את LMU הרגיל בכל התצורות. כלומר, המעבר למבנה מבוסס ספייקים שיפור את הדיוק והיציבות.

LMUFormer 4.2.2

לאחר שראינו כיצד ניתן ליישם את LMU בצורה טבעית על גבי רשתות נוירומורפיות כמו ב- L2MU, פרק זה עוסק בהרחבה ארכיטקטונית שממקמת את LMU בתוך הקשר רחב יותר של מודלים מודרניים לעיבוד רצפים. LMUFormer הוא מודל חדשני המשלב את מנגנון הזיכרון של LMU עם עקרונות מבניים של טרנספורמרים מודרניים, ובמיוחד כאלה מבוססות קונבולוציה.

בשנים האחרונות טרנספורמרים, באמצעות מנגנון תשומת הלב שלהם מאפשרים לאחוז בקשרים ותלויות ארוכות טווח. המבנה המיוחד שלהם מאפשר להם לבצע את תהליך האימון במקביל. במטרה לצמצם פערים בין LMU לטרנספורמרים, וכדי לשפר את היכולת של LMU להתמודד עם מידע סדרתי ולשלב בו זמנית עיבוד מקומי וגלובלי, הוצעה ארכיטקטורה חדשה בשם LMUFormer, במודל זה משולבים מנגנוני זיכרון של LMU עם כלים מעולם הרשתות הקונבולוציוניות.
ראשית, הקלט מחולק לחלקים קטנים (patches) שעוברים עיבוד באמצעות פילטרים, כך שהמערכת מזהה דפוסים מקומיים בקלט. לאחר מכן, רכיב נוסף בשם "מיקסר של ערוצים" משלב את המידע שמגיע ממקורות שונים (למשל ערוצי אותות או תכונות שונות), ומייצר תמונה מאוחדת של המצב. כל העיבוד הזה מתמקד במצב הנוכחי של יחידת הזיכרון LMU, מה שמאפשר למודל לעבד רצפים בצורה מדויקת, תוך שמירה על יעילות חישובית ויכולת זיכרון ארוכת טווח.

מעבר לגרסה הקלאסית של LMUFormer פותחה גם גרסה נוירומורפית של המודל, שבה כל שלבי העיבוד ממומשים באמצעות נוירונים יורים. גרסה זו שומרת על עקרונות המבנה של LMUFormer, אך ממירה את יחידות החישוב למרכיבים הפועלים לפי דינמיקת ירי ביולוגית, בדומה למערכות עצביות אמתיות. כך מתקבל מודל שמסוגל לשלב בין היתרונות של עיבוד סדרתי מדויק, שימוש בזיכרון קומפקטי, וחסכון אנרגטי משמעותי ,המאפשר פריסה על גבי חומרת קצה נוירומורפית.

4.2.2.1 ארכיטקטורת LMUFormer: שילוב של LMU עם מודלים קונבולוציונים

מודל LMUFormer [7] הוא ניסיון חדשני לשלב בין שני עולמות חזקים: מצד אחד, את יחידות הזיכרון של LMU, שמצטיינות ביכולת לשמר מידע לאורך זמן. מצד שני, את רשתות העומק הקונבולוציוניות, שמצליחות בעיבוד מקבילי של מידע ומציאת דפוסים מקומיים ביעילות. השילוב הזה נועד לתת מענה לצורך במודל שמסוגל גם לזכור מידע מהעבר הרחוק וגם לעבוד בצורה מהירה ומדרגית על רצפי נתונים כמו טקסט, דיבור, או אותות רציפים.

כפי שניתן לראות באיור 23, המודל מורכב משלושה מרכיבים עיקריים:

1. Patch Embedding: בשלב הראשון, הקלט מחולק לקטעים קצרים, שנקראים patches, כל patch מכיל מספר דגימות עוקבות מתוך הרצף (למשל, מילים בטקסט או פריימים באודיו). כל patch כזה עובר דרך שכבת קונבולוציה שלומדת לתרגם אותו לייצוג מספרי, כלומר לווקטור מאפיינים שמייצג את התוכן המקומי של אותו קטע. הרעיון כאן דומה לאיך שהעין האנושית קולטת אזורים קטנים בתמונה ומעבדת אותם בנפרד.

2. Channel Mixer: לאחר שכל ה- patches עברו המרה, מגיע שלב בו המודל לומד כיצד המידע מהערוצים השונים משתלב יחד. גם כאן נעשה שימוש בשכבות קונבולוציה, אך הפעם הכיוון הוא לא מרחבי אלא בין ערוצים. כלומר, המודל לומד את הקשרים בין מאפיינים שונים בכל patch. שלב זה מאפשר למודל ליצור הבנה עמוקה יותר של הנתונים על ידי שילוב של סוגי מידע שונים בו זמנית.

3. LMU Block: לאחר שה- patches עובדו ונעשו עשירים יותר במידע, הם מוזנים למנגנון ה- LMU. כאן נכנס לפעולה הכוח של LMU, היכולת לעקוב לאורך זמן אחרי שינויים בקלט, ולשמר קשרים בין קטעים רחוקים ברצף. LMU עושה זאת על ידי ייצוג קומפקטי של ההיסטוריה של הקלט, כך שכל רגע בזמן מתבסס גם על מה שהתרחש קודם לכן. בשונה ממודלים רגילים שמתבססים על שכבות רבות כדי לזכור, LMU עושה זאת עם מנגנון מתמטי מדויק שמאפשר חישוב ליניארי בזמן, כלומר עיבוד מהיר גם של רצפים ארוכים.

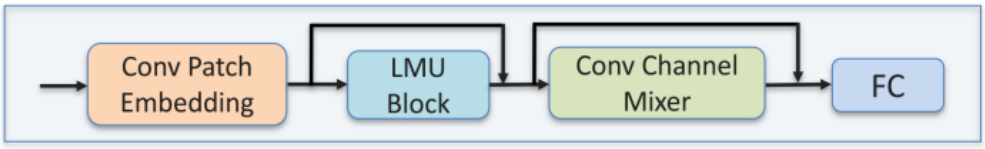
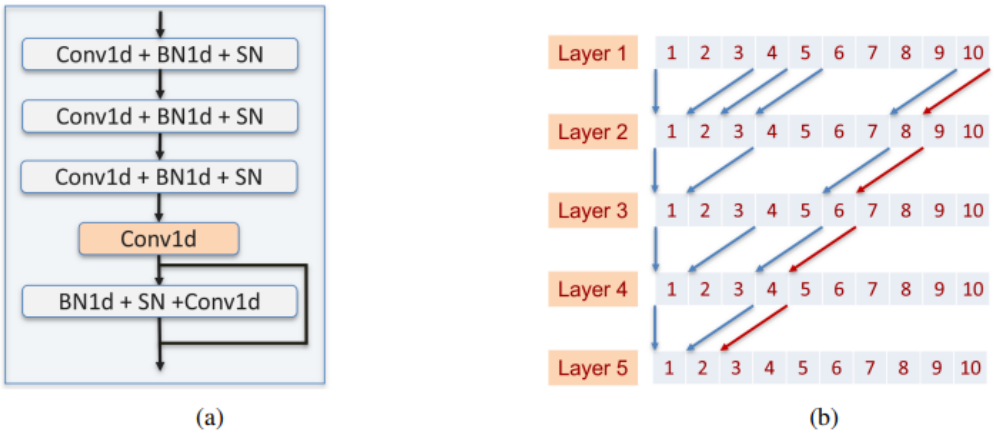


Figure 1: The overall network architectures of LMUFormer.



איור 23: ארטיקטורת LMUFormer (a) המבנה של בלוק הconvolutional patch Embedding, (b) ניתוח ההשהיה של דגימות בקלט לאורך שכבות [7]

איור 23: האיור העליון מציג את הזרימה הכללית של המידע במודל: הקלט עובר קודם דרך שכבת קונבולוציה שמחלקת אותו ל- patches, לאחר מכן הוא מוזן לבלוק של LMU שמעבד את המידע עם זיכרון לטווח ארוך, ממשיך למיקסר שמחבר בין הערוצים השונים של המידע, ולבסוף עובר דרך שכבה סופית FC שמפיקה את הפלט. בתמונה השמאלית התחתונה (a) רואים איך בנוי שלב ה"Conv patch Embedding" כמה שכבות קונבולוציה עם נרמול וסידור שמטרתן ללמוד דפוסים מתוך חלקים מקומיים של הקלט. התרשים הימני (b) מראה איך כל שכבה במודל מצליחה לכסות יותר דגימות לאורך זמן. ככל שמעמיקים בשכבות המודל קולט מידע ממקטעים רחוקים יותר ברצף. זה מדגים כיצד המודל מצליח לשלב בין עיבוד מקומי מיידי לבין שמירה על תלות ארוכת טווח.

4.2.2.2 גרסה נוירומורפית של LMUFormer

בעוד ש- LMUFormer המקורי כבר מציע יתרונות משמעותיים בשמירה על זיכרון ארוך טווח ויעילות חישובית, במאמר [7] תרגמו את הארכיטקטורה לגרסה שמתאימה לחומרה נוירומורפית, ויצרו גרסה שפועלת באמצעות נוירונים יורי ספייקים. תוצאה של ניסיון זה היא Spiking LMUFormer, גרסה חדשה של המודל שממירה את הרשת לרשת SNN, תוך שמירה על עקרונות ה- LMU והמבנה הקונבולוציוני שלו. מטרת ההמרה היא לשמר את יכולות העיבוד הסדרתיות והזיכרון ארוך הטווח של LMUFormer, תוך הפחתה משמעותית בצריכת האנרגיה והתאמה לפלטפורמות נוירומורפיות.

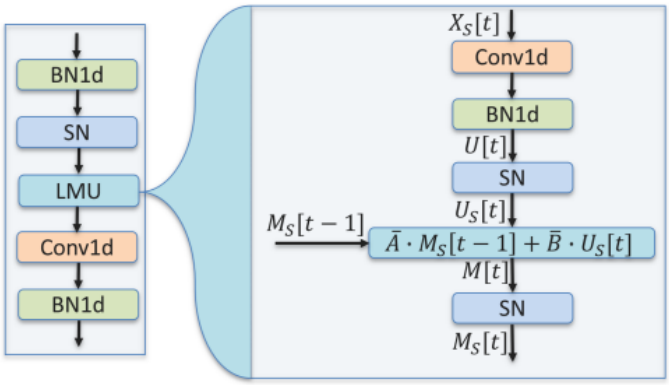
<u>מעבר ממודל רגיל למודל מבוסס ספייקים:</u>

תהליך ההמרה של המודל לרשת נוירומורפית מתחיל בשינוי הדרך שבה הנתונים מעובדים. במקום לעבוד עם מספרים רגילים, המודל משתמש בספייקים בתצורה של אותות בינאריים שמדמים ירי של נוירון. בשלב

הראשון של המודל, שבו החלק מחולק ל- patches ועובר דרך שכבת קונבולוציה, מחליפים את השכבה הרגילה בגרסה מיוחדת שמתאימה לרשתות נוירומורפיות. היא כוללת קודם כל שכבת נרמול, ואחריה שכבה של Spiking Neurons. השכבות האלה עובדות ביחד כדי לייצר רצף של ספייקים, שהם בעצם ייצוג של הקלט בפורמט שמתאים לרשת נוירומורפית. ספייקים אלו מועברים הלאה כקלט ליחידת הזיכרון של LMU.

בלב הארכיטקטורה של Spiking LMUFormer נמצא רכיב מרכזי spiking LMU cell ראה איור 24, אשר מחקה את הדינמיקה של LMU הקלאסי אך בעזרת נוירוני LIF [11]. כל צעד בזמן כולל חישוב פוטנציאל של הממברנה, בדיקת חציית רף, יצירת ספייק, ואיפוס הפוטנציאל בהתאם. הנוסחאות הפנימיות של התא מעדכנות את מצב הזיכרון והמצב החבוי באמצעות ספייקים בלבד, תוך שימוש בגרסה דיסקרטית של מטריצות המצב $\overline{A}$ ו- $\overline{B}$ שמקורת בנוסחאות LMU המקוריות (משוואה 12).
החידוש הוא שבמקום להשתמש ישירות במצב הזיכרון הרציף, הוא מוזן לנוירונים ספייקים (SN) כדי לקבל את הספייקים של מצב הזיכרון $M_S[t]$ בזמן t. לאחר מכן מצב הזיכרון והקלט הספייקי משורשרים יחד לאורך ממד התכונות ומוזנים לשכבת קונבולוציה חד ממדית, ואז לשכבת BN ולבסוף לשכבת SN כדי לקבל את הספייקים שמייצגים את הפלט של בלוק ה- LMU בזמן t.
השימוש בספייקים מאפשר חיסכון משמעותי באנרגיה חישובית. פעולות עם ספייקים צורכות פחות אנרגיה בהשוואה לפעולות כפל צבירה המשמשות ברשתות נוירונים רגילות. הדלילות של הספייקים שנובעת מהעבודה שרק חלק קטן מהנוירונים יורים בכל רגע זמן, תורמת עוד יותר ליעילות האנרגטית.

בסופו של דבר, המודל מציג ארכיטקטורה חדשנית המשלבת את היתרונות של יחידות זיכרון מתקדמות עם יעילות אנרגטית של רשתות נוירונים ספייקיות, מה שהופך אותו לפתרון מבטיח למשימות עיבוד רצפים במכשירים מוגבלי משאבים.



איור 24 : המבנה הפנימי של בלוק Spiking LMU ואת פעולת הנוירון הבודד בתרשים הימני. [7]

איור 24 : התרשים מציג את הארכיטקטורה הפנימית של בלוק ה Spiking LMU במודל.
התרשים השמאלי מראה את זרימת המידע דרך הבלוק כולו, ואילו התרשים הימני מפרק את מנגנון ה Spiking LMU הפנימי, מסביר איך מצב הזיכרון מתעדכן דרך חישוב מתמטי ונוירונים ספייקים.
בצד השמאלי של התרשים, רואים את זרימת המידע הכללית בתוך Spiking LMU block, בלוק העיבוד המרכזי. בתרשים הימני רואים מה קורה בתוך הבלוק LMU. הוא מקבל שני קלטים : הקלט הספייקי

הנוכחי $X_S[t]$ ומצב הזיכרון הספייקי מהצעד הקודם $M_S[t-1]$. שני הקלטים האלה עוברים דרך חיבורים עם משקלים מיוחדים, שמייצגים את משוואות ה- LMU, ומחשבים את מצב הזיכרון החדש $M[t]$. לאחר מכן הבלוק ממיר את הזיכרון הרציף הזה לפעולות של נוירונים יורים $M_S[t]$, מאחד אותו עם הקלט, ומעביר את התוצאה דרך שלב עיבוד נוסף, עד שנוצר הפלט הסופי.

4.2.2.3 ניסויים והשוואות ביצועים

כדי להעריך את הביצועים של המודל LMUFormer, ובפרט את גרסתו מבוססת הספייקים, החוקרים ערכו סדרת ניסויים המופיע בטבלאות 6-8 על מגוון מערכי נתונים שנועדו לבדוק יכולת של עיבוד רצפים ארוכים, דיוק, ויעילות חישובית.

בין הדאטאסטים שבהם נבדקו המודלים היו:

psMNIST – גרסה של המאגר MNIST שבה סדר הפקסלים מעורבב.

Speech Commands V2 – אוסף של מעל 100,000 דגימות אודיו של מילים קצרות.

LRA – סט של משימות לבחינת יכולת עיבוד תלות ארוכת טווח ברצפים.

Model	Sequential Inference	Parallel Training	SNN	Accuracy (%)
RNN (Bittar & Garner, 2022)	Yes	No	No	92.09
Attention RNN (De Andrade et al., 2018)	No	No	No	93.9
liBRU (Bittar & Garner, 2022)	Yes	No	No	95.06
Res15 (Vygon & Mikhaylovskiy, 2021)	Yes	Yes	No	97.00
KWT2 (Berg et al., 2021)	No	Yes	No	97.74
AST (Gong et al., 2021)	No	Yes	No	98.11
LIF (Bittar & Garner, 2022)	Yes	Yes	Yes	83.03
SFA (Salaj et al., 2021)	Yes	No	Yes	91.21
Spikformer* (Zhou et al., 2022)	No	Yes	Yes	93.38
RadLIF (Bittar & Garner, 2022)	Yes	No	Yes	94.51
Spike-driven ViT* (Yao et al., 2023)	No	Yes	Yes	94.85
LMUFormer	Yes	Yes	No	**96.53**
LMUFormer (with states)	Yes	Yes	No	**96.92**
Spiking LMUFormer	Yes	Yes	Yes	**96.12**

טבלה 6 : השוואת ביצועים על גרסאות שונות של LMUFormer במשימת זיהוי דיבור במערך הנתונים Speech Commands V2 [7]

במשימת זיהוי דיבור, המודל Spiking LMUFormer השיג תוצאה של 96.12% - תוצאה מרשימה במיוחד, כאשר משווים אותה למודלים מתחרים בעולם הSNN כמו Spikformer ו- Spike-Driven ViT. בנוסף לכך הוא השיג תוצאה כמעט זהה לגרסה הלא ספייקית של LMUFormer שהשיגה 96.53% תוך שימוש בפחות אנרגיה וחישוב.

Model	ListOps (2K)	Text(4K)	Retrieval (4K)	Image(1K)	Pathfinder (1K)	Avg
S4	58.35	76.02	87.09	87.26	86.05	80.48
Linear Trans.	16.13	65.90	53.09	42.34	75.30	50.55
Linformer	35.70	53.94	52.27	38.56	**76.34**	51.36
Transformer	36.37	64.27	57.46	42.44	71.40	54.39
BigBird	36.05	64.02	59.29	40.83	74.87	55.01
Nystromformer	37.15	65.52	79.56	41.58	70.94	58.95
LMUFormer	34.43	**68.27**	78.65	54.16	69.9	61.08
Spiking LMUFormer	**37.30**	65.80	**79.76**	**55.65**	72.68	**62.24**

טבלה 7 : השוואת ביצועים על גרסאות שונות של LMUFormer במשימות לבחינת יכולת עיבוד תלות ארוכת טווח ברצפים [7]

במשימות LRA, מודל Spiking LMUFormer השיג ביצועים טובין במיוחד במשימת ListOps הבודקת אם המודל יודע להבין ולנתח מבנים היררכיים ברצף, כמו ביטויים מתמטיים עם סוגריים, ובמשימת Retrieval שבה המודל צריך לזהות פריט מסוים מתוך רצף מידע ארוך על פי שאלה או רמז. המודל התעלה על מודלים בסיסיים מבוססי טרנספורמרים. באופן כללי, ניתן לראות כי המודל שמר על דיוק גבוה מאוד בממוצע, עם יתרון ממוצע של 0.53% על הגרסה הרגילה של LMUFormer, דבר המדגיש פעם נוספת את היכולת של המודל הספייקי לנצל את הדינמיקה הטמפורלית הפנימית לעיבוד רצפים ארוכים.

Model	Params. (M)	OPs (G)
AST (Gong et al., 2021)	86.93	12.4
LMUFormer	1.62	0.189
Spiking LMUFormer	1.69	0.0309

טבלה 8 : השוואה בין מספר פרמטרים ומספר פעולות [7]

אחד היתרונות הבולטים של Spiking LMUFormer הוא היעילות האנרגטית הגבוהה שלו. החישוב במודל מבוסס על ספייקים, כלומר רוב הנוירונים לא פעילים ולכן הוא צורך פחות משאבים. המודל משתמש בפי 400 פעולות פחות למול טרנספורמר על אותה משימה.

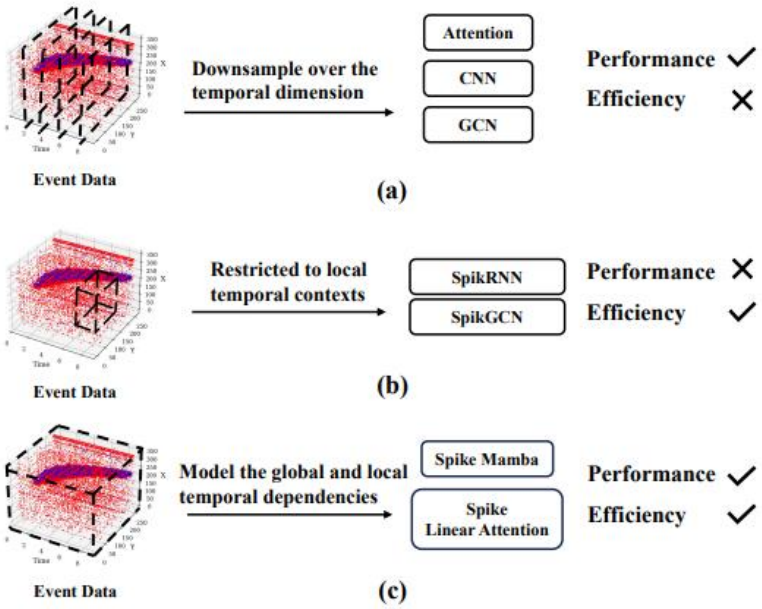
הניסויים הראו ש- Spiking LMUFormer מצליח לשמור על רמת דיוק גבוהה מאוד, לעיתים אפילו יותר מהגרסה הרגילה. תוך חסכון משמעותי באנרגיה ובכמות החישובים. המודל הפגין ביצועים מצוינים במשימות שונות והוכיח שהוא מתאים במיוחד לעיבוד סדרתי יעיל בזמן אמת.

SpikMamba 4.2.3

לאחר שראינו כיצד יחידת זיכרון מסוג LMU משתלבת במודלים מתקדמים כמו LMUFormer וכיצד ניתן לממש גרסה נוירומורפית מלאה של LMU באמצעות נוירוני LIF במודל L2MU. כעת נפנה למודל חדשני מסוג אחר, המבוסס על עקרונות דומים אך עושה שימוש ב- State Space Models [10] עיבור עיבוד נתונים נוירומורפי.

SpikMamba [9] מבוסס על מודל מתקדם בשם Mamba אשר פותח לאחרונה על מנת לאפשר עיבוד יעיל של רצפים ארוכים. מודל זה משלב את היתרונות של SNN עם היכולת המתאפיינת בחישוב חסכוני ואסיכנרוני עם היכולת של Mamba לתפוס תלות ארוכת טווח לאורך הרצף, תוך שמירה על יעילות חישובית ומספר פרמטרים נמוך. שילוב זה הופך את המודל למתאים במיוחד לעבודה עם נתוני חיישני מצלמות אירועים, המתאפיינים בדלילות מרחבית ובקצב גבוה שח שינויים עיתיים. מודל המשלב בין ראייה מקומית לבין ראייה גלובלית כמופיע באיור 25.

במקום להסתמך על מנגנוני תשומת לב מסורתיים או רשתות קונבלוציה כבדות. המודל משלב שני רכיבים עיקריים: מנגנון attention מבוסס חלונות בפורמט spikes, ובלוקים של spike Mamba המיישמים את משוואות המצב של SSM בצורה מותאמת לפלטפורמה נוירומורפית.



איור 25 : שיטות לזיהוי HAR [9]

איור 25 : באיור מוצגות 3 גישות שונות לעיבוד נתוני אירועים לצורך זיהוי HAR, הראשונה מבוססת ANN, לפיה דוחסים את רצף האירועים כדי להתמודד עם עומס נתונים גבוה, הבעייתיות בגישה זו היא עיבוד מידע עקב הדחיסה. הגישה השנייה, משתמשת ב- Spiking RNN או GCN. שם בודקים רק קשרים באופן מקומי, אך אין ראייה רחבה לאורך זמן. בגישה השלישית, משלבים בין ראייה מקומית לבין ראייה גלובלית – Spike Mamba. האיור מתאר בצורה ברורה את המוטיבציה לפיתוח המודל SpikMamba, הוא ממחיש את הפער בין הגישות הקיימות ומראה את הצורך בפתרון המשלב את שני העולמות.

4.2.3.1 ארכיטקטורת SpikMamba שילוב Mamba עם SNN

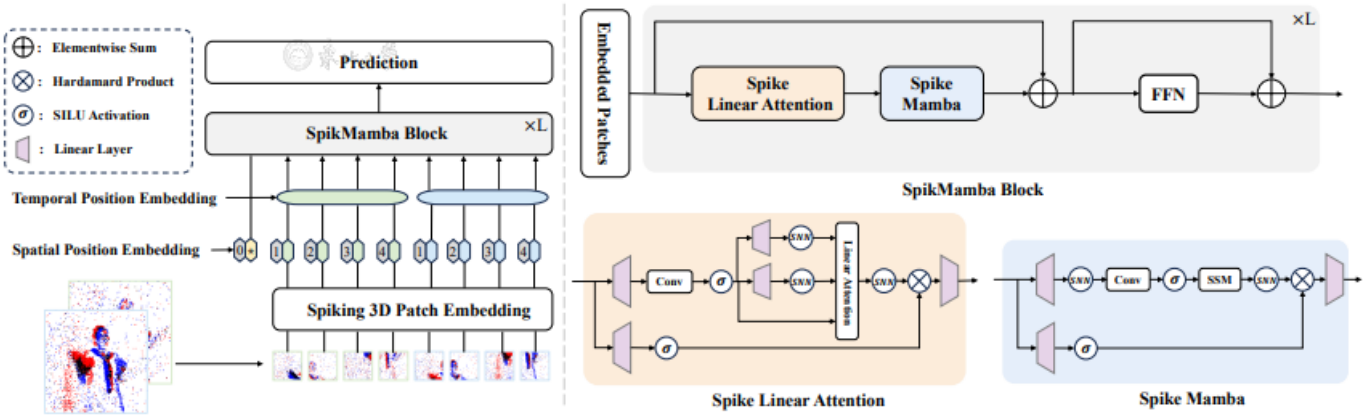
המודל נבנה במיוחד כדי לזהות פעולות של בני אדם מתוך מידע שמתקבל ממצלמות אירועים, כידוע המידע שהם מספקים מאוד שונה מתמונות רגילות, הוא מגיע בקצב גבוה מאוד, בצורה לא אחידה, והוא דליל (ברוב הזמן לא קורה כלום). כדי להתמודד עם האתגר הזה המודל משתמש בשני חלקים עיקריים כפי שמופיע באיור 26 :

Spiking patch Embedding : בשלב הראשון מידע שמגיע מחולק ל patches קטנים, כמו חלונות קטנים בתמונה, והמודל מחלץ מהם features בעזרת שכבת קונבולוציה תחת ממדית. כל patch כזה מומר לייצוג מבוסס ספייקים, במקום ערכים רגילים, המידע נשמר כסדרה של אפסים ואחדים המיצגים האם נוירון ירה באותו הרגע או לא. המודל מוסיף גם מידע על המיקום של כל patch בזמן ומרחב כדי שלא יאבד את ההקשר.

spikMamba block : אחרי שנוצר ה- patch הוא עובר דרך מספר שלבים, הראשון הוא תשומת לב לוקאלית. המודל מסתכל על קטעים קצרים בזמן ובודק מה חשוב יותר וחשוב פחות.

לאחר מכן מגיע השלב של חישוב תלות גלובלית. בחלק זה המודל מסתכל על דברים שקרו הרבה קודם, לצורך זה נכנס לתמונה מודל Mamba שהצגנו מקודם [6], שמטרתו לזהות יקשרים ארוכי טווח. הוא עושה את זה באמצעות משוואות מתמטיות שמסבירות איך המצב הפנימי של המערכת משתנה לאורך זמן, במודל מבוסס ספייקים נעשתה המרה למשוואות אלו שיפעלו גם הן עם ספייקים, כך שהחישוב נשאר יעיל ומהיר.

לבסוף המידע עובר דרך שכבת עיבוד נוספת, שמסייעת להוציא ממנו עוד תובנות לפני שהמודל מגיע לשלב החיזוי. לאחר מכן אחרי שכל ה- patches עברו את הבלוקים השונים, המודל מאחד את כל המידע ומנבא איזה פעולה מתבצעת. המודל משלב בין הסתכלות מקומית, מה שקורה עכשיו ולבין הסתכלות גלובלית לאורך הזמן שעבר.



איור 26 : ארכיטקטורת מודל SpikMamba [9]

איור 26 : באיור מוצגת תצוגה ויזואלית של הארכיטקטורה של המודל. האיור מחולק לשלושה חלקים עיקריים : בצד שמאל אפשר לראות את תהליך העיבוד המלא מהקלט עד ה- prediction. באמצע למעלה, ניתן לראות את המבנה הכללי של בלוק SpikMamba אחד, ובתחתית מופיע מבנה פנימי של שתי תתי

שכבות מרכזיות בבלוק SpikMamba הנקראות Spike Linear Attention ו- Spike Mamba שאחריות על קשרים מקומיים וגלובליים בהתאמה.

4.2.3.2 שלבי המודל

אחרי שסקרנו בקצרה על המודל נצלול לעומק למבנה שלו. המודל בנוי ממספר שלבים עוקבים, שכל אחד מהם ממלא תפקיד חשוב בזיהוי פעולה אנושית מתוך זרם הנתונים של מצלמות האירועים. להלן תיאור של שלושת השלבים המרכזיים בתהליך:

<u>השלב הראשון</u>: <u>המרת הנתונים</u>
הנתונים המגיעים ממצלמות אירועים אינם תמונות רגילות אלא רצף של נקודות שבהן התרחש שינוי בעוצמת האור, עם רזולוציה עיתית גבוהה מאוד. כדי לאפשר למודל להבין את המידע ולעבד אותו ביעילות, השלב הראשון כולל חלוקה של הנתונים ליחידות קטנות הנקראות patches. כל patch הוא חלק קטן מהתמונה שמכיל מידע מסוים.
לצורך הפקת ה- patch משתמשים בשכבת Conv3D אשר לומדת תבניות מתוך ה- patch וממירה אותם לייצוג שניתן להשתמש בו בהמשך במודל. בנוסף המודל מוסיף הטמעת מיקום מרחבית וזמנית לכל patch, כדי לשמר את מיקומו בזמן ובמרחב.

<u>השלב השני</u>: <u>תשומת לב לוקאלית</u>
בשלב זה המודל בוחן את הקשרים בין patches סמוכים לאורך ציר הזמן. חישוב תשומת לב מקומית וגלובלית, בשלב זה המודל מחפש קשרים חשובים בין החלקים שונים, על מנת לזהות דפוסים בזמן.
מחשבים לכל patch שלושה ייצוגים:
Q – השאלה, מה אנחנו מחפשים.
K – מפתח, איזה מידע קיים בpatch.
V – ערך, מה המידע החשוב בpatch.

לאחר מכן מבצעים השוואה בין query ל- Key כדי לזהות קשרים בין ה- patches, באופן דומה לתהליך המוכר לנו מהטרנספורמרים [14]. השוני העיקרי הוא שפה משתמשים בשכבות ספייקיות המעבדות את השאלה והמפתח. כלומר נוירונים שמופעלים רק כאשר המידע מספיק חזק, כמו ירי של נוירון ביולוגי. תהליך זה נעשה בתוך שכבת Spike Linear Attention ומטרתו לזהות את ההתאמות בין patch אחד לאחר לפי הדמיון ביניהם.

<u>השלב השלישי</u>: <u>חישוב תלות גלובלית</u>
כדי להבין פעולות אנושיות שלמות, לא מספיק רק לראות קשרים בין רגעים סמוכים, לעיתים צריך להבין גם קשרים בין אירועים שהתחרשו שניות קודם לכן. לשם כך המודל משתמש בגרסה ספייקית של מודל Mamba [6]. מודל Mamba מבוסס על משוואות מסוג SSM, שמתארות איך מצב פנימי של מערכת משתנה לאורך זמן בהתאם לקלט. במילים פשוטות הוא יודע לשמור זיכרון מתמשך ולהעביר מידע חשוב מהעבר אל ההווה.

ב- SpikMamba השכבה הזו מופעלת גם היא בצורה ספייקית: כל patch עובר דרך שכבת הרחבה (למשל Conv1D), וחישוב משוואות המצב מתבצע עם פרמטרים A, B, Δ שנלמדים מהקלט עצמו.
בכך המודל מצליח לראות רחוק, כלומר לתפוס תבניות עיתיות רחבות ולא רק מה קרה ברגע זה.
השילוב בין שלב תשומת הלב המקומית לשלב המצב הגלובלי מאפשר למודל לזהות גם דפוסים עדינים בטווח קצר וגם הקשרים רחבים בטווח ארוך, וזה בדיוק מה שנדרש כדי להבין פעולה אנושית מורכבת מתוך מידע מבוסס אירועים.

4.2.3.3 מעבר למימוש נוירומורפי

המודל SpikMamba נועד לפעול בסביבה נוירומורפית, כלומר בפורמט שמחקה את אופן הפעולה של מערכת העצבים הביולוגית, בעיקר באמצעות עיבוד מבוסס ספייקים. בשונה מרשתות ANN, אשר פועלות עם ערכים רציפים, נעשה כאן שימוש ב- SNN שבהן כפי שראינו קודם לכן, המידע מועבר באמצעות ספייקים, בדומה לירי של נוירון ביולוגי. עקרון פעולה זה מאפשר למודל להיות חסכוני באנרגיה, רגיש לשינויים בזמן, ולתפקד היטב עם מידע דליל ואסינכרוני, בדיוק כמו המידע שמתקבל ממצלמות אירועים.

ליבת המימוש הנוירומורפית ב- SpikMamba מבוססת על נוירונים מהסוג LIF [11]. כפי שראינו בפרקים קודמים נוירונים אלו מצטיינים בפשטות חישובית וביכולת לשמור מידע זמני. במודל כל אחת מהשכבות כולל ה- Attention ומנגנוני המעבר, בנויה על סמך נוירונים אלו, וכך נשמרת עקביות עם עקרונות חישוב ביולוגיים.

<u>התאמת מודל Mamba לפורמט מבוסס ספייקים:</u>

אחת ההברקות במימוש של המודל היא ההתאמה של מודל Mamba שמבוסס על משוואות מצב SSM לפעולה בפורמט של ספייקים. Mamba אשר במקורו תוכנן לפעול על רצפים רציפים, עבר התאמה כך שיוכל לפעול באופן נוירומורפי:

המשוואות הדיפרנציאליות של Mamba הומרו לגרסה בדידה באמצעות שיטת Zero Order hold, המתרגמת את המשוואות הרציפות לגרסה שמתאימה לחישוב שלב אחרי שלב (נספח 2).
השכבה עצמה פועלת כחלק מתהליך הכולל המרה קונבולוציונית והחרבת ממדים, בפורמט ספייקים.

לצד SpikeMamba המודל עושה גם שימוש במנגנון Spike Linear Attention, שכבת Attention מותאמת לפורמט של SNN. מנגנון זה מחשב את ה- Query וה - Key בפורמט של ספייקים, ובכך מאפשר לזהות קשרים קצרי טווח בין patches סמוכים ביעילות ובחסכנות.

4.2.3.4 ניסויים ותוצאות

כדי להעריך את יעילותו ודיוקו של המודל, המחברים ביצעו סדרת ניסויים נרחבת על ארבעה מערכי נתונים מרכזיים בתחום HAR מבוסס אירועים. מטרת הניסויים הייתה לבחון את ביצועי המודל הן בהיבטי דיוק סיווג והן בהיבטי יעילות חישובית, תוך השוואה למודלים מהשורה הראשונה, כולל מודלים מבוססי ANN ומודלים של SNN.

הניסויים המופיעים בטבלאות 9-10 התבצעו על ארבעה datasets נפוצים:

1. PAF – מאגר קטן יחסית המורכב מ10 קטגוריות פעולה, עם 45 דגימות לכל קטגוריה, שצולמו במצלמת אירועים DVS346.
2. HARDVS – מאגר רחב היקף עם 300 קטגוריות פעולה ויותר מ107,000 דגימות, נחשב לאתגרי במיוחד בשל הגיוון הרב.
3. DVSGesture – כולל 11 קטגוריות תנועה של ידיים וזרועות, ברזולוציה של 128x128 פיקסלים.
4. E-Faction – כולל 128 קטגוריות פעולה שונות, עם 1024 הקלטות ב 346x260 פיקסלים, מיועד לזיהוי פעולות מורכבות בזמן.

Dataset	Model	SNN	Acc(%)
PAF	HMAX SNN [60]	✔	55.00
	STCA [15]	✔	71.20
	Motion SNN [29]	✔	78.10
	MST [57]	✔	88.21
	Swin-T (BN) [57]	✔	90.14
	EV-ACT [11]	✗	92.60
	ExACT [66]	✗	94.83
	SpikMamba(Ours)	✔	**96.28**
HARDVS	X3D [8]	✗	45.82
	SlowFast [9]	✗	46.54
	ACTION-Net [58]	✗	46.85
	R2Plus1D [53]	✗	49.06
	ResNet18 [17]	✗	49.20
	TAM [32]	✗	50.41
	C3D [52]	✗	50.52
	ESTF [55]	✗	51.22
	Video-SwinTrans [31]	✗	51.91
	TSM [27]	✗	52.63
	ExACT [66]	✗	90.10
	SpikMamba(Ours)	✔	**97.32**
DVSGesture	Time-surfaces [34]	✔	90.62
	SNN eRBP [20]	✔	92.70
	Slayer [47]	✔	93.64
	DECOLLE [21]	✔	95.54
	EvT [46]	✔	96.20
	TBR [19]	✗	97.73
	EventTransAct [6]	✗	97.92
	ExACT [66]	✗	98.86
	SpikMamba(Ours)	✔	**99.01**
E-FAction	CLIP-L [44]	✗	61.90
	ResNet3D-N [23]	✗	65.60
	ResNet3D-K [22]	✗	66.30
	MASTAF [30]	✗	67.10
	ExACT [66]	✗	67.93
	SpikMamba(Ours)	✔	**71.02**

טבלה 9 : השוואת ביצועי המודל למול מודלים חדישים לזיהוי HAR על מגוון מאגרי נתונים [9]

טבלת התוצאות מתוך המאמר מראה כי SpikMamba השיג את אחוזי הדיוק הגבוהים ביותר בכל אחד מהמאגרים.

Model	GLOPs	#Params.
ExACT [66]	1.1	2.13M
EVT [46]	0.2	0.48M
SpikMamba(Ours)	**0.12**	**0.18M**

טבלה 10 :השוואת יעילות חישובית במשימת זיהוי HAR, בין SpikMamba לבין ExACT וEvT המובילים בתחום [9]

בנוסף לדיוק, נמדדו גם מדדים של כמות פרמטרים, וכמות חישובים. התוצאות מראות כי SpikMamba אינו רק מדויק יותר, אלא גם קל ויעיל יותר. הוא חוסך כ-91% מהפרמטרים בהשוואה למודל ExACT וכ-40% בהשוואה למודל EvT ובו בזמן משיג ביצועים טובים יותר.

ANN מול SNN: החוקרים בדקו מה קורה אם מסירים את שכבות ה-SNN והופכים את המודל למבוסס ANN בלבד. הם קיבלו דיוק נמוך יותר בהשוואה למודל המלא. ירידה בביצועים היא בממוצע 2.71%, מה שמראה שרשתות SNN מתאימות יותר לניתוח נתוני אירועים. השילוב של SNN עם Mamba מספק את היכולת הטובה ביותר ללמוד תלות גלובלית ומקומית.

5. יישומים מעשיים של LMU

בהמשך ישיר לפרקים הקודמים, בהם סקרנו את מגבלות השיטות המסורתיות לעיבוד נתונים סדרתיים כגון LSTM, RNN, וטרנספורמרים, ואת הצורך בגישות חדשות שתשלבנה יכולת זיכרון אפקטיבית יחד עם חסכון חישובי. בפרק זה אעסוק ביישומים מעשיים של LMU. לאחר שהוצגו היתרונות התאורטיים של LMU, הארכיטקטורה שלו, שיפורים וכן יכולותיו בהקשר של חישוב נוירומורפי, נבחן כעת כיצד יכולות אלו באות לידי ביטוי במצבים מציאותיים.

ראינו כי LMU מהווה דוגמה מובהקת למודל שמצליח לגשר בין עולם החישוב הרציף והביולוגי לבין הדרישות של מערכות חישוב מודרניות, לרבות מערכות הפועלות בזמן אמת, חומרה מוגבלת משאבים, או סביבת קצה. בזכות יכולתו לשמר מידע לאורך טווחים ארוכים באופן חסכוני, ולא פחות חשוב, בזכות האפשרות לשלב אותו בתוך רשתות נוירונים קלאסיות או נוירומורפיות. בפרק אסקור חמישה תחומים מרכזיים בהם נבחן השימוש הממשי במודל: עיבוד אותות ביולוגיים כגון דיבור, משימות של HAR, מערכות IOT, NLP, ו- Edge Computing.

5.1 עיבוד אותות ביולוגיים

אחת התחומים בהם בולטת היעילות של LMU הוא בעיבוד אותות ביולוגיים, כלומר מידע שמגיע מהגוף שלנו, כמו אותות מהמוח EEG או דיבור. אותות כאלה משתנים לאורך הזמן באופן רציף ומהיר, ולכן חשוב שהמערכת שתנתח אותם תדע לזכור פרטים חשובים שקרו לפני כמה שניות, ולהתייחס אליהם בזמן הנכון. ראינו כי LMU בנוי במיוחד למטרה הזאת, היא יודעת לשמור ביכרון מידע שמגיע לאורך זמן, ולעשות זאת בצורה חכמה וחסכונית.

אחד התחומים שבהם LMU כבר מוכיחה את עצמה הוא זיהוי דיבור, תחום שדורש תגובה מהירה, רגישות לפרטים קטנים ויכולת לזכור רצף של צלילים. במחקר מהזמן האחרון בשם LMUFormer, הציגו החוקרים גרסה חדשה של המודל המשלבת את עקרונות ה- LMU עם רכיבים נוספים מותאמים לרשתות מודרניות, כולל גרסה שפועלת בצורה נוירומורפית [12]. במבחן שבוצע על הדאטאסט Speech Commands V2, הצליחה גרסת המודל להגיע לדיוק מרשים של 96.12% תוך שמירה על יעילות חישובית מרשימה. מדובר בביצועים שנחשבים לשיא חדש בקרב מודלים נוירומורפים.

יחד עם זאת, כאשר בוחנים את הביצועים המובילים כיום על מאגר הנתונים [24].

נמצא כי המודל QNN השיג את הדיוק הגבוה ביותר – 98.53%, תוך שימוש ב78k פרמטרים בלבד.

אך למרות זאת, עדיין יש ל- LMU יתרונות מהותיים. LMU מציע גמישות מבנית רחבה יותר, בזכות הארכיטקטורה הרציפה שלו, התאמתו למידע מבוסס זמן, והיכולת לשלבו בקלות עם מערכות נוירומורפיות ועיבוד מקבילי של אותות ביולוגיים.

ראינו כי מודלים מבוססי LMU עשויים לשפר את ניתוח נתוני EEG, MRI ואותות אלקטרו פיזיולוגיים אחרים, בזכות יכולתו להתמודד עם תלות טמפורלית ארוכת טווח ביעילות חישובית גבוהה.

5.2 זיהוי פעילות אנושית HAR

זיהוי פעילות אנושית HAR, הוא תחום שבו נדרש לנתח תבניות של תנועה או פעילות גופנית מתוך אותות שמגיעים מחיישנים כמו אלקטרומטר וג׳יירוסקופ, שמותקנים בטלפונים חכמים או בשעונים לבישים. מדובר במשימה מאתגרת כי האותות משתנים בזמן, הם רועשים ולעיתים קרובות לא מובנים מראש. ראינו כי מודלים מסורתיים ופשוטים לא מצליחים לסווג בצורה טובה פעילות מתוך האותות, דורשים כוח חישובי משמעותי ומתקשים לפעול בזמן אמת על חומרה מוגבלת.
כדי לפתור בעיה זו, מציגים החוקרים במאמר את L2MU, גרסה נוירומורפית מלאה של LMU, הבנויה מאוכלוסיות של נוירונים מסוג LIF, ומותאמת לעבודה עם אותות חיישנים גולמיים ללא צורך בקידוד חיצוני [11].

המודל הותאם במיוחד למשימת HAR ונבחן על דאטאסט של WISDM, כולל נתוני חיישני תנועה שנאספו מטלפונים חכמים או שעונים חכמים, במטרה לזהות פעילויות אנושיות שונות.
הפעילויות חולקו לקטגוריות שונות כדי לבדוק את ביצועי המודל במגוון מצבים, והוא הראה יכולת להבחין בין הפעולות ברמת דיוק גבוהה. במהלך האימון, L2MU הראה דיוק מרשים של עד 94.14% על קבוצת הבדיקה, גם בגרסאות קומפקטיות של המודל לאחר דחיסה, תוך שמירה על מהירות חיזוי של פחות מ300 מילישניות, דבר שמאפשר שימוש במערכות קצה.

המודל הופעל על מערכות חומרה ללא צורך בשבבים נוירומורפים ייעודיים, מה שמחזק את הפוטנציאל שלו לפריסה מעשית באפליקציות רפואיות, ניטור ספורטיבי ועוד. מעבר לביצועים הגבוהים, היתרון המרכזי של L2MU הוא היותו ״encoding free״, כלומר הוא מקבל את האותות כפי שהם מהחיישן, וממיר אותם לפעילות ספייקית פנימית, מה שחוסך שלבי עיבוד מורכבים ויקרי אנרגיה שקיימים במודלים אחרים.

5.3 שילוב במערכות IOT וEdge Computing

הצורך להפעיל רשתות נוירונים על מכשירים קטנים וחסכוניים כמו חיישנים לבישים, מצלמות חכמות או רכיבי IoT תעשייתיים - מציב אתגר כפול: מצד אחד, יש לנתח מידע טמפורלי מורכב, לרוב בזמן אמת, ומצד שני יש מגבלות ברורות על זיכרון, עוצמת עיבוד וצריכת אנרגיה. כאן בא לידי ביטוי היתרון המהותי של LMU, שנבנה כיחידת זיכרון המייצגת סדרות זמן באופן מתמטי, תוך שימוש במערכת לינארית רציפה LTI החוסכת משמעותית במשאבים לעומת מודלים נשנים מסורתיים [1].

החיבור הישיר של LMU לעולמות ה- IoT הודגם במאמר על L2MU [8], שם פותח מודל נוירומורפי מלא שהתעלה בביצועים מהמודל הלא נוירומורפי, המבוסס על LMU שפעל על אותות חיישנים גולמיים והורץ על גבי חומרות מסחריות פשוטות, כמו Raspberry Pi ו- STM32. המודל השיג דיוק של מעל 93% במשימות סיווג HAR עליהם דיברנו קודם לכן, תוך חיזוי בזמני אמת ושימוש בזיכרון נמוך.
פריצת הדרך שמגיעה במודל הזה הייתה שהוא פועל כולו על ספייקים, דבר שהפחית את הצורך באנרגיה ושיפר את התאמתו למערכות קצה. יתרה מכך בזכות הארכיטקטורה שכללה את מודל הקידוד החדשני שלו, שאפשר הזנה ישירה של נתוני חיישנים ללא צורך בהמרה, אפשר יישום יעיל וחסכוני בסביבה מציאותית כדוגמת מערכת IoT ועיבוד תנועה בזמן אמת.

5.4 פוטנציאל בעיבוד שפה טבעית NLP

למודלים מבוססי LMU, וכן למימושים נוירומורפים של LMU, יש פוטנציאל ממשי ומוכח בעיבוד שפה טבעית, בעיקר במצבים שבהם נדרש איזון בין ביצועים לבין יעילות חישובית.

כפי שהצגתי קודם לכן היתרון המרכזי של LMU נעוץ ביכולתו לייצג מידע סדרתי לאורך זמן באמצעות מערכת דיפרנציאלית לינארית (LTI), תוך שימוש בבסיס פולינומי אורתוגונלי. יתרון זה תורגם לביצועים מעשיים במגוון מודלים, מהבולטים ביניהם היה FF-LMU [2], שהציע תחליף לא רק יעיל אלה גם בעל ביצועים גבוהים במשימות בעולמות NLP. לדוגמה בדאטאסט IMDB [21], הגיע המודל לדיוק של 89.1% עם פחות מ100 פרמטרים בלבד, בהשוואה למודלים סטנדרטיים כמו LSTM שהגיע ל87.3% עם יותר מ50,000 פרמטרים. במאגרים נוספים כמו SNL ו- QQP, המודל הצליח להגיע לתוצאות שקולות או טובות יותר ממודלים גדולים הרבה, תוך שהוא משתמש בפחות מ1% מהמשאבים החישוביים.

עם זאת, בהשוואה למודלים מבוססי טרנספורמרים, כמו BERT ו- GPT מודלים מבוססי LMU עדיין נופלים בדיוק סופי במשימות NLP. מודלים מבוססי LMU מצליחים להתחרות במודלים פשוטים כמו RNN ו- GRU, אך עדיין אינם משתווים לטרנספורמרים ברוב משימות השפה הטבעית.
ובכל זאת, מה שמייחד את LMU ובמיוחד את המימושים הנוירומורפים שלו, הוא לא רק הדיוק, אלה השילוב בין זיכרון טמפורלי חזק, חסכון בפרמטרים, ויכולת לרוץ על חומרות פשוטות ודלות במשאבים. כלומר מודלים מבוססי LMU עשויים להוות פתרון אידיאלי לעיבוד שפה טבעית בסביבות דלות אנרגיה כמו מכשירים ניידים, מערכות IoT ועוד.

6. אתגרים ומבט לעתיד

ההתקדמות בתחום של מודלים מבוססי LMU מהווה עדות ליכולת לשלב תובנות מתמטיות מדויקות עם דרישות יישומיות של יעילות חישובית, חיסכון בזיכרון ויכולת כללה של הכללה טובה. LMU, כארכיטקטורה המתוכננת מראש לייצג זיכרון טמפורלי באמצעות פתרון דיפרנציאלי של משוואות לינאריות, מציע חלופה פורצת דרך למודלים "רקורסיביים" קלאסיים ולטרנספורמרים עתירי משאבים. יתרה מזאת, ראינו כיצד התאמת LMU למודלים נוירומורפים, פותחת פתח ליישומים חדשניים במערכות קצה ובחומרות ביולוגיות שונות.

עם זאת, למרות ההבטחות וההישגים המרשימים, כפי שנראו במאמרים שהצגתי, עדיין ניצבים בפני הקהילה המחקרית מספר אתגרים עקרוניים. מגבלות תאורטיות, סוגיות בייצוב ההכשרה, פערי מימוש בין ייצוג המשוואות הרציפות לבין ההערכה הדיסקרטית שלהן, וקושי בשילוב מלא בחומרות נוירומורפיות קיימות, כל אלו מהווים חסמים בדרך להטעמה רחבת היקף של LMU ובפרט של גרסאותיו הנוירומורפיות.

בפרק זה אסקור כמה היבטים מרכזיים שמרכיבים את האתגרים של מודלים מבוססי LMU: מגבלות תאורטיות ויישומיות הקיימות במודל כיום. אתגרים הנדסיים ושיטתיים בשילוב LMU עם חומרה נוירומורפית, וכן כיווני המחקר גם כיווני מחקר עתידיים המסתמנים כהכרחיים כדי למצות את הפוטנציאל הגלום בגישות אלו. החל ממודלים אדפטיביים, דרך אינטגרציה עם מנגנוני Attention ועד להתאמה ליישומים רחבים בעיבוד שפה, ראייה ממוחשבת ועוד.

6.1 מגבלות מחקריות קיימות

מודל ה- LMU הציג קפיצת מדרגה ביכולת ייצוג זיכרון ארוך טווח ברשתות חוזרות בהשוואה למודלים מסורתיים כמו LSTM [1], שם הצליח ה- LMU ללכוד תלות רציפה בטווח של 100,000 צעדים. סדר גודל גבוה בהרבה מהמודל המסורתי, עם זאת קיימים עדיין מספר אתגרים תיאורטיים וביצועים.

6.1.1 קשיי אימון ורציפות בזמן

ה- LMU המקורי פועל כרשת RNN, ולכן הוא סובל מהמגבלות הקלאסיות של מודלים חוזרים – בהן דנתי בהרחבה בפרק [2.2.1] – ובראשן - אימון רציפים ארוכים. מודלים חוזרים חייבים להתעדכן בצורה סדרתית, כלומר צעד אחר צעד לאורך הרצף, מה שמוביל לרשת עמוקה מאוד כשפורשים אותה בזמן, ולשימוש ב- BPTT שמייצר בעיות דעיכה או התפוצצות גרדיאנטים. כתוצאה מכך, אימון RNN הופך לארוך, איטי ולא מקבילי על חומרת GPU מודרנית. גם ה- LMU, על אף מבנה הזיכרון הלינארי החסכוני שלו, סובל מאותה בעיה. הצורך באימון סדרתי מגביל את המודל, מקשה לאמן אותו על כמויות גדולות של נתונים או במשימות מורכבות בזמן סביר.
כדי להתגבר על מגבלות אלו, פותחה גרסה מקבילה של המודל FF-LMU [2], המבוססת על אותם משוואות דינמיות, אך בנויה כמבנה הזנה קדמית בלבד. גרסה זו אכן מאפשרת אימון מהיר ומקבילי, ומדגימה ביצועים מרשימים במגוון משימות, כולל NLP, תוך שימוש במספר מועט של פרמטרים. עם זאת FF-LMU מוותר על האופי הרקורסיבי של LMU, ולכן אינו שומר על מצב פנימי לאורך זמן. המשמעות היא שהמודל

מתאים יותר למשימות שבהם תלות הזמן היא מקומית או ניתנת לייצוג מראש. ופחות מתאים למשימות הדרושות זיכרון הסתגלותי מצטבר או שליפה תלוית הקשר לאורך זמן ארוך.

לכן, ניתן לומר שבעוד ש- FF-LMU מציע פתרון הנדסי לבעיית האימון הסדרתי, הוא אינו מחליף באופן מלא את היכולות של LMU המקורי. נשאר עדיין אתגר: איך לשמור על הזיכרון החכם והדינמי של LMU מבלי שהאימון יהיה איטי או מסובך, ובעיקר בלי לוותר על היכולת לאמן את המודל על כמויות גדולות של נתונים שמגיעים בזמן אמת.

6.1.2 מגבלות בזיכרון וביכולת הכללה

מודל ה LMU נבנה כך שהוא שומר מידע מהעבר בתוך חלון זמן קבוע – כלומר, הוא זוכר רק את מה שקורה בטווח זמן מסוים אחורה. לשם כך הוא משתמש בשיטה מתמטית שנקראת פולינומי לג׳נדר. מבחינה תיאורטית, זה מאפשר לו לייצג את המידע מהעבר בצורה מדויקת ויעילה כל עוד המידע נמצא בתוך אותו חלון. אבל כאן גם הבעיה, אם יש מידע חשוב שהתרחש לפני תחילת החלון המודל כבר לא זוכר אותו. בנוסף אין למודל דרך לדעת לבד איזה חלקים מהמידע חשובים ואילו לא, ואין לו מנגנון שמאפשר לו לבחור על מה להתמקד כמו שמודלים אחרים כן יודעים לעשות.
במחקרים חדשים, במיוחד על NLP, מצאו שזו נקודת חולשה של מודלים כמו LMU בהשוואה למודלים מבוססי טרנספורמרים. שכן למודלים כאלה יש מנגנון קשב שמאפשר להם לשים לב רק לחלקים החשובים של הטקסט, גם אם התרחשו לפני הרבה טוקנים. זה עוזר להם להבין קשר עמוק ולבצע בחירות חכמות בזמן העיבוד.

למעשה ניתן לראות כי חוסר היכולת של SSM קלאסיים כמו LMU לבצע בחירה מבוססת תוכן פוגע בביצועיהם. במאמרים מדגימים שניתן לשפר זאת ע״י הפיכת פרמטרי ה SSM במקרה שלנו המטריצות ב- LMU לתלויות קלט, אנחנו בעצם רוצים להפוך את החישובים של המודל לדינמיים. כך שהוא יוכל להחליט בכל רגע אם כדאי לו לזכור או לשכוח מידע מסוים, לפי הקלט הנוכחי. זה בעצם שער שמתאים את הזיכרון למה שצריך לזכור באותו רגע. לכן, ללא מנגנון שכזה, ה- LMU מתקשה להתמודד עם משימות שדורשות זיכרון מדויק לטווח ארוך.

אחד מהפתרונות הוא מודל בשם Mamba [6], שנבנה מתוך אותה משפחת מודלים מתמטיים שאליה שייך גם LMU. שלא כמו LMU, שבו הדרך לזכור מידע קבוע מראש, במודל זה החישוב הפנימי משתנה בכל רגע לפי הקלט שנכנס. המשמעות היא שהמודל יכול לבחור בעצמו איזה מידע לשמור ואיזה לשכוח, ממש כמו מנגנון זיכרון חכם שמתאים את עצמו לתוכן. בזכות זה הוא מצליח להתמודד טוב יותר עם משימות שדורשות זיכרון לטווח רחוק, כמו הבנת שפה.
אבל למרות היתרון הזה, Mamba לא פותר לגמרי את הבעיה. קודם כל הוא מודל שונה מ- LMU, הוא יותר כבד, מכיל הרבה יותר פרמטרים, ודורש משאבי חישוב גדולים יותר. כלומר, הוא לא מתאים כמו LMU למשימות שפועלות על מחשבים קטנים או מכשירי קצה. לכן, למרות שמודל כמו Mamba מציע כיוון מבטיח לשיפור הגמישות של הזיכרון, הוא עדיין לא מחליף באופן מלא את הפשטות, היציבות והיעילות של LMU. האתגר שנותר פתוח הוא איך לשלב בין היתרונות של שני העולמות, מצד אחד זיכרון אדפטיבי וחכם, מצד שני פשטות חישובית.

6.1.3 פערי ביצועים במשימות גדולות

למרות ש- LMU הוכיח שהוא יכול להיות מודל יעיל וחסכוני, בעיקר כשמדובר במספר הפרמטרים שהוא צריך, הוא עדיין לא הצליח להחליף את הטרנספורמר באף תחום גדול ומבוסס. למשל, במשימות קטנות יחסית כמו psMNIST [19], המודל הגיע לדיוק של כ98.5% תוך שימוש בפחות מ0.2 מיליון פרמטרים [2]. בזמן שמודלים כמו טרנספורמר צריכים פי עשרות אלפים יותר פרמטרים כדי להגיע לאותה רמה של דיוק. אבל כשעוברים למשימות גדולות יותר, כמו מודלים למשימות של עיבוד שפה טבעית בהיקף רחב, היתרון הזה מצטמצם. מודלים כמו טרנספורמר מצליחים הרבה יותר בזכות שני דברים עיקריים:
האחד, יכולת חישוב מקבילית מלאה, מתאים לאימון על GPU. והשני שימוש ב- Pre-training, כלומר הם נלמדים מראש על מיליארדי דוגמאות ורק לאחר מכן מותאמים למשימה ספציפית. זה מעניק להם יתרון אדיר בביצועים. מודלים מבוססי LMU, ביניהם LMUFormer [7], לא תומכים באימון מקדים שכזה. המחברים במאמר מודים שהמודל שלהם לא תומך באימון מקדים כמו הטרנספורמר, ולכן יש לאמן אותו מחדש על כל משימה. חיסרון זה מגביל של היכולת להשתמש בו באותן משימות גדולות כמו שמשתמשים במודלים הנפוצים כיום GPT ו- BERT.

לכן גם עם ההתקדמות שראינו, זו עדיין מגבלה מחקרית פתוחה: איך להפוך את מודל ל- LMU לכאלו שמתאים לא רק למשימות קטנות ויעילות, אלא גם לאימון עמוק ומורכב כמו שעושים בעולם של הטרנספורמרים.

6.2 אתגרים בשילוב LMU עם ייצוג וחומרה נוירומופיים

הניסיון לשלב את מודל ה- LMU בתוך מערכות נוירומורפיות, בין אם רמת הייצוג רשתות, ובין אם ברמת החומרה, מעלה מספר אתגרים מהותיים. LMU כזכור, מבוסס על חישובים לינאריים מדויקים שמתבצעים לאורך זמן באופן סדרתי. כאשר מנסים לממש את החישובים הללו באמצעות ספייקים או להריץ אותם על חומרות נוירומורפיות כמו Loihi או SpiNNaker [25], מתעוררות בעיות תאימות. קשה לתרגם את הדינמיקה הרציפה של LMU לרצף של אירועים דלילים, והמרה זו עלולה לפגוע ביציבות של המודל.

בנוסף, החומרה הנוירומורפית עצמה כוללת מגבלות פיזיות כמו מספר מוגבל של קישורים בין נוירונים. זמני תגובה לא אחידים, ורזולוציה מוגבלת של פרמטרים, מה שמקשה על שימור התכונות המתמטיות של LMU בתוך מערכת פיזית, יש trade-off בין דיוק לבין יעילות [3].

6.2.1 ייצוג חישוב לינארי באמצעות ספייקים

LMU מבוסס חישוב מתמטי רציף, בכל שלב הוא מבצע חישובים על הנתונים שהוא מקבל, בעזרת פעולות על המטריצות שהוא מגדיר. אבל כשמנסים להריץ את LMU על חומרה נוירומורפית כמו שבבים שמדמים את פעולת המוח יש קושי. כפי שתואר בפרק (2) החומרות לא פועלות בצורה רציפה, אלא רק כשמגיע מידע חשוב. הן עובדות עם ספייקים, ולא עם זרם רצף של מספרים. לכן צריך לתרגם את החישובים הרציפים שלו לרשת של נויורונים שיורים ספייקים.

על מנת לייצג תא זיכרון של LMU, השתמשו באוסף גדול של נוירונים (אוכלוסיית נוירונים) שכל אחד מהם תורם ערך קטן לחישוב, וביחד הם מדמים את הפעולה של תא זיכרון. כל ערך בודד שהמודל רוצה לזכור מיוצג על ידי קבוצת נוירונים ולא נוירון אחד. ככל שרוצים לייצג ערכים בתצורה מדויקת יותר צריך להשתמש ביותר נוירונים. לרוב צריך לכלול במודלים כאלה שלב שנקרא "שלב הקידוד" להמרת האות לפורמט מתאים, אך לא תמיד שלב זה הכרחי, במחקר שהציג את המודל L2MU [8] בנו את המודל כולו כרשת של נוירונים שיורים ספייקים, הייחוד הוא שכל חלק במודל הופך לאוכלוסייה של נוירונים. למודל יש יכולת לעבד נתוני חיישנים גולמיים ישירות, מבלי להזדקק לשלב מקדים של המרת הקלט לפורמט של ספייקים.

אבל גם עם כל הפתרונות האלה, נשארת בעיה מרכזית : דיוק. בשביל שהמערכת באמת תדמה את החישובים של המשוואות הדיפרנציאליות שמייצגות את LMU, צריך שהחומרה תוכל לייצג מספרים ברמת דיוק מאוד גבוהה. לכן צריך לפעמים להוסיף עוד ועוד נויורונים כדי לפצות במחסור בדיוק שהחומרה הנוירומורפית מציבה בפנינו, דבר שמעלה את הצריכה של משאבים. שכן ככל שמוסיפים יותר נויורונים, החישוב נעשה מדויק יותר, אבל גם כבד יותר מבחינת חומרה [3].

6.2.2 מגבלות קישוריות ותזמון בחומרה נוירומורפית

כשמנסים להריץ את מודל ה- LMU על חומרות נוירומורפיות כמו Loihi [26] או TrueNorth [25], מגלים כמה מגבלות מהותיות שקשורות לאופן שבו החומרה בנויה. כזכור ב- LMU, על מנת לייצג בצורה טובה יותר את המידע יש לדאוג שכמה שיותר יחידות זיכרון יהיו מחוברות אחת לשנייה. אבל בשבבים

נוירומורפים יש לכל נוירון מגבלה על מספר החיבורים האפשריים. דבר שמקשה על המימוש ודורש פיצול על פני מספר ליבות. בנוסף הפעולה הבסיסית ביותר של LMU כוללת מכפלת מטריצה, פעולה שקשה לממש אותה בצורה טבעית בחומרה מבוססת ספייקים. Loihi כמו שבבים נוירומורפים אחרים מבצע חישוב מבוסס אירועים שבו נוירונים סוכמים זרמים, לרוב ללא מכפלת מטריצות ולא מבצעים באופן טבעי פעולות לינאריות שלמות.

היום אין כמעט חומרות נוירומורפיות זמינות ונוחות לשימוש, השבבים שיש עדיין יחסית ניסיוניים, מסובכים לתכנות, ומוגבלים בכמות החיבורים והדיוק המספרי. לרוב קשה להפעיל עליהם מודלים מורכבים מבלי לבצע שינויים והתאמות קשות. לכן מה שעושים זה מימוש נוירומורפי על חומרה רגילה הרשת לא נהנית מהחיסכון האנרגטי שהתיאוריה מבטיחה אבל עדיין רעיון זה טומן ברובו כמה יתרונות:

יוצרים רשת של נוירונים מסוג LIF, משתמשים באותות מבוססים ספייקים, אבל מריצים הכל בתוך סימולציה על מחשב רגיל. המודל מתנהג כמו מוח אבל רץ על מחשב רגיל.

יש לגישה זו כמה יתרונות, כתוצאה מכך אפשר לבדוק רעיונות נוירומורפים בלי צורך בחומרה יקרה ומסובכת, להשוות ביצועים למול מודלים מבוססי ANN ולבצע מחקר תיאורטי ומעשי במקביל.

בפועל, ההוכחות הקונספט הקיימות מראות שאפשר להריץ LMU על שבב נוירומורפי [26]
יחד עם זאת ישנם גם פשרות, יש פה איזון עדין בין נאמנות המודל תיאורטי לבין אילוצי החומרה.

6.3 כיווני מחקר ופיתוח עתידיים

בעקבות ההצלחה והאתגרים של מודלים מבוססי LMU, צמחו כמה מגמות מחקר חדשניות לשיפור המודלים הללו ולהתאמתם טוב יותר מבחינת ביצועים מדעיים ומבחינת יעילות אנרגטית.

חוקרים מנסים כיום לשלב בין היתרונות של LMU לבין יכולות מתקדמות כמו קשב סלקטיבי, קידוד אדפטיבי או ארכיטקטורות היברידיות. בפרק זה אסקור רעיונות מרכזיים שמובילים את המחקר בתחום, ואשר עשויים לשפר את הדיוק, היעילות האנרגטית, והיכולת להתמודד עם רצפים ארוכים ונתונים בזמן אמת.

6.3.1 הוספת קשב וארכיטקטורות היברידיות

אחת הבעיות שדיברנו עליה קודם היא שמודלים כמו LMU קלאסיים לא יודעים להתמקד בתוכן החשוב במידע, כמו שעושה מנגנון הקשב (Attention) המצוי במודלים כמו טרנספורמרים. מודלים מבוססי LMU שומר זיכרון באופן כללי, אמנם בצורה יעילה ודחוסה, אך בלי יכולת לדעת מה חשוב ומה לא. בשביל לפתור את זה חוקרים התחילו לשלב בתוך מודלים כאלה יכולת של בחירה סלקטיבית. ראינו דוגמה חזקה לכך במאמר [6] שהציג את המודל Mamba. המודל ממשיך את קו המחשבה של מודלים כמו LMU, אבל מוסיף להם שכבה של "שיקול דעת" שמתבסס על הקלט. במילים אחרות במקום שהמטריצות יהיו קבועות הם משתנות בכל צעד לפי מה שקורה עכשיו. כך המודל יכול להחליט איפה להגביר את זרימת המידע לאורך הרצף ואיפה להחליש אותה, ממש כמו קשב, רק מבלי להשתמש במנגנון מפורש. החיסרון הוא שזה הופך את המודל לפחות פשוט, כבר אי אפשר לחשב הכל בעזרת קונבולוציות רגילות, וגם מבחינת יעילות ומורכבות המודל נהיה מסובך יותר. אך ללא ספק זו התחלה של כיוון חדש שצריך לחקור, זה מעלה בפנינו שאלות, כיצד ניתן לשלב מנגנונים אלה במודלים מבוססי LMU מבלי לפגוע ביעילות האנרגטית.

אחד הרעיונות הבולטים בתחום הוא ליצור מודלים היברידיים, כלומר ליצור מודלים שמשלבים בין שכבות שמוכרות לנו מ- LMU לבין שכבות קשב שמופעלות על טווח קצר. הגישה הזאת מאפשרת למודל לנצל את הזיכרון החזק והחסכוני של LMU כדי להבין הקשר רחב, אבל גם להציץ לפרטים הקטנים כשהוא צריך. הרציונל בגישה זו היא ששכבות הSSM (כמו Mamba ו- LMU) יטפלו בהקשר הרחוק באופן חסכוני, לינארי בזמן ואנרגיה. בעוד שכבות אחרות יהיו אחראיות לתשומת לב מקומית, כדי לטפל בפרטים חשובים בטווח הקצר. גישה זו בולטת במודל Samba שנחשף לאחרונה, מודל זה בונה ארכיטקטורה שכבתית שבה כל בלוק משלב שכבת Mamba ואחריה מנגנון Sliding Window Attention מקומי. כך הרצף נדחס למצב חבוי קומפקטי תוך שימור היכולת להיזכר בפרטים, באמצעות מנגנון הקשב בחלון האחרון.

ולגבי מודלים נוירומורפים מבוססי LMU, כאן האתגר אפילו יותר מורכב. כשאנחנו מנסים לממש מודלים כאלה על שבבים שמבוססים על ספייקים, קשה מאוד להכניס לתוכם מנגנוני קשב מורכבים, פשוט כי החומרה לא בנויה לזה. לכן חוקרים מנסים לבנות מנגנון קשב פנימי בתור הזיכרון עצמו, כלומר לגרום למנגנון הזיכרון להתנהג כאילו הוא בוחר מה חשוב, על בסיס זרימת הספייקים או על פי תכונות משתנות של נוירונים. המטרה היא להצליח לשלב בין עיבוד מבוסס אירועים לבין בחירה דינמית במידע, וכל זאת מבלי להכניס חישובים כבדים.
השילוב בין LMU לבין רכיבי SSM מתקדמים, או רכיבי Attention קלים, צפוי להפוך לגישה מרכזית בעיצוב מודלים חסכוניים שמתחרים בביצועים עם רשתות גדולות בהרבה. זה תחום שנמצא בהתפתחות מואצת, והיכולת לשלב את שני העולמות, זיכרון דינמי יעיל וקשב חכם, היא אולי המפתח לדור הבא של רשתות עמוקות.

6.3.2 גרסאות ספייקיות מתקדמות ומשולבות

אף שהושגו התקדמויות משמעותיות בפיתוח גרסאות מבוססות ספייקים של מודלים מבוססי LMU, התחום עדיין נמצא בחיתוליו ויש לא מעט שאלות פתוחות שדורשות מחקר המשך. אחת השאלות המרכזיות היא כיצד לשמר את המבנה המתמטי המדויק של LMU, את אורתוגונליות הזיכרון, גם כאשר ממשים אותו באמצעות נוירונים יורים ספייקים כמו LIF. אנחנו רוצים לגרום לרשת מבוססת ספייקים לייצג את ההתנהגות הרציפה של LMU, מבלי לאבד דיוק בגלל קוונטיזציה, רעש, או מגבלות בחומרה.

מבחינה יישומית, יש צורך לנתח את יכולת ההרחבה של מודלים מבוססי LMU בגרסאות הספייקיות שלהם. אנחנו רוצים לבנות מודלים יותר ויותר מסובכים שמצליחים להתמודד עם משימות שדורשות מורכבות, כמו מנגנוני Attention שהצגתי קודם לכן, ועדיין להצליח להריץ אותם על חומרה אמתית כמו Loihi.

רשתות מבוססות ספייקים מתאימות במיוחד למשימות שמערבות זרמי אירועים בזמן אמת, בזכות ההתנהגות הטבעית שלהן בתחום הזמן. לכן, יש משימות, כמו עיבוד אותות מסנסורים או זיהוי תבניות בזמן, שבהן ייתכן שמודל ספייקי יביא לביצועים טובים יותר מאשר רשת רגילה, גם ברמת הדיוק וגם מבחינת צריכת משאבים. ייתכן גם שהשימוש האופטימלי יהיה חלקי כלומר, שרק חלק מהמודל כמו שלב הקידוד או הזיכרון, יהיה מבוסס ספייקים, בעוד השלבים האחרים יישארו דטרמיניסטיים ורגילים.
לכן, אחד הכיוונים המרכזיים למחקר עתידי הוא לבדוק כיצד ניתן לשלב מודולים נוירומורפים ובפרט מודלים LMU נוירומורפים, כחלק ממערכות מורכבות ,ולא בהכרח כפתרון עצמאי. האם נכון למקם אותם

בתחילת הרשת לצורך מיצוי תכונות זמניות? או דווקא כשכבת זיכרון פנימית שמסננת מידע לאורך זמן? שאלות אלו טרם זכו למענה מספק, והן עשויות להשפיע בצורה מכרעת על היכולת לשלב את היתרונות הנוירומורפים של LMU במערכות בינה מלאכותית מבצעיות.

6.3.3 התאמה למשימות בעולם האמתי

אחד הכיוונים החשובים שעדיין לא נחקרו לעומק במודלים מבוססי LMU הוא היכולת להתאים את עצמם למשימות בעולם האמתי, משימות מחוץ לתנאי מעבדה. ברוב המחקרים שראינו נבחנים המודלים על סטים מלאכותיים, כמו psMNIST ו- ListOps ואפילו משימות NLP סטנדרטיות. אבל כשעוברים למשימות מורכבות יותר, מהעולם האמתי, כמו בינה לרכב אוטונומי, ניטור אירועים רפואיים מחיישנים (תחום עם פוטנציאל עצום למודלים מבוססי LMU שלא נחקר מספיק) ועוד, נוצרים אתגרים חדשים.

דוגמה לכך היא במערכות זמן אמת, בהם הנתונים אינם מגיעים מראש כקובץ סגור, אלא זורמים בקצב לא אחיד, עם רעש, תקלות, ודינמיקה משתנה. במצבים כאלה יש צורך במודלים בעלי חסינות לרעש, עם תגובה מהירה ויכולת להסתגל בזמן ריצה. דברים שלא קיימים במודל ה- LMU הקלאסי, אשר נבנה לעבודה על רצף חלק וקבוע בזמן.
אחד הרעיונות שכדאי לחקור הוא האם ניתן לכוונן את המודל כך שיתאים לקצב הדינמי של המערכת. למשל לגרום לו לשנות את תדירות הדגימה של הנתונים או מבנה הזיכרון שלו בהתאם למהירות השינוי בנתונים. אפשרות נוספת היא לאמן את המודל תוך כדי תהליך השימוש, כלומר לגרום לו להשתפר במהלך ההפעלה, אולי על ידי הוספת שכבות נוספות שיהוו כחלק האדפטיבי. או על ידי שילוב של למידה מתמשכת עם מנגנון זיכרון גמיש. בין השאלות הבולטות שעדיין נותרו בפנינו פתוחות, האם ניתן לבנות גרסה של LMU ששומרת על המבנה האורתוגונלי של הזיכרון ובו בזמן מסוגלת להסתגל לסביבה משתנה בזמן אמת? האם ניתן לשלב את המודל עם מנגנוני תיקון עצמי ? ואילו התאמות נוספות נדרשות כדי להפוך את המודל לרכיב בר שימוש במערכות זמן אמת מורכבות?
כיוון מחקר זה דורש חיבור עמוק לעולם ההנדסה המעשית ובפרט מערכות הפועלות בסביבה חיה, שבה התנאים משתנים כל הזמן. זהו תחום שעדיין לא נחקר מספיק אבל יש לו פוטנציאל עצום.

7. סיכום

עבודת גמר זו התמקדה בניתוח השוואתי של מודלים מבוססי LMU ובחינת יכולת השילוב שלהם בטכנולוגיות נוירומורפיות, תוך מטרה לזהות את יתרונותיהם וחסרונותיהם בעיבוד מידע טמפורלי ואת התאמתם למימוש בסביבות קצה חסכוניות בזמן ובאנרגיה.
במהלך העבודה נסקר תחום הזיכרון הרציף והצגנו את המסגרת התאורטית HiPPO אשר משמשת בסיס לגישה של LMU לשימור מידע היסטורי באמצעות היטל אורתוגונלי על בסיס פולינומי. לאחר מכן הוצגו עקרונות הפעולה של מודל LMU המקורי, תכונותיו, מגבלותיו, והשוואתו למודלים נשניים מסורתיים דוגמת LSTM ו- GRU.

לאחר מכן הוצגו מספר וריאציות מתקדמות של LMU בהן FF-LMU, LMUFormer, ו- SpikMamba.
אשר נועדו לשפר ביצועים, לאפשר חישוב מקבילי ולהתאים את הארכיטקטורה למימוש נוירומורפי.
כמו כן, נסקרו מודלים המבוססים על שילוב של LMU עם SNN, הבולט מבניהם היה L2MU אשר אפשר עבודה ישירה עם נתונים מחיישנים ללא צורך בקידוד מקדים.
העבודה שמה דגש מיוחד על שילוב המודלים בסביבות חישוב נוירומורפיות, בין אם בחומרה ייעודית ובין אם על התקני קצה, והציגה את האתגרים וההזדמנויות הגלומות בכך.
הדגמנו כיצד מודלים כמו L2MU ו- SpikMamba מסוגלים לשמר את היתרונות התאורטיים של LMU תוך שמירה על יעילות חישובית והתאמה למערכות זמן אמת.
על אף שמודלים מסוימים הדגימו ביצועים מדויקים ויציבים יותר מאחרים, קיימת מגבלה בהשוואת התוצאות בין המאמרים שנבחנו, וזאת עקב שימוש במערכי נתונים שונים, מטרות ניסוי שונות, וכן דגשים שונים במדדי ההערכה, כגון דיוק, מהירות עיבוד, כמות פרמטרים וצריכת אנרגיה.

יחד עם זאת, ניתן לומר בזהירות כי המודלים אשר שילבו את מבנה הזיכרון של LMU עם עקרונות נוירומורפים, הציגו פוטנציאל גבוה להתמודדות עם משימות מורכבות בזמן אמת, תוך שמירה על יעילות גבוהה ואפשרות למימוש מעשי.
בנימה אישית, עבודה זו תרמה רבות להבנתי את עקרונות הזיכרון הרציף, את היתרונות של מודלים אורתוגונליים כדוגמת LMU, ואת הפוטנציאל הטמון בשילובם עם רשתות נוירומורפיות לשם בניית מערכות חישוב חכמות, מהירות וחסכוניות.

9. ביבליוגרפיה:

[1] Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014, December 11). *Empirical evaluation of gated recurrent neural networks on sequence modeling*. arXiv.org. https://arxiv.org/abs/1412.3555

[2] Chilkuri, N. R., & Eliasmith, C. (n.d.). *Feedforward Legendre Memory Unit*. OpenReview. https://openreview.net/forum?id=qkJevgi2Fp3

[3] Voelker, A., Kajić, I., & Eliasmith, C. (2019). *Legendre Memory Units: Continuous-Time Representation in Recurrent Neural Networks*. https://proceedings.neurips.cc/paper/2019/file/952285b9b7e7a1be5aa7849f32ffff05-Paper.pdf

[4] Shi, X., Chen, Z., Wang, H., Yeung, D., Wong, W., & Woo, W. (2015, June 13). *Convolutional LSTM network: a machine learning approach for precipitation nowcasting*. arXiv.org. https://arxiv.org/abs/1506.04214

[5] Gu, A., Dao, T., Ermon, S., Rudra, A., & Re, C. (2020, August 17). *HiPPO: Recurrent Memory with Optimal Polynomial Projections*. arXiv.org. https://arxiv.org/abs/2008.07669

[6] Gu, A., & Dao, T. (2023, December 1). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv.org. https://arxiv.org/abs/2312.00752

[7] Liu, Z., Datta, G., Li, A., & Beerel, P. A. (2024, January 20). *LMUFormer: Low complexity yet powerful spiking model with Legendre memory units*. arXiv.org. https://arxiv.org/abs/2402.04882

[8] Fra, V., Leto, B., Pignata, A., Macii, E., & Urgese, G. (2024, July 4). *Natively neuromorphic LMU architecture for encoding-free SNN-based HAR on commercial edge devices*. arXiv.org. https://arxiv.org/abs/2407.04076

[9] Chen, J., Yang, Y., Deng, S., Teng, D., & Pan, L. (2024, October 22). *SpikMamba: When SNN meets Mamba in Event-based Human Action Recognition*. arXiv.org. https://arxiv.org/abs/2410.16746

[10] Gu, A. (2023). *Modeling sequences with structured state spaces*. Stanford Digital Repository. https://stacks.stanford.edu/file/druid:mb976vf9362/gu_dissertation-augmented.pdf

[11] Chen, X., Yajima, T., Inoue, I. H., & Iizuka, T. (2021). An ultra-compact leaky integrate-and-fire neuron with long and tunable time constant utilizing pseudo resistors for spiking neural networks. *Japanese Journal of Applied Physics*, *61*(SC), SC1051. https://doi.org/10.35848/1347-4065/ac43e4

[12] Yang, J., Wang, R., Ren, Y., Mao, J., Wang, Z., Zhou, Y., & Han, S. (2020). Neuromorphic engineering: From biological to Spike-Based hardware nervous systems. *Advanced Materials*, *32*(52). https://doi.org/10.1002/adma.202003610

[13] Pascanu, R., Mikolov, T., & Bengio, Y. (2012, November 21). *On the difficulty of training Recurrent Neural Networks*. arXiv.org. https://arxiv.org/abs/1211.5063

[14] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017, June 12). *Attention is all you need*. arXiv.org. https://arxiv.org/abs/1706.03762

[15] Ivanov, D., Chezhegov, A., Grunin, A., Kiselev, M., & Larionov, D. (2022, May 25). *Neuromorphic Artificial intelligence systems*. arXiv.org. https://arxiv.org/abs/2205.13037

[16] *Neurons | Organismal Biology*. (n.d.). https://organismalbio.biosci.gatech.edu/chemical-and-electrical-signals/neurons/

[17] Jiang, Y., Lu, S., & Sengupta, A. (2024, April 26). *Stochastic Spiking Neural Networks with First-to-Spike Coding*. arXiv.org. https://arxiv.org/abs/2404.17719

[18] Kim, Y., Park, H., Moitra, A., Bhattacharjee, A., Venkatesha, Y., & Panda, P. (2022, January 31). *Rate Coding or Direct Coding: Which One is Better for Accurate, Robust, and Energy-efficient Spiking Neural Networks?* arXiv.org. https://arxiv.org/abs/2202.03133

[19] Le, Q., V., Jaitly, N., & Hinton, G. E. (2015, April 3). *A simple way to initialize recurrent networks of rectified linear units*. arXiv.org. https://arxiv.org/abs/1504.00941

[20] Chilkuri, N., & Eliasmith, C. (2021, February 22). *Parallelizing Legendre Memory unit training*. arXiv.org. https://arxiv.org/abs/2102.11417

[21] *IMDB dataset of 50K movie reviews*. (2019b, March 9). Kaggle. https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews

[22] Sharma, L., Graesser, L., Nangia, N., & Evci, U. (2019b, July 1). Natural Language Understanding with the Quora Question Pairs Dataset. arXiv.org. https://arxiv.org/abs/1907.01041

[23] Bouanane, M. S., Cherifi, D., Chicca, E., & Khacef, L. (2022b, November 14). *Impact of spiking neurons leakages and network recurrences on event-based spatio-temporal pattern recognition*. arXiv.org. https://arxiv.org/abs/2211.07761

[24] *Papers with code*. (n.d.-b). GitHub. https://paperswithcode.com/sota/keyword-spotting-on-google-speech-commands-v2-3

[25] Young, A. R., Dean, M., Plank, J. S., & Rose, G. S. (2019). *A review of spiking Neuromorphic Hardware Communication Systems*. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8843969

[26] *Legendre Memory Units on Loihi — NengoExamples 22.1.22*. https://www.nengo.ai/nengo-examples/loihi/lmu.html

[27] Wikipedia contributors. (2025, June 24). *State-space representation*. Wikipedia. https://en.wikipedia.org/wiki/State-space_representation

[28] Chen, Y., You, Z., Zhang, S., Li, H., Li, Y., Wang, Y., & Tan, M. (2024, December 17). *Core context aware transformers for long context language modeling*. arXiv.org. https://arxiv.org/abs/2412.12465

[29] Payong, A. (n.d.). *Neuromorphic computing explained*. Baeldung on Computer Science. https://www.baeldung.com/cs/neuromorphic-computing-explained

10. נספחים:

Fourier Transform:

התמרת פורייה F היא טכניקה מתמטית שמשנה את האות מתחום הזמן לתחום התדר. במקום לייצג את הנתונים כערכים לאורך זמן x(t) פורייה מייצגת אותם כשילוב של תדרים שונים.
בתחום הזמן: הנתונים מתוארים כסדרה של ערכים.
בתחום התדר: הנתונים מתוארים כסדרה של תדרים עם העוצמה שלהם.

למה הכפל בתחום פורייה יעיל יותר?
בפורייה , פעולת קונבולוציה הופכת לכפל פשוט:
כזכור קונבולוציה היא פעולה מורכבת שמצריכה שילוב של כל ערך בסדרה אחת עם ערכים מסדרה אחרת. לאורך כל הסדרה. מתמטית קונבולוציה בין $h(t)$ ל- $u(t)$ מוגדרת כך:

$$f(t) = (h * g)(t) = \int_{-\infty}^{\infty} h(t-\tau)g(\tau)d\tau$$

פעולה זו דורשת חישובים רבים ומורכבות של $O(n^2)$ כאשר n הוא אורך הרצף.

הכפל בתחום התדר:
לפי משפט הקונבולוציה אם נעשה התמרת פורייה על שתי סדרות, הקונבולוציה שלהם בתחום הזמן הופכת לפעולת כפל פשוטה בתחום התדר:

$$\mathrm{F}\{h * g\} = \sqrt{2\pi} \cdot F\{h\} \cdot F\{g\}$$

כלומר, במקום לבצע אינטגרציה או חישוב סדרתי, פשוט מכפילים את הספקטרום של שתי הסדרות, פעולה שהיא הרבה יותר יעילה חישובית.

FFT נקרא גם התמרת פורייה מהירה, הוא אלגוריתם שמבצע את ההתמרה עם מורכבות של $O(nlogn)$ במקום $O(n^2)$.
כתוצאה מכך ביצוע קונבולוציה באמצעות התמרת פורייה ולאחר מכן כפל בתחום התדר ($O(n)$) והתמרה הפוכה חזרה לתחום הזמן, עדיין יעיל יותר מסתם קונבולוציה ישירה.

פעולה זו משתלמת במיוחד כשהאותות ארוכים, לעומת זאת אם $n \gg k$ השיטה הנאיבית עשויה להיות מהירה יותר.

Zero Order Hold – ZOH:

בהנחה שיש לנו קלט שמשתנה בזמן, במציאות הוא משתנה באופן רציף, אבל המחשב יכול לקרוא אותו רק כל שנייה. השיטה מניחה שהקלט לא משתנה עד הקריאה הבאה. זוהי החזקה מדרגה אפס, הקלט מוחזק קבוע בין צעדים.

במשוואות SSM:

המשוואה הרציפה הבסיסית ב- SSM נראית כך:

$$\frac{dz(t)}{dt} = \mathrm{B}x(t) + Az(t)$$

$z(t)$ – המצב הפנימי של המערכת.

$x(t)$ – הקלט בזמן t.

B,A – מטריצות שמגדירות איך המצב משתנה עם הזמן.

כדי לעבוד עם זה במודל שפועל שלב אחר שלב (בזמן בדיד), אנחנו צריכים לכתוב את זה בצורה שמתאימה לחישוב בכל צעד t:

$$\mathrm{Z}'[t] = AZ[t] + BX[t]$$

כדי לעבור ממשוואות הזמן הרציף לצורת הזמן הבדיד משתמשים בZOH והוא נותן לנו את החישוב של המקדמים החדשים $\bar{A}$ ו-$\bar{B}$:

$$\bar{A} = \exp(\Delta A)$$

$$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$$

Δ - משך הזמן בין שלב לשלב.

$\exp(\Delta A)$ – אם A מתאר איך משהו משתנה עם הזמן, אז ככה הוא ישתנה אחרי זמן של Δ שניות. לפיכך זה מתאר כמה מהמצב הקודם עובר הלאה.

$\bar{B}$ – מייצג כמה מהקלט הנוכחי משפיע על המצב החדש.

Abstract

Time dependent data processing and computation present a major challenge in the field of machine learning, primarily due to the need to retain information over long periods while maintaining computational efficiency. While old models such as LSTM and GRU have offered partial solutions, they suffer from limitations including gradual information loss, high computational overhead, and difficulties in capturing long range dependencies. At the same time, Transformer based models have become dominant across many domains. however, their substantial computational demands render them less suitable for real time systems and energy efficient hardware. To address these limitations, the Legendre Memory Unit (LMU) was developed as a lightweight and efficient alternative, specifically tailored for neuromorphic computing systems where energy conservation is critical.

This work analyzes the evolution of the LMU, its advantages compared to modern architectures, and its applications across diverse domains, with a particular emphasis on its neuromorphic aspects and its potential to serve as an alternative to Transformers. The findings demonstrate that the LMU excels in tasks requiring long term memory retention with low energy consumption, especially within spiking systems and neuromorphic hardware, and offer insights for its continued development and integration into advanced computing technologies.

Table of Contents

1. Introduction.....5
1.1 Objective and Importance of the Study.....5
2. Theoretical Background.....6
2.1 Time-Dependent Data Processing.....6
2.1.1 Structure of Recurrent Neural Networks (RNNs) and Their Core Limitations.....7
2.1.2 Prior Developments: ConvLSTM, GRU, LSTM.....8
2.2 Problem Statement and Motivation.....12
2.2.1 Computational Limitations of Recurrent Networks.....12
2.2.2 Transformers as a Partial Solution.....14
2.2.3 The Need for a New Computational Approach.....15
2.3 Theoretical Background on Neuromorphic Computing.....17
2.3.1 Foundations of Neuromorphic Computing and Biological Inspiration.....17
2.3.2 Event-Based Computation.....21
2.3.3 Differences Between Neuromorphic and Classical Computing.....22
3. LMU.....24
3.1 Overview of Models based LMU 24
3.1.1 Existing Computational Approaches.....24
3.1.2 Neuromorphic-Oriented Solutions.....26
3.2 LMU – Operational Principles and Implementation.....27
3.2.1 Basic Architecture.....27
3.2.2 HiPPO as a Theoretical Framework.....31
3.2.3 SSM – Dynamic Management of Sequential Data.....38
3.3 Model Enhancements Based on LMU.....42
3.3.1 Parallelized LMU Computation.....42
3.3.2 Mamba – Integration with SSM.....47
3.3.3 Comparison Between HiPPO, FF-LMU, and Mamba.....52
4. LMU in Neuromorphic Computing.....56
4.1 LMU Integration into Neuromorphic Architectures.....56
4.1.1 Adapting LMU to Spiking Architectures.....57
4.1.2 Advantages of LMU in Neuromorphic Contexts.....59
4.2 LMU Enhancements for Neuromorphic Environments.....61
4.2.1 L2MU: Native Implementation Using LIF Neurons.....61
4.2.2 LMUFormer.....67
4.2.3 SpikMamba.....73
5. Practical Applications of LMU.....79
5.1 Biomedical Signal Processing.....79
5.2 Human Activity Recognition (HAR).....80

5.3 IoT and Edge Computing Integration....80
5.4 Potential for Natural Language Processing (NLP)....81
6. Challenges and Future Outlook....82
6.1 Current Research Limitations....82
6.1.1 Training and Temporal Continuity Issues....82
6.1.2 Memory and Generalization Gaps....83
6.1.3 Performance in Large-Scale Tasks....84
6.2 Hardware and Representation Challenges....85
6.2.1 Linear Computation via Spikes....85
6.2.2 Connectivity and Timing in Neuromorphic Chips....85
6.3 Future Research Directions....86
6.3.1 Attention and Hybrid Architectures....86
6.3.2 Advanced Spiking Variants....87
6.3.3 Real-World Task Adaptation....88
7. Conclusion....89
8. Bibliography....90
9. Appendices....92
Abstract (english)....94
Table of Contents (english)....95

The Open University of Israel
Department of Mathematics and Computer sciences

A Comprehensive Review of LMU-Based Models and Their Integration into Advanced Neuromorphic Systems

Final Paper submitted as partial fulfillment of the requirements
towards an M.Sc degree in Computer Sciences
The Open Univerity of Israel
Department of Mathematics and Computer Sciences

By
Gal Bar Lev

Prepared under the supervision of Dr. Hadar Cohen Duwek

02 / 2025